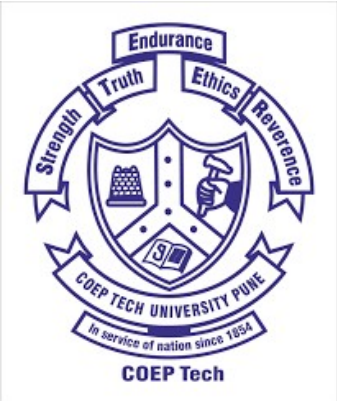
COEP Technological University

Department of Computer Science

IOT And Edge Computing

Virtual Labs Assignment



Submitted By: Bhavesh Gadling
MIS No: 612415061
TY BTech Computer Science and Engineering

Date:
October 31, 2026

Contents

CSE Experiments 2

1 Experiment 1: Introduction to Raspberry Pi 3

2 Experiment 2: Interfacing an LED with Raspberry Pi 7

3 Experiment 3: Raspberry Pi Push Button LED Blink Program 10

4 Experiment 4: Interfacing an LED with Raspberry Pi 14

5 Experiment 5: To interface an Ultrasonic sensor with Raspberry Pi 16

6 Experiment 6: Raspberry Pi Motor Control with Relay 18

7 Experiment 7: Raspberry Pi Bluetooth Data Transfer to Smartphone 22

8 Experiment 8: Raspberry Pi Pico-LED Blink Program Via Bluetooth and Smartphone 25

ENTC Experiments 28

9 Experiment 1: Interfacing Basic Input Components with Arduino 29

10 Experiment 2: Interfacing Basic Output Components with Arduino 35

11 Experiment 3: Interfacing Multiple Motors with Arduino 41

12 Experiment 4: Interfacing Multiple Sensors with Arduino 47

13 Experiment 5: Sending Live Sensor Data to an IoT Platform using HTTP 53

14 Experiment 6: Sending Live Sensor Data to an IoT Platform using MQTT 58

15 Experiment 7: Master–Slave Model using Arduino and I2C 63

16 Experiment 8: Interrelation between Cloud Platforms and ESP8266/ESP32 68

17 Experiment 9: IoT Dashboard for Remote Monitoring 73

18 Experiment 10: IoT Application using LoRaWAN and ESP8266/ESP32 78

CSE Experiments

IoT and Edge Computing – Virtual Labs

Experiment 1: Introduction to Raspberry Pi

Aim

The aim of this experiment is to gain an understanding of the Raspberry Pi, including its features, capabilities, and applications. Raspberry Pi is a versatile device that allows users to learn about programming, electronics, and computer systems.

Pre-Test

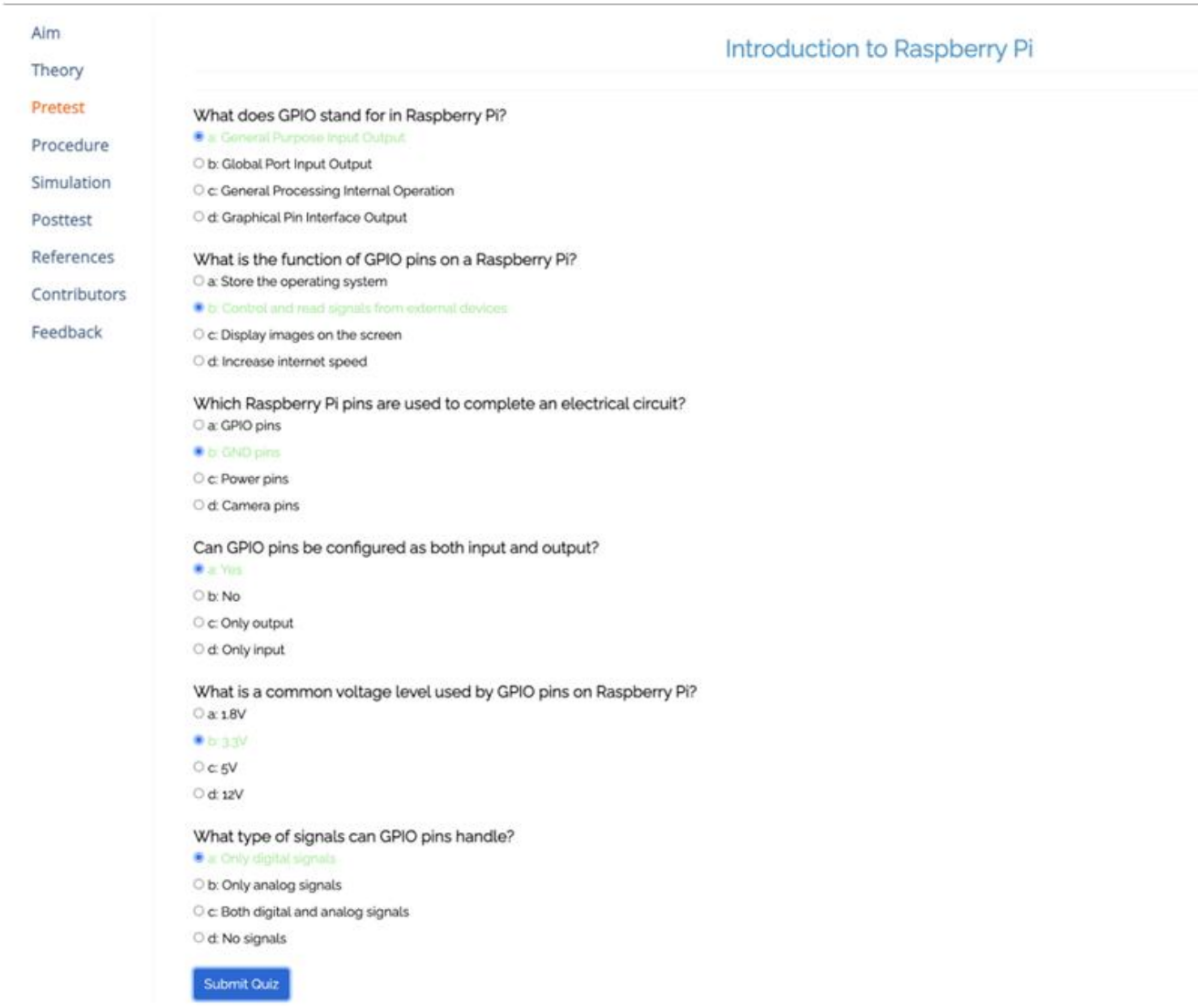


Figure 1: Pre-Test Screenshot

Experiment Performed on Virtual Lab

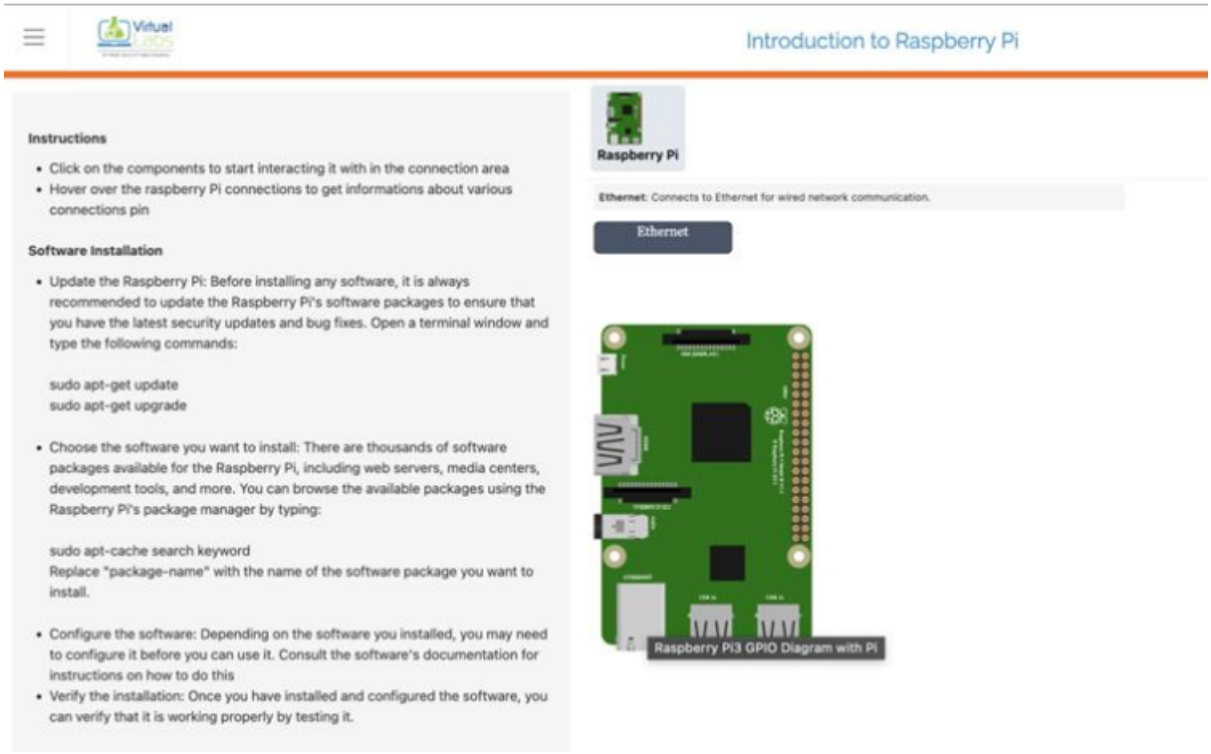


Figure 2: Rasbeerry Pi board

Post-Test

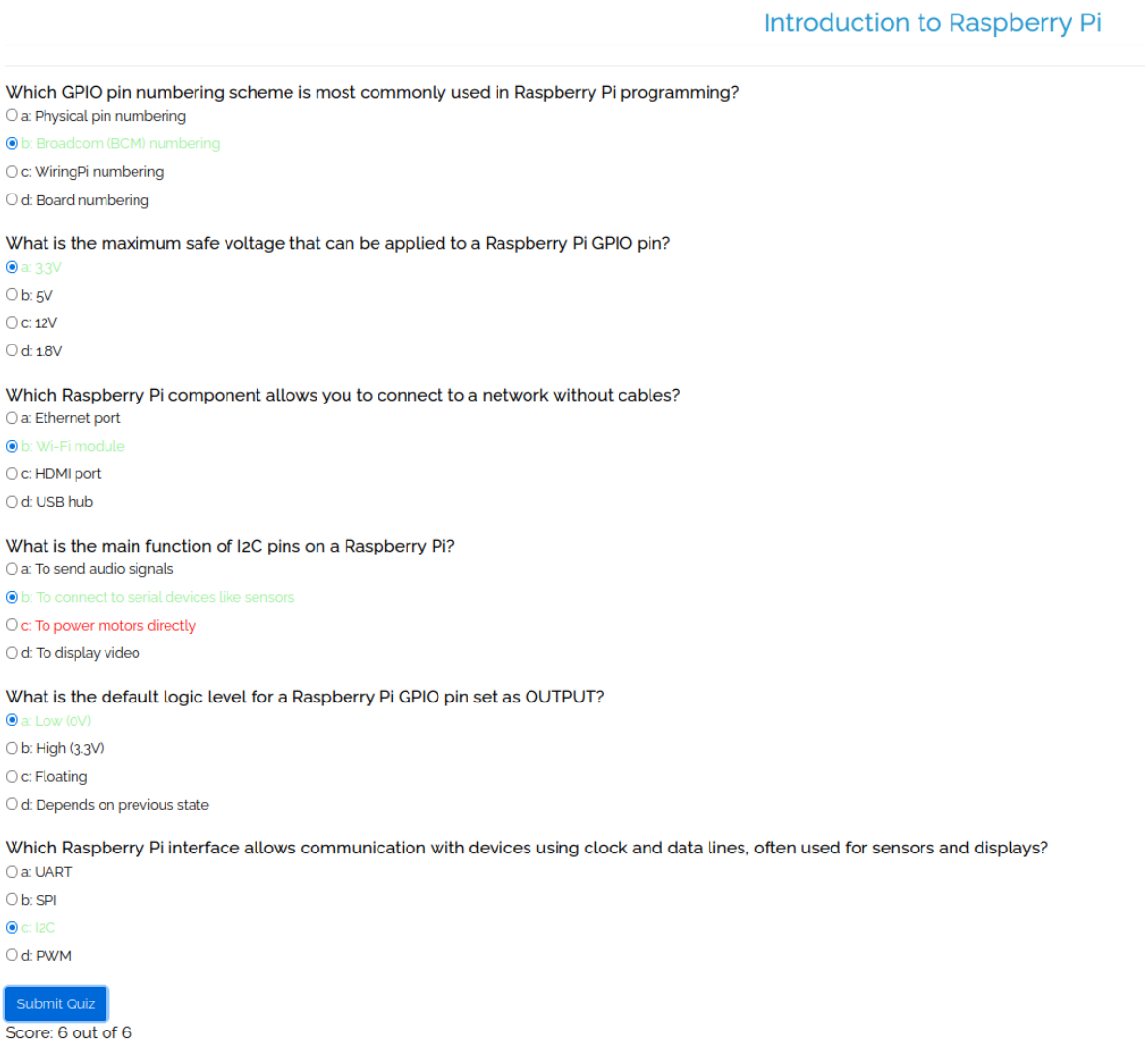


Figure 3: Post-Test Screenshot

Applications

- Home automation: Control devices like lights, fans, and appliances remotely using sensors and scripts. Media centers: Turn your Raspberry Pi into a smart media hub using software like Kodi or Plex.
- IoT systems: Connect sensors and devices to collect and share data over the internet.
- Robotics: Control motors and sensors to build intelligent robots for various tasks.
- Web servers: Host simple websites or web apps using lightweight server software.
- Network monitoring: Monitor traffic, block ads (e.g., with Pi-hole), or track device activity.
- Educational tools: Learn programming, Linux, and basic electronics in a hands-on way.
- AI experiments: Run lightweight machine learning models for speech, vision, or automation tasks. Basic security systems: Set up cameras, motion sensors, or facial recognition for home surveillance

Conclusion

In this experiment, we studied the Raspberry Pi using the Virtual Lab and became familiar with the Raspberry Pi 4 Model B, which is available with 2GB, 4GB or 8GB of RAM (2GB is sufficient for most educational purposes). By interacting with the components in the connection area and hovering over the GPIO pins, we learned how the board is laid out and what the different pins are used for. We also went through the software installation process needed to get the board running. The experiment gave a clear foundation in the hardware and setup of the Raspberry Pi, which we will use in later IoT experiments for interfacing sensors and building applications.

References

- A Review Paper on Raspberry Pi and its Applications
- Raspberry Pi – Official Website

Experiment 2: Interfacing an LED with Raspberry Pi

Aim

The objective of this experiment is to interface an LED with a Raspberry Pi and develop a Python program that controls the LED's state. Specifically, the program will turn the LED ON for a duration of 1 second, followed by turning it OFF for 2 seconds, repeating this cycle indefinitely. This task will help in understanding the basic functionality of the General Purpose Input/Output (GPIO) pins on the Raspberry Pi and how to control external hardware via programming.

Pre-Test

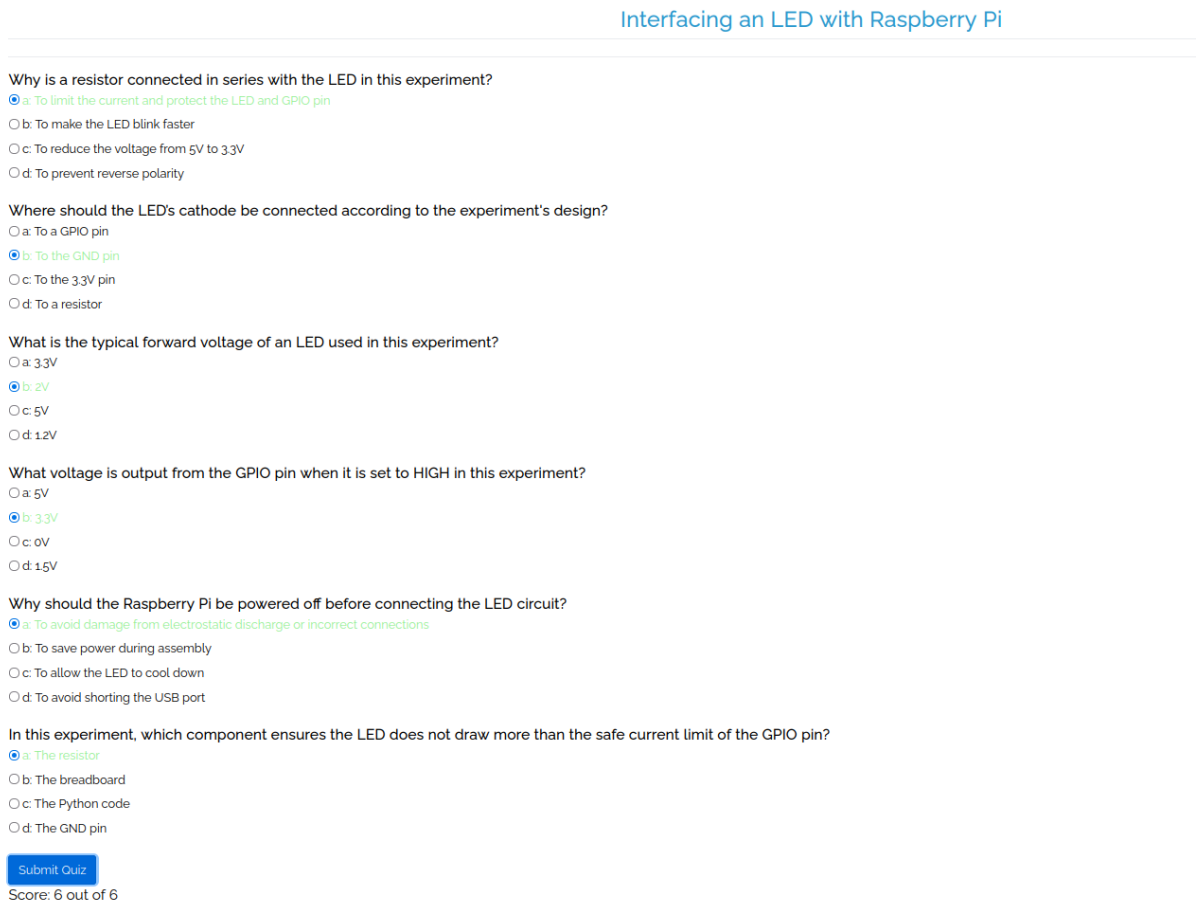


Figure 4: Pre-Test Screenshot

Experiment Performed on Virtual Lab

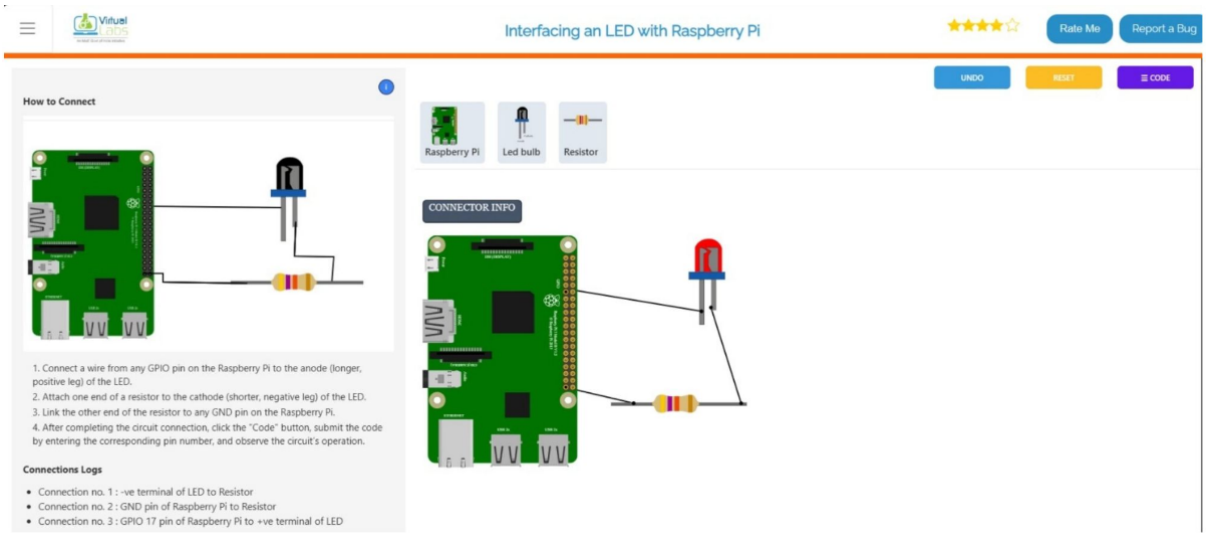


Figure 5: LED Setup

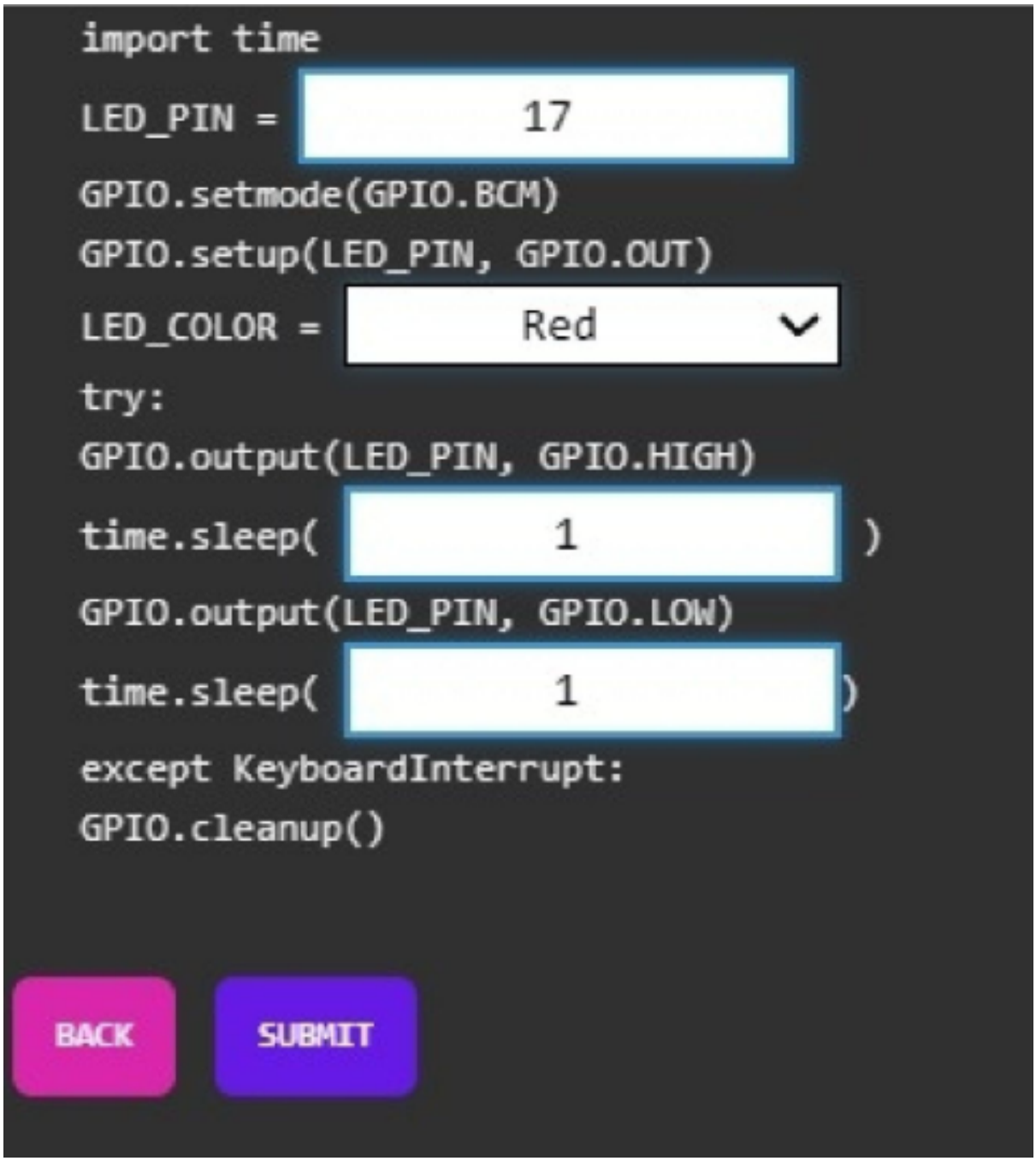


Figure 6: Python Program

Post-Test

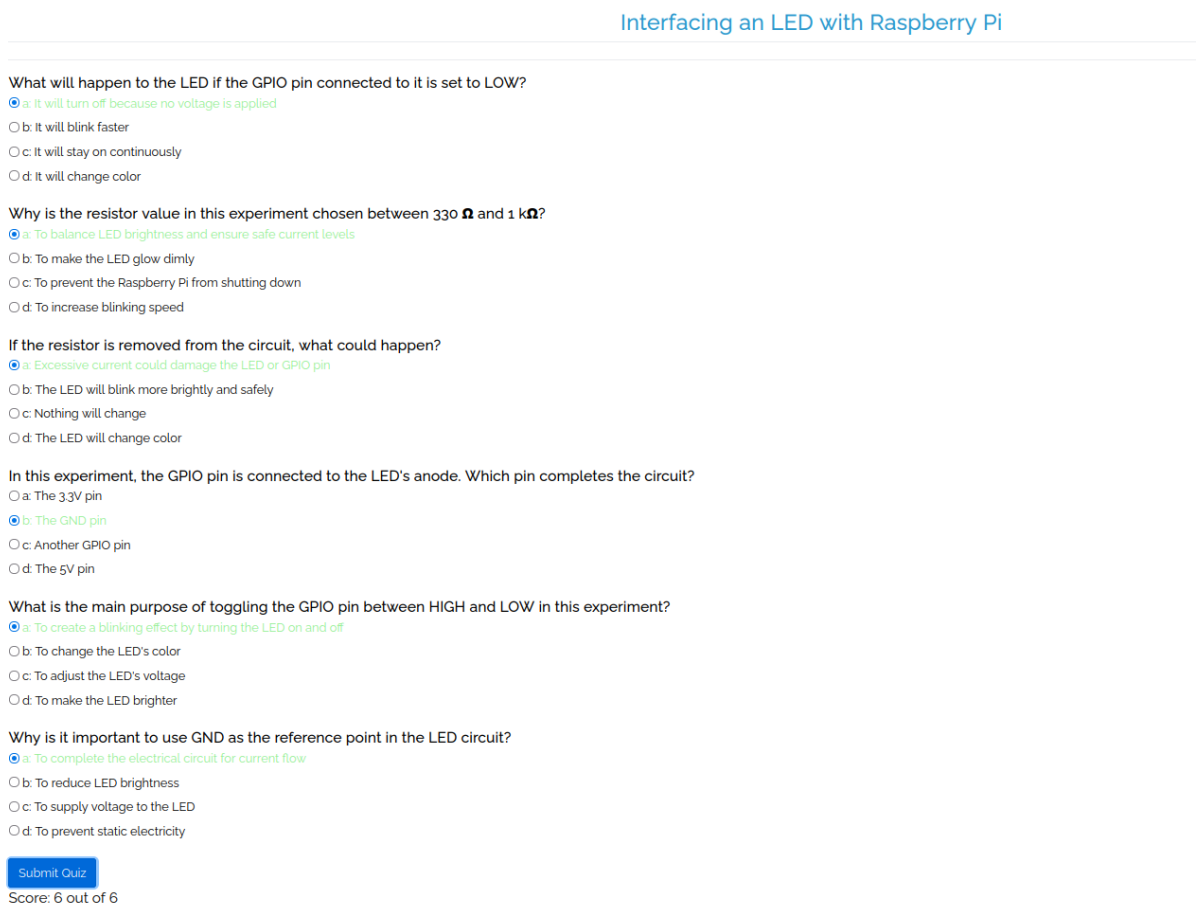


Figure 7: Post-Test Screenshot

Conclusion

In this experiment, we interfaced an LED with the Raspberry Pi and wrote a Python program that uses a GPIO pin to turn the LED ON for 1 second and OFF for 2 seconds, repeating indefinitely. This showed how GPIO pins can be set up as outputs to control external hardware from code. The same idea applies to status indicators, embedded systems, home automation and IoT projects.

Experiment 3: Raspberry Pi Push Button LED Blink Program

Aim

To interface a push button with a Raspberry Pi and develop a Python program to control an LED based on the button's state. The LED turns ON when the button is pressed.

Pre-Test

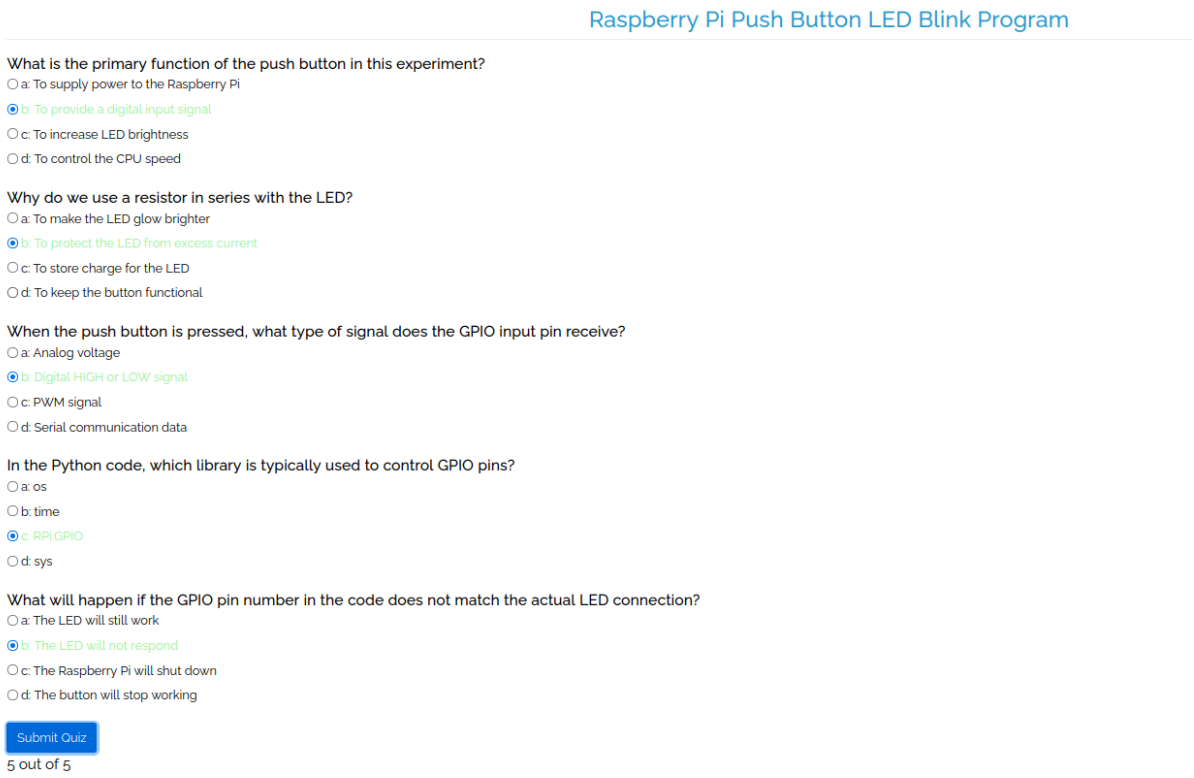


Figure 8: Pre-Test Screenshot

Experiment Performed on Virtual Lab

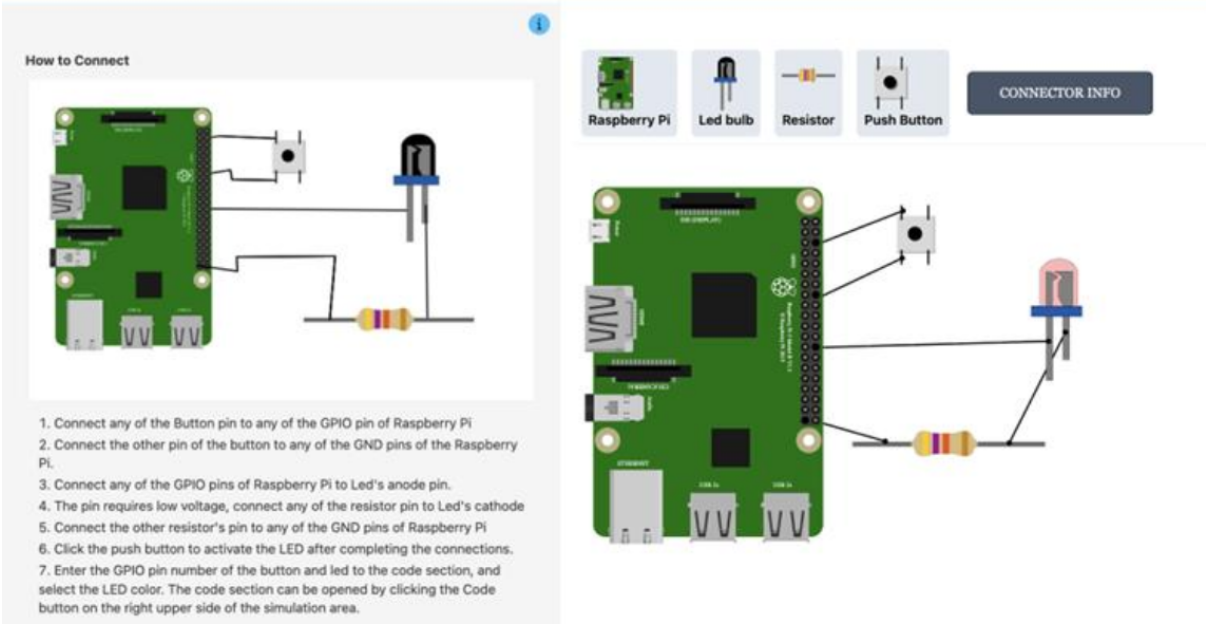


Figure 9: LED Setup

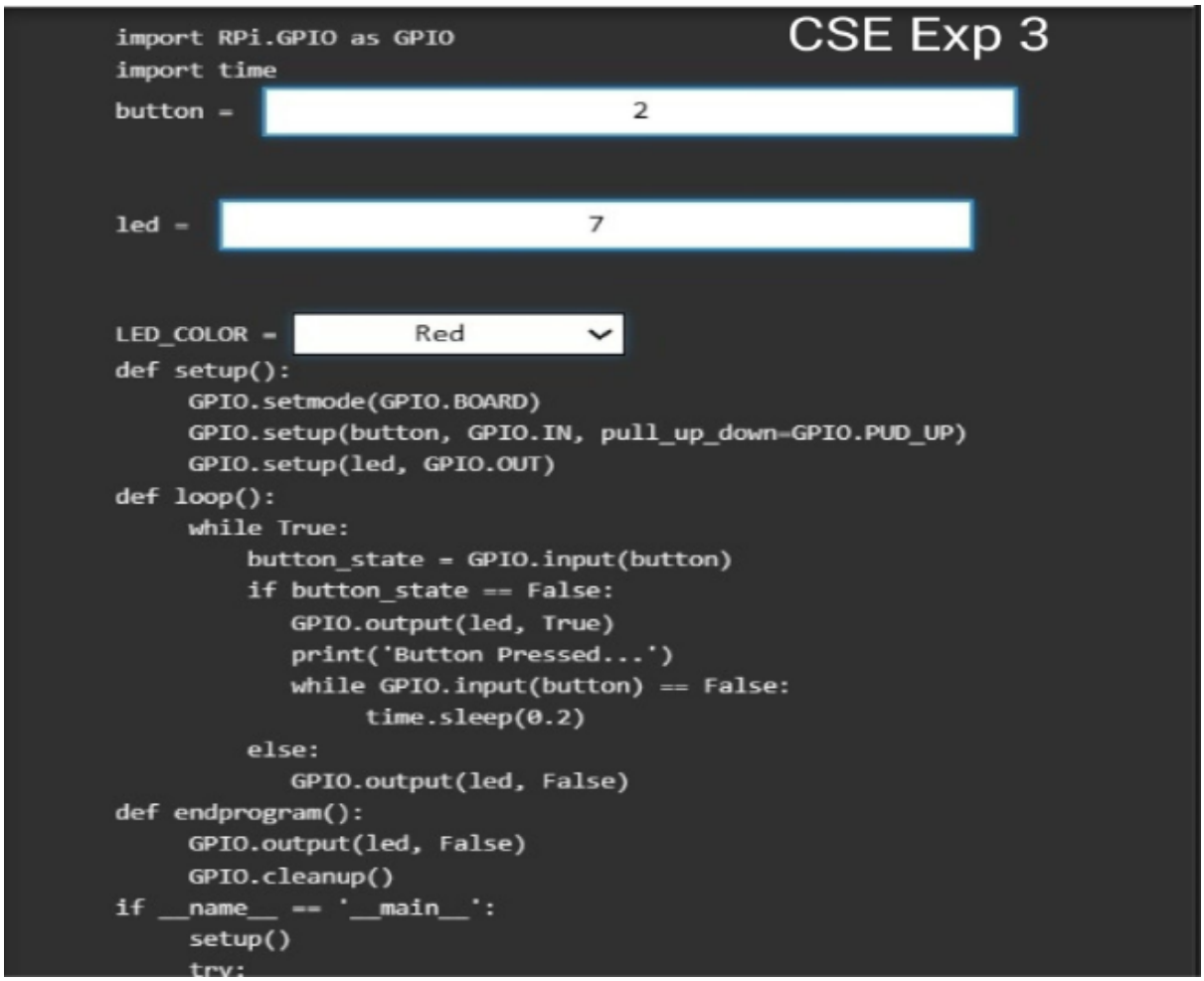


Figure 10: Python Program

Post-Test

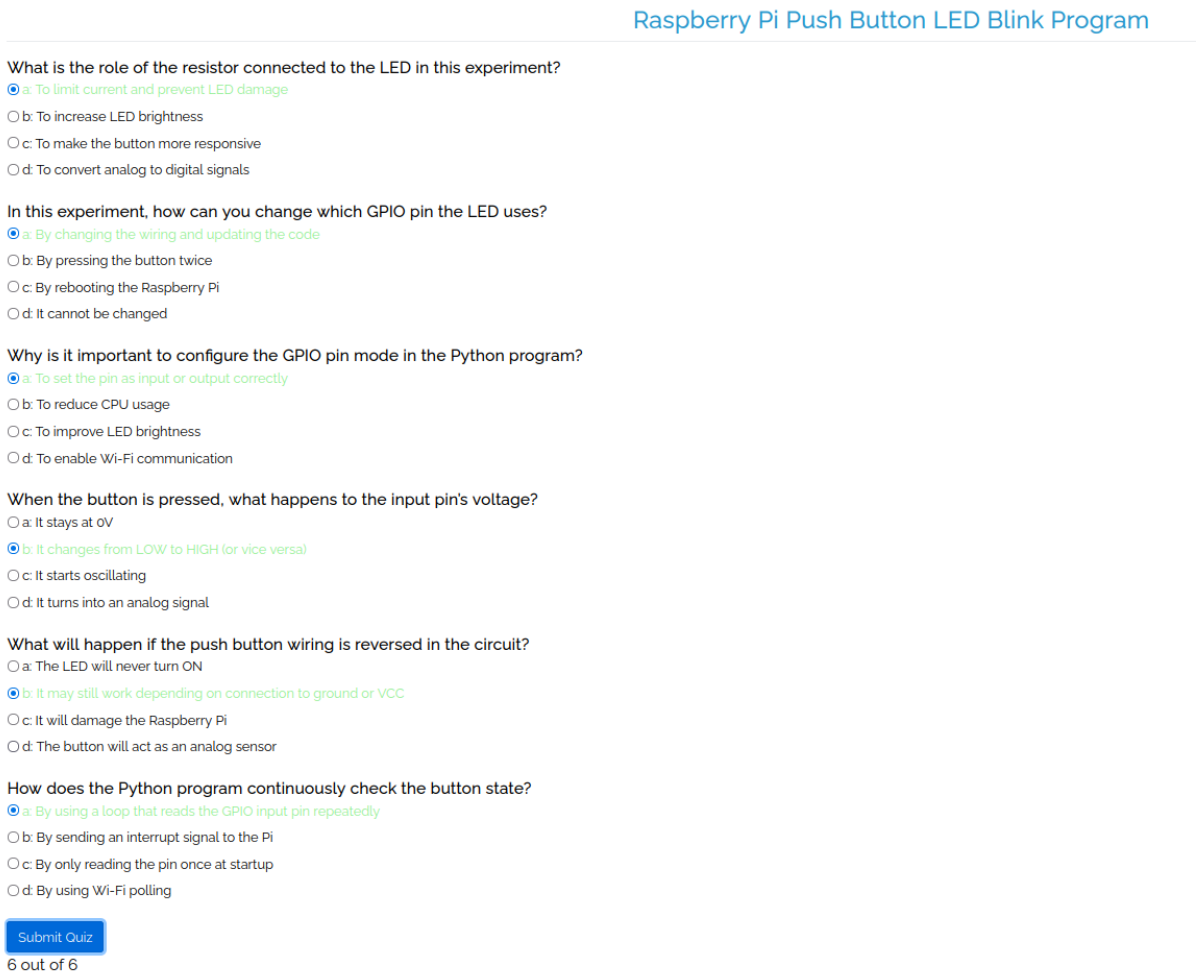


Figure 11: Post-Test Screenshot

Conclusion

In this experiment, we interfaced an LED with the Raspberry Pi and wrote a Python program that uses a GPIO pin to turn the LED ON for 1 second and OFF for 2 seconds, repeating indefinitely. This showed how GPIO pins can be set up as outputs to control external hardware from code. The same idea applies to status indicators, embedded systems, home automation and IoT projects.

Experiment 4: Interfacing an LED with Raspberry Pi

Aim

The objective of this experiment is to interface a DHT11 sensor with a Raspberry Pi and develop a Python program to read and display real-time temperature and humidity data. This experiment highlights the process of sensor integration with the Raspberry Pi using its GPIO (General-Purpose Input/Output) pins, demonstrating how embedded systems can interact with the physical environment.

Pre-Test

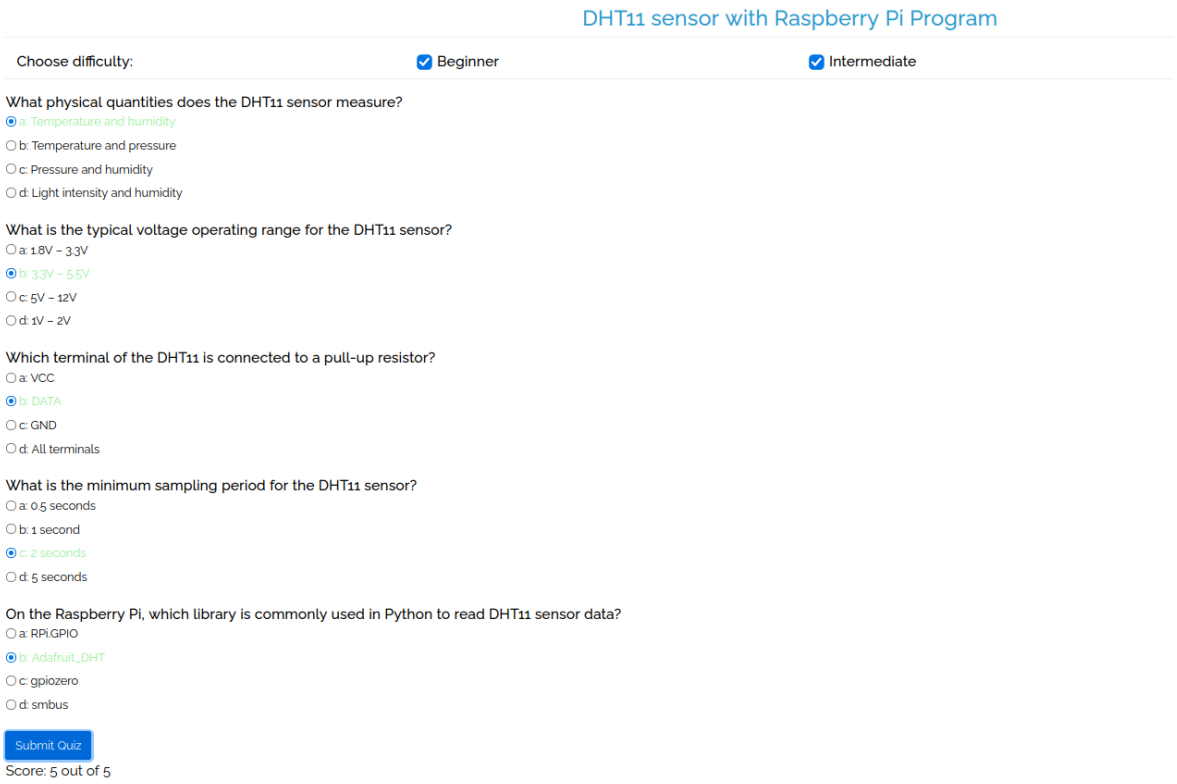


Figure 12: Pre-Test Screenshot

Experiment Performed on Virtual Lab

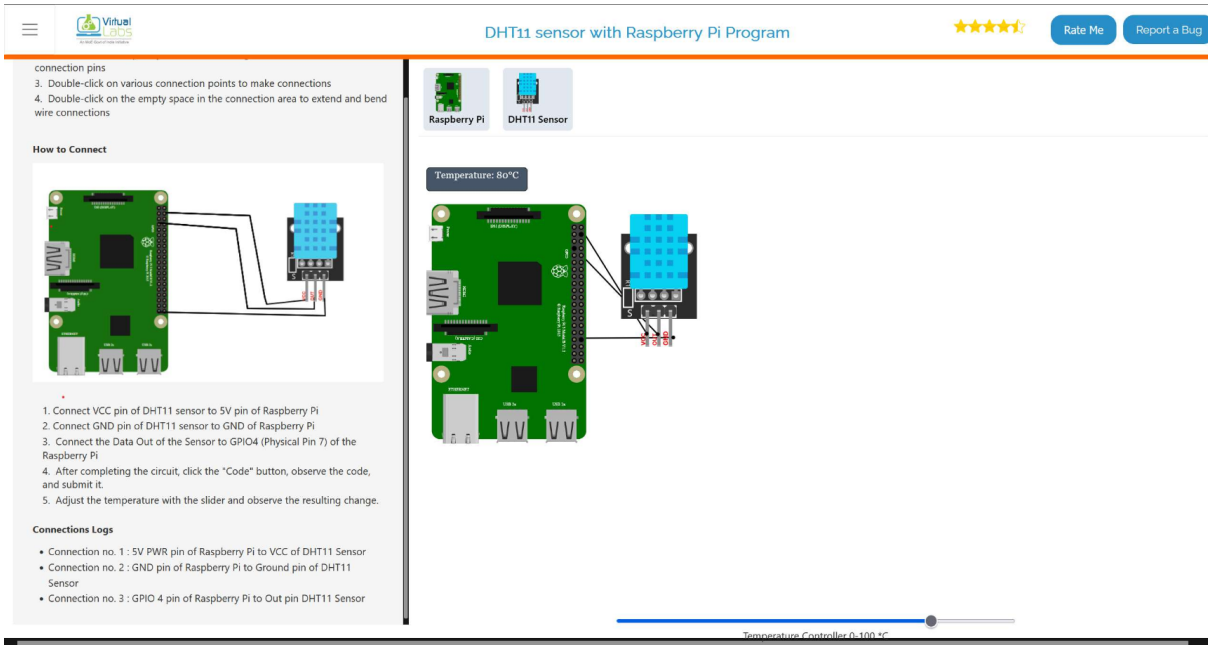


Figure 13: Raspberry pi Setup

Post-Test

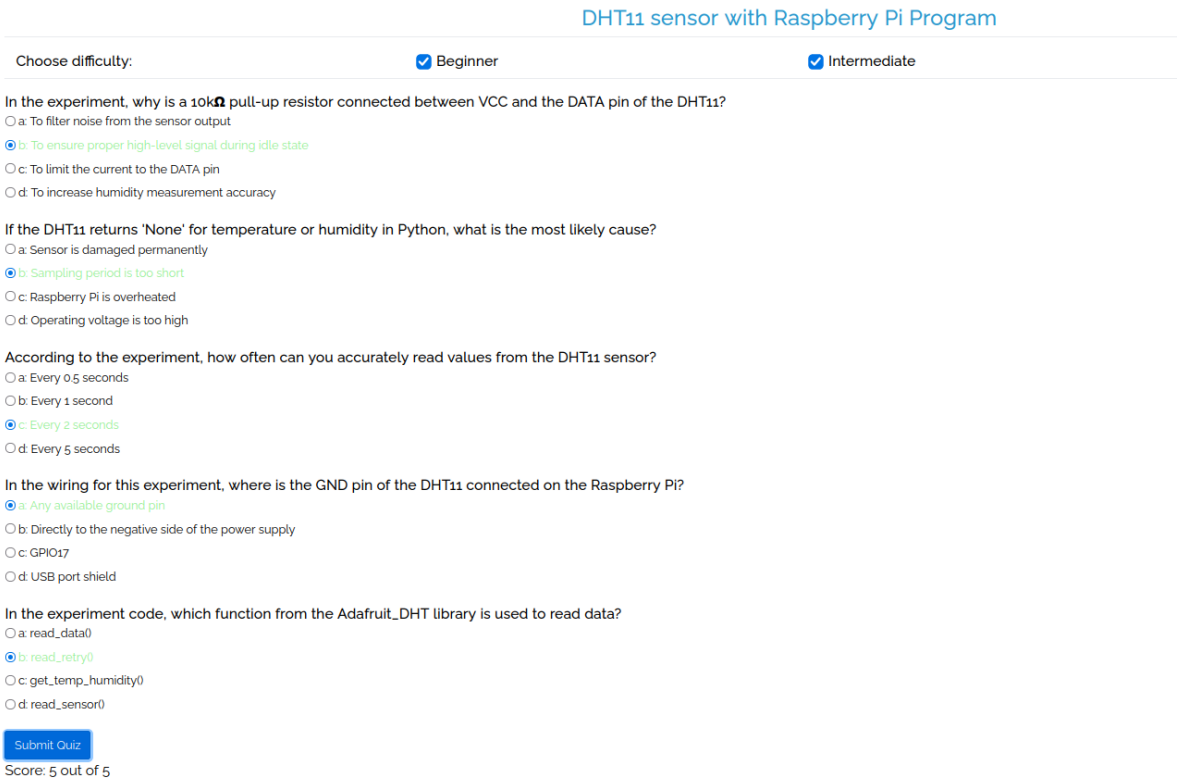


Figure 14: Post-Test Screenshot

Conclusion

In this experiment, we interfaced a DHT11 sensor with the Raspberry Pi through its GPIO pins and wrote a Python program that reads and displays real-time temperature and humidity values. This showed how an embedded system can sense its physical environment. The same setup is used in weather monitoring, smart agriculture (greenhouses) and home automation.

Experiment 5: To interface an Ultrasonic sensor with Raspberry Pi

Aim

The objective of this experiment is to interface an ultrasonic sensor (HC-SR04) with a Raspberry Pi and develop a Python program to measure distances with high accuracy. The experiment demonstrates how to utilize the Raspberry Pi's General-Purpose Input/Output (GPIO) pins for real-time distance sensing using ultrasonic waves.

Pre-Test

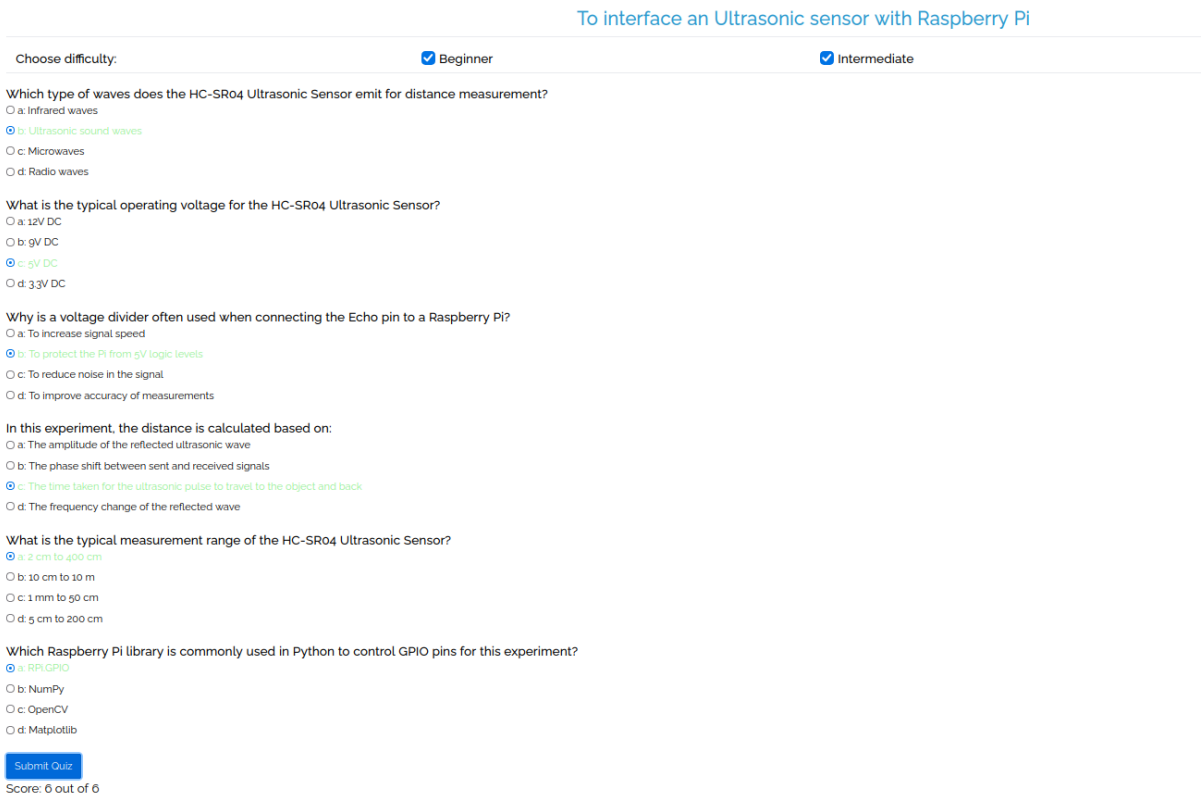


Figure 15: Pre-Test Screenshot

Experiment Performed on Virtual Lab

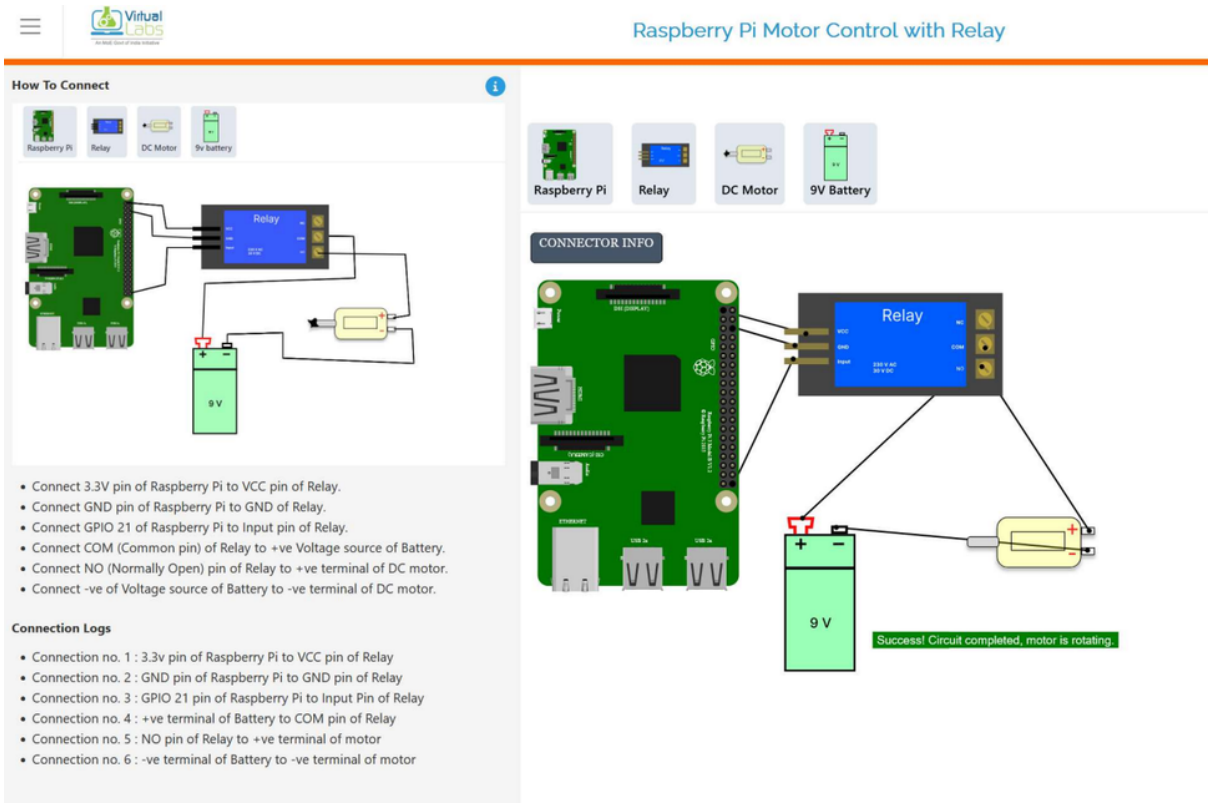


Figure 16: Raspberry pi Setup

Post-Test

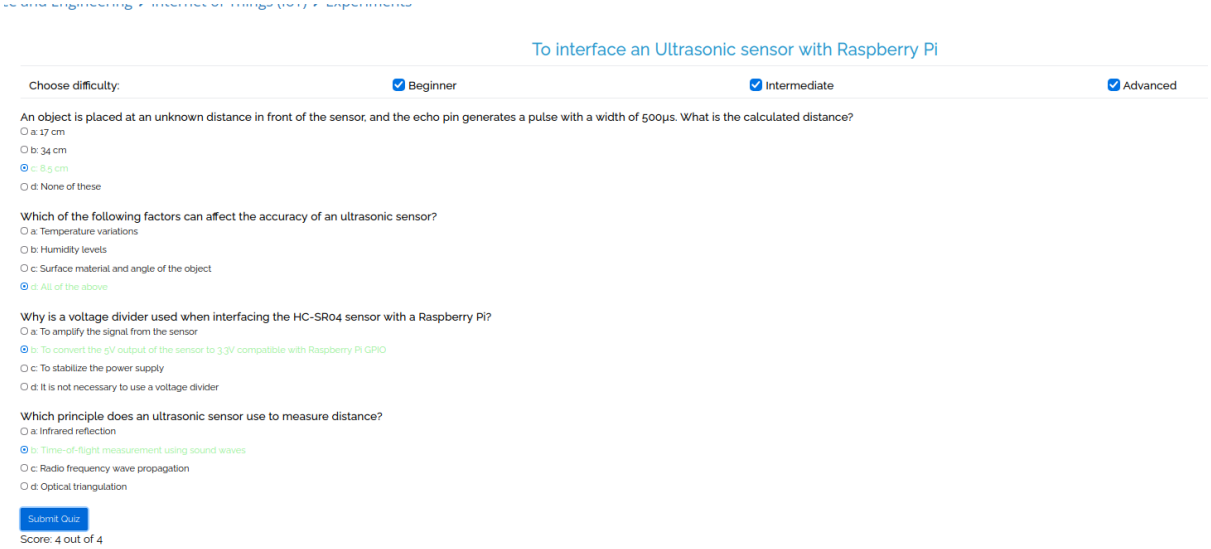


Figure 17: Post-Test Screenshot

Conclusion

In this experiment, we interfaced an HC-SR04 ultrasonic sensor with the Raspberry Pi through its GPIO pins and wrote a Python program that measures distance in real time using ultrasonic waves. This showed how a non-contact sensor can give accurate distance readings to an embedded system. The same setup is used in obstacle detection for robots, parking assist in vehicles, liquid level monitoring, security systems and smart home automation.

Experiment 6: Raspberry Pi Motor Control with Relay

Aim

The aim of this experiment is to interface a DC motor with a Raspberry Pi using a relay module, enabling control of the motor's operation through the General-Purpose Input/Output (GPIO) pins of the Raspberry Pi. This setup demonstrates the capability of the Raspberry Pi to control high-power electrical components indirectly through relays, using Python-based programming logic. The experiment also explores the safe switching of AC/DC loads using a low-power microcontroller.

Pre-Test

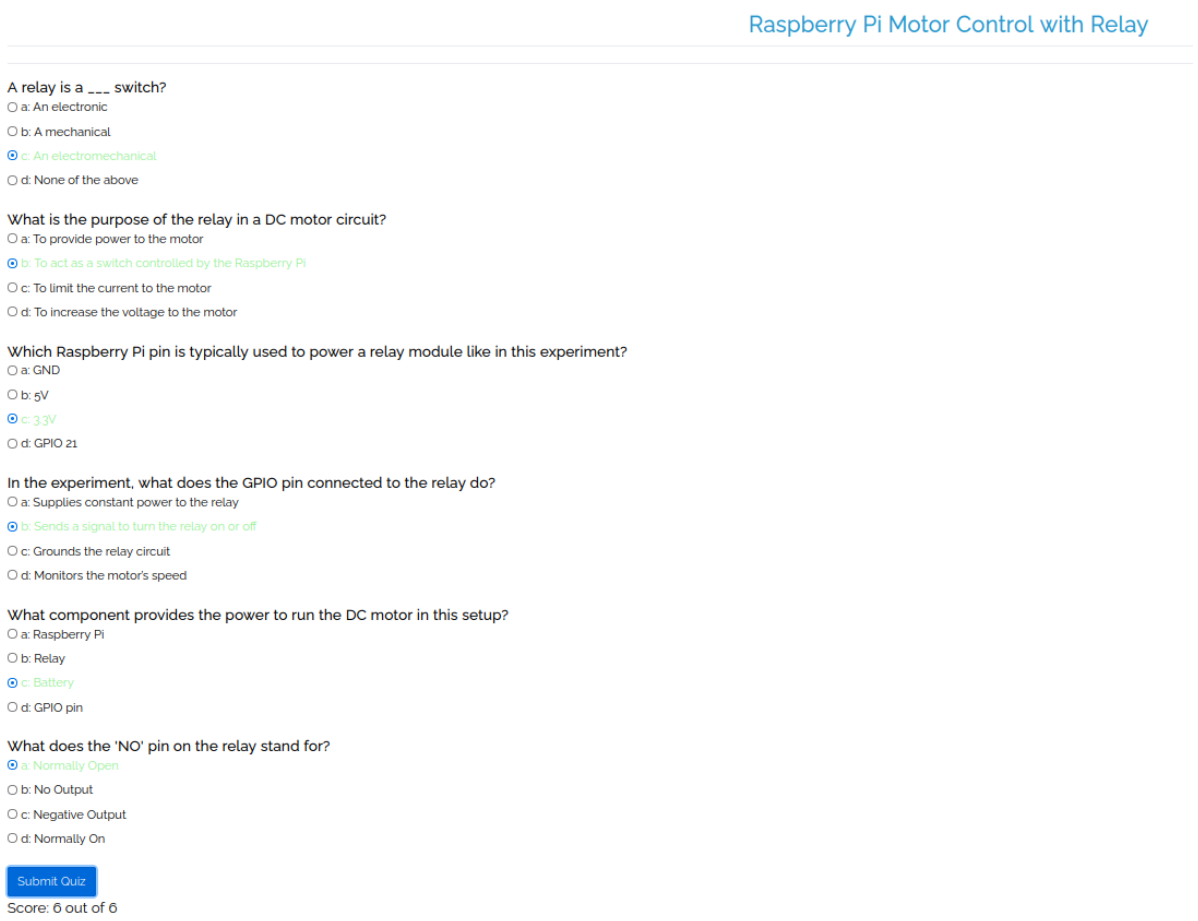


Figure 18: Pre-Test Screenshot

Experiment Performed on Virtual Lab

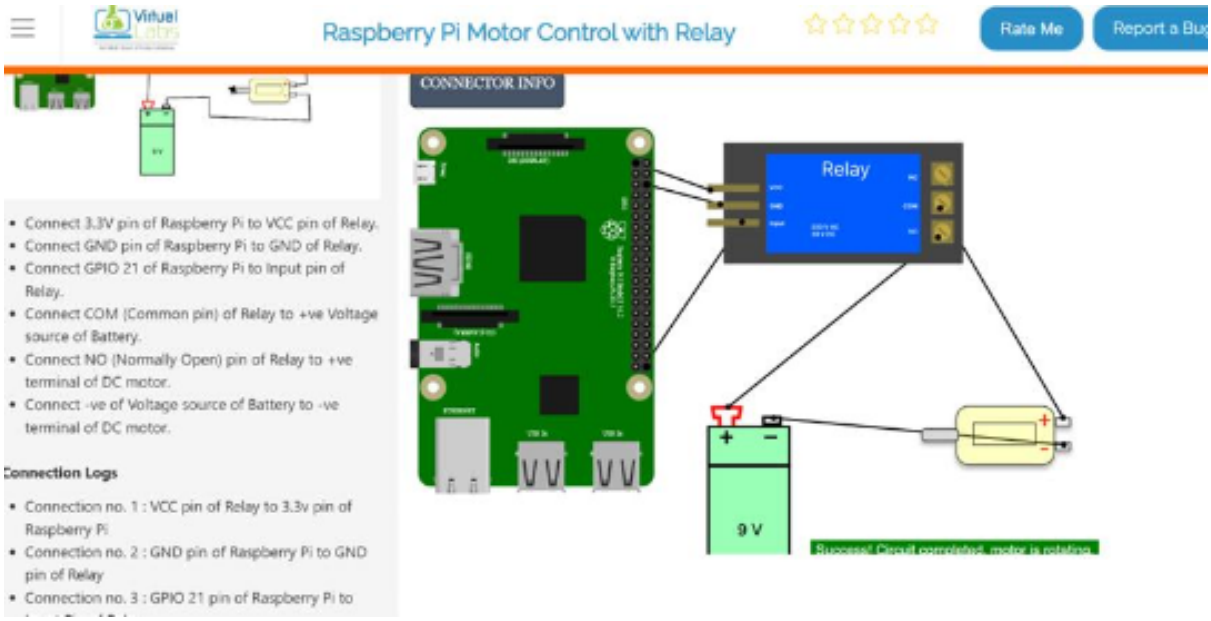


Figure 19: Raspberry pi Setup

```
# Set up GPIO
GPIO.setmode(GPIO.BCM)
relay_pin = 21
GPIO.setup(relay_pin, GPIO.OUT)

# Function to turn on the motor
def turn_on_motor():
    GPIO.output(relay_pin, GPIO.HIGH)
    print("Motor ON")

# Function to turn off the motor
def turn_off_motor():
    GPIO.output(relay_pin, GPIO.LOW)
    print("Motor OFF")

try:
    # Main loop
    while True:
        turn_on_motor()
        time.sleep(1)  # Run the motor
        turn_off_motor()
        time.sleep(2)  # Wait before turning on again
except KeyboardInterrupt:
    # Clean up GPIO on keyboard interrupt
    GPIO.cleanup()
```

BACK SUBMIT

Figure 20: Python Code

Post-Test

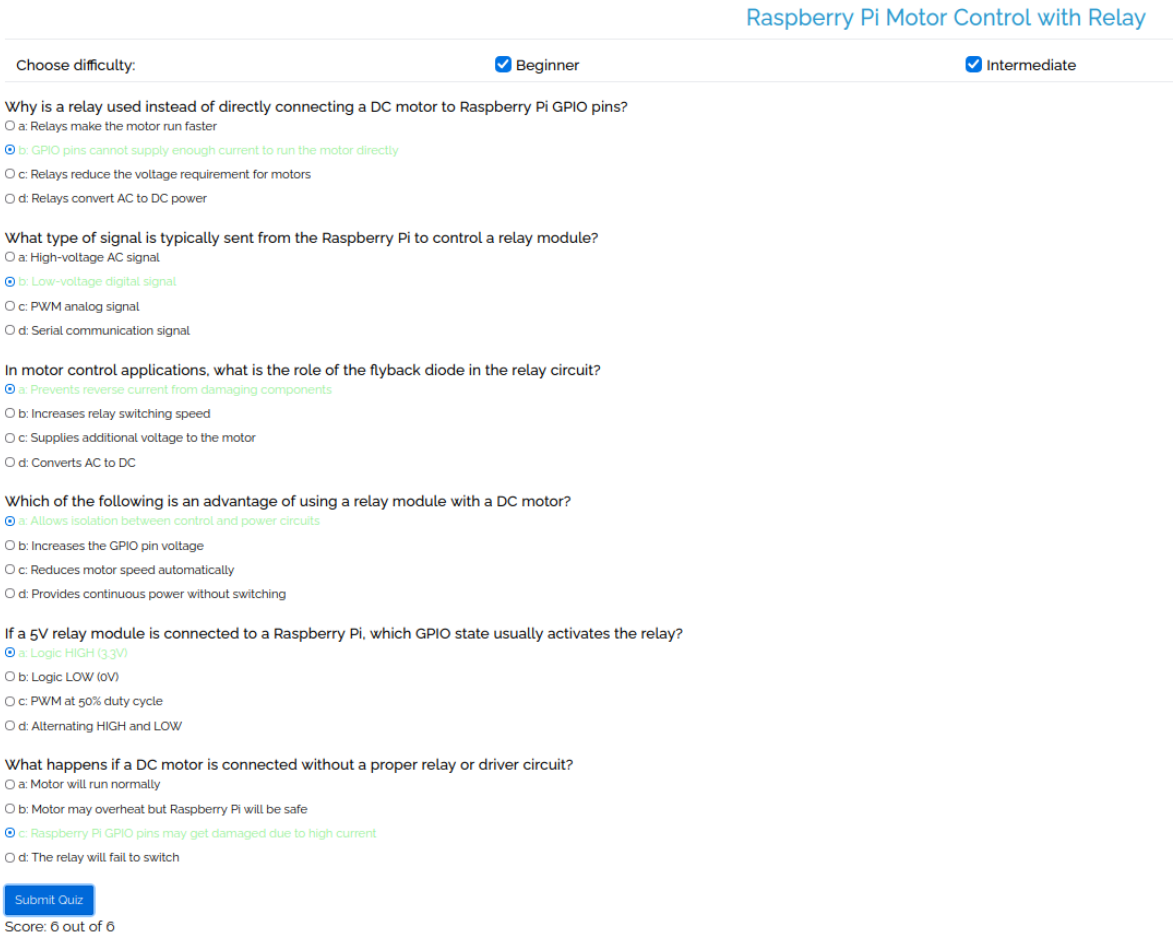


Figure 21: Post-Test Screenshot

Conclusion

In this experiment, we interfaced a DC motor with the Raspberry Pi using a relay module and controlled the motor through the GPIO pins with a Python program. This showed how a low-power board can safely switch high-power AC/DC loads indirectly through a relay. The same approach is used in home automation, industrial automation, smart agriculture (motorized pumps), IoT device control and robotics.

Experiment 7: Raspberry Pi Bluetooth Data Transfer to Smartphone

Aim

The primary objective of this experiment is to establish wireless communication between a Raspberry Pi and a smartphone using Bluetooth technology. This involves interfacing a Bluetooth module (such as HC-05/HC-06 or using the built-in Bluetooth of Raspberry Pi) with the Raspberry Pi, configuring the necessary software and hardware settings, and

implementing a program in Python to enable data transmission. The experiment demonstrates the capability of the Raspberry Pi to function as a Bluetooth server or client, thereby facilitating the exchange of data with other Bluetooth-enabled devices, particularly smartphones. This forms a foundational step in building wireless IoT systems, smart automation, and remote control applications.

Pre-Test

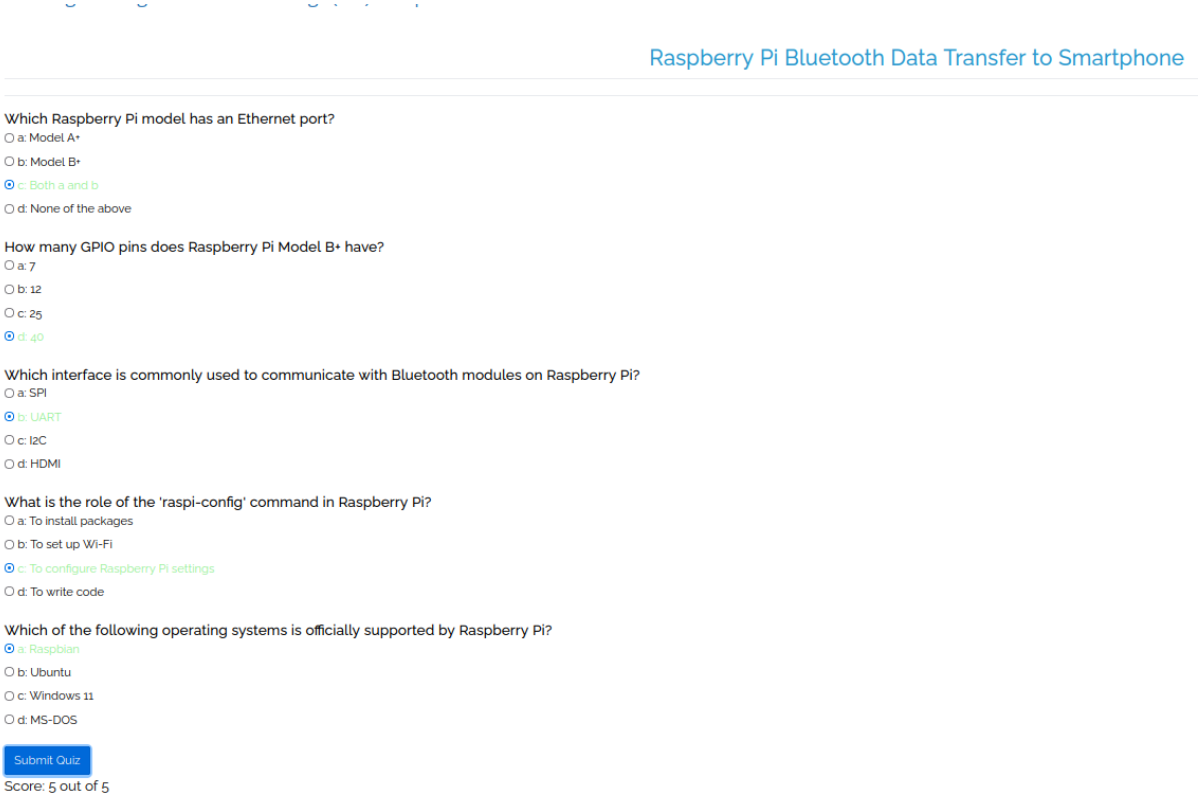


Figure 22: Pre-Test Screenshot

Experiment Performed on Virtual Lab

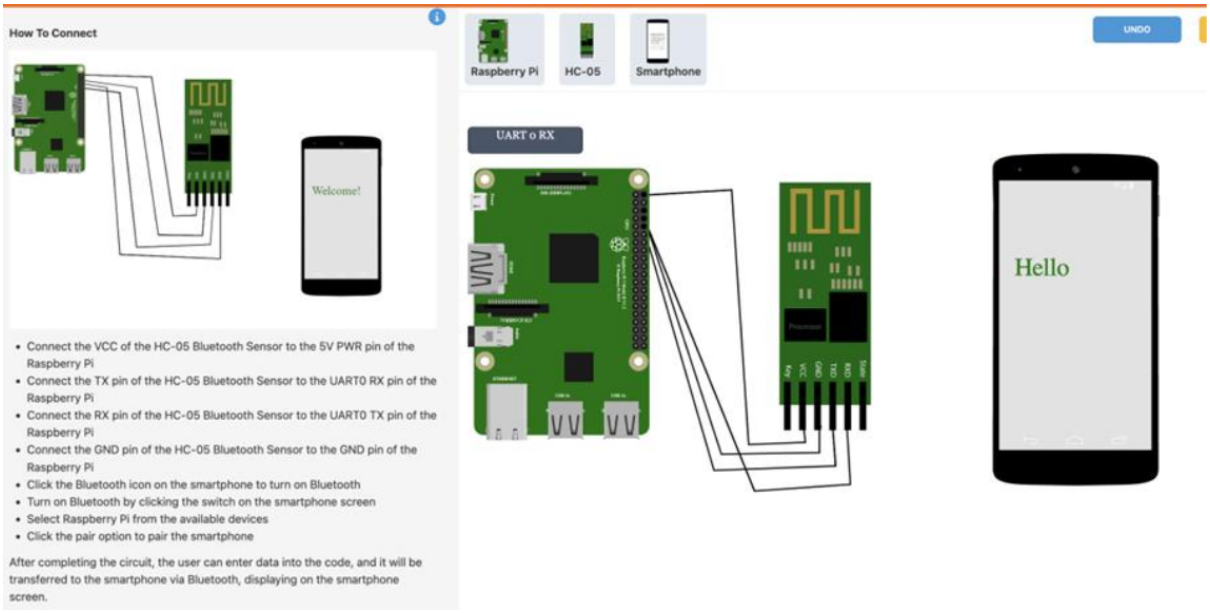


Figure 23: Raspberry pi Setup

Post-Test

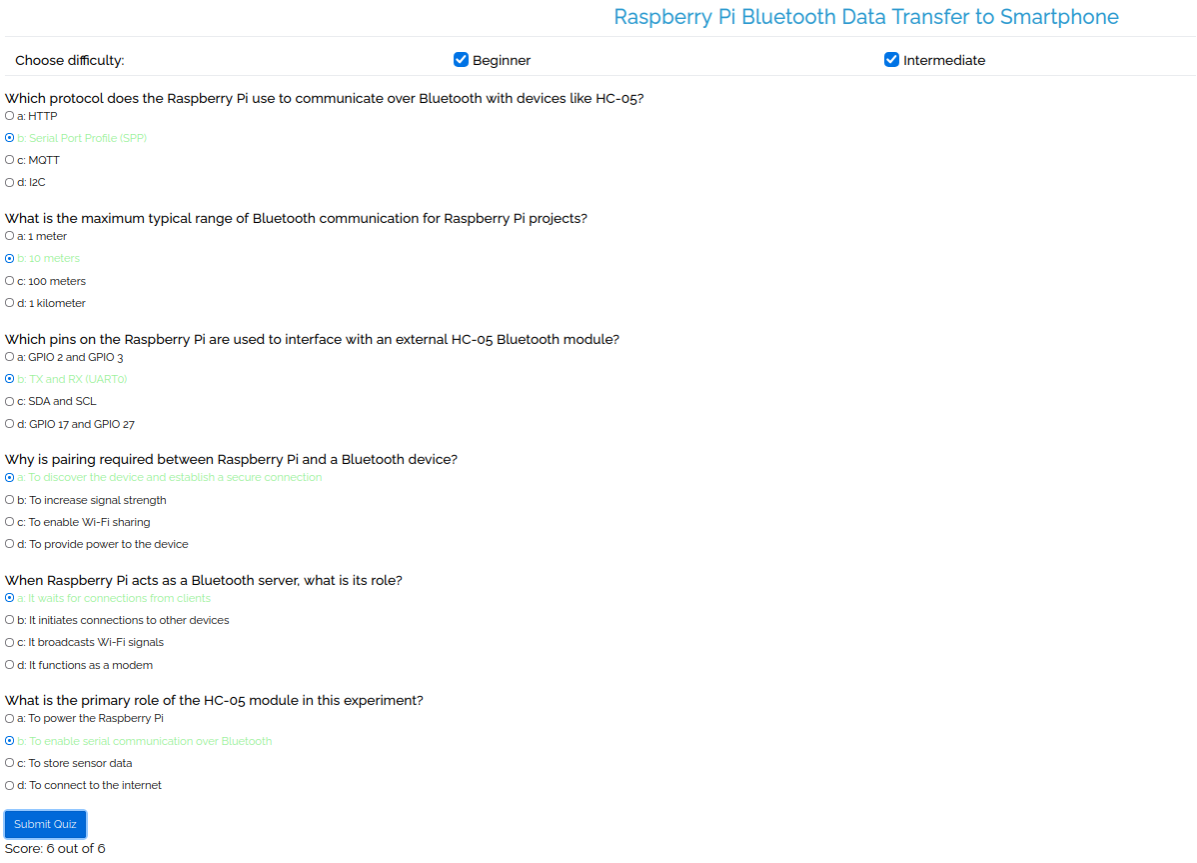


Figure 24: Post-Test Screenshot

Conclusion

In this experiment, we set up Bluetooth communication between the Raspberry Pi and a smartphone, using a Bluetooth module (HC-05/HC-06 or the Pi's built-in Bluetooth) and a Python program to send and receive data. This showed that the Raspberry Pi can work as a Bluetooth server or client and exchange data wirelessly with other Bluetooth devices. The same setup is used in wireless data logging, home automation, healthcare monitoring and IoT prototyping.

Experiment 8: Raspberry Pi Pico-LED Blink Program Via Bluetooth and Smartphone

Aim

To establish Bluetooth communication between a Raspberry Pi Pico and a smartphone, enabling remote control of an LED. The system should allow the LED to turn ON upon receiving a '1' signal and turn OFF upon receiving a '0' signal from the smartphone.

Pre-Test

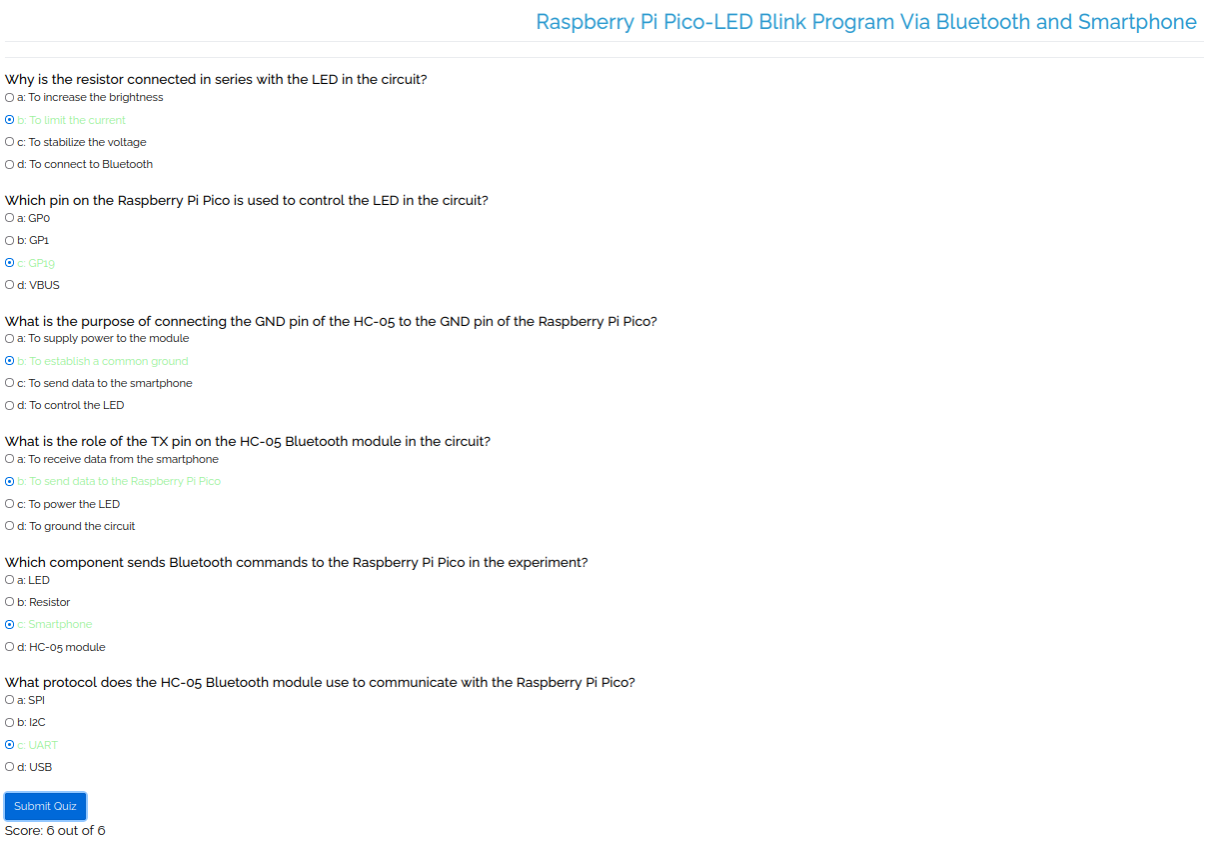


Figure 25: Pre-Test Screenshot

Experiment Performed on Virtual Lab

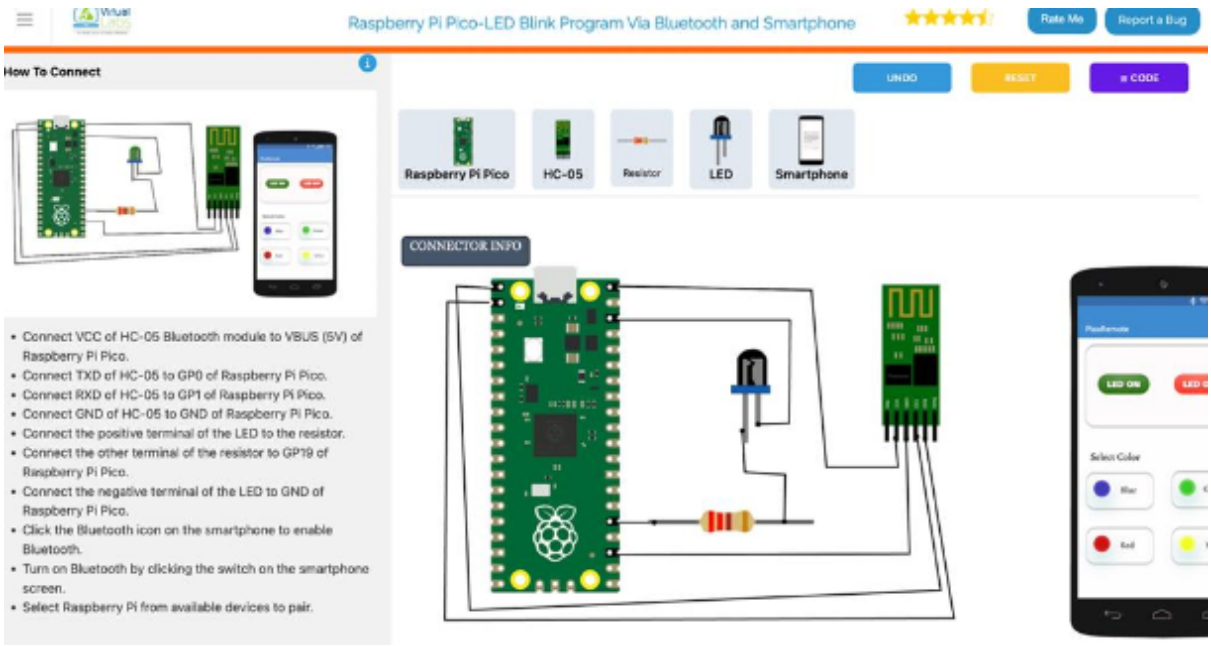


Figure 26: Raspberry pi Setup

Post-Test

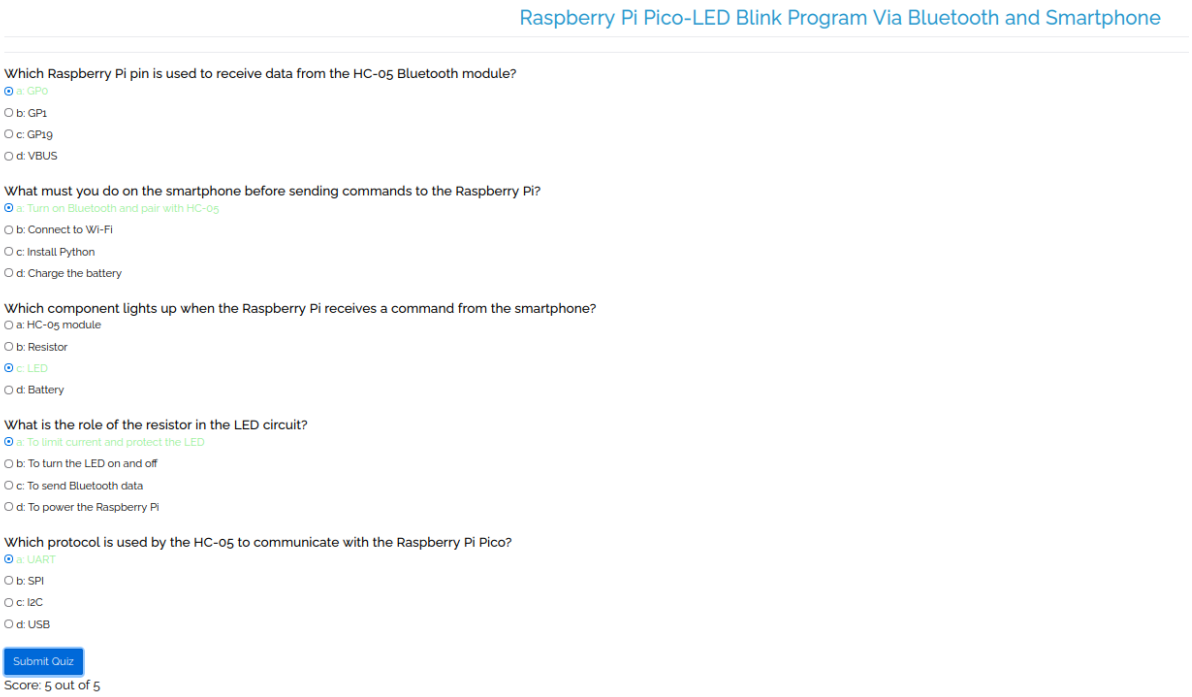


Figure 27: Post-Test Screenshot

Conclusion

In this experiment, we set up Bluetooth communication between a Raspberry Pi Pico and a smartphone to control an LED remotely: the LED turns ON when the phone sends '1' and OFF when it sends '0'. This showed how Bluetooth, serial communication and microcontroller programming work together for wireless device control. The same idea is used in home automation, smart lighting, wireless sensor networks, medical alert systems and remote-controlled robots.

ENTC Experiments

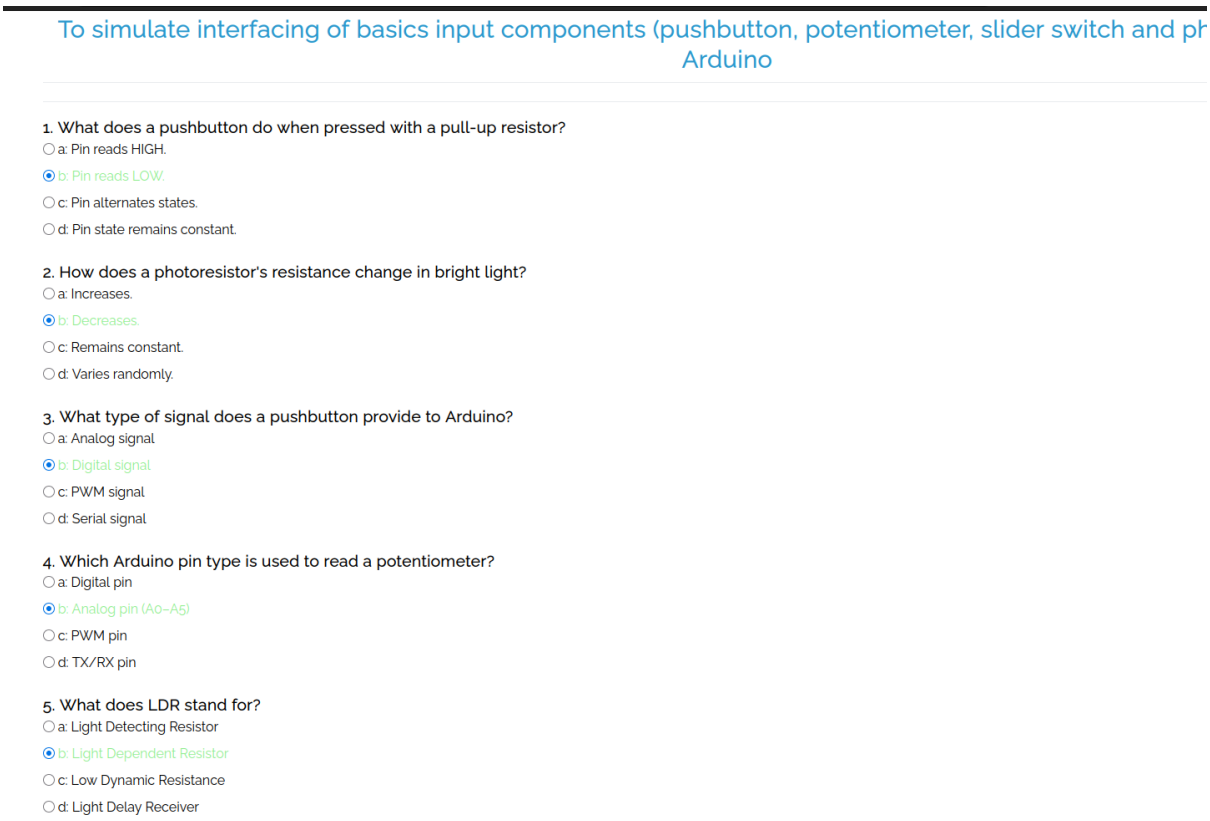
IoT and Edge Computing – Virtual Labs

Experiment 1: Interfacing Basic Input Components with Arduino

Aim

To simulate the interfacing of basic input components (pushbutton, potentiometer, slider switch and photoresistor) with Arduino and observe how each device generates digital or analog signals. The experiment aims to understand how user actions and environmental changes are captured as inputs in an IoT system.

Pre-Test


To simulate interfacing of basics input components (pushbutton, potentiometer, slider switch and pl
Arduino

1. What does a pushbutton do when pressed with a pull-up resistor?
○ a: Pin reads HIGH.
◉ b: Pin reads LOW.
○ c: Pin alternates states.
○ d: Pin state remains constant.

2. How does a photoresistor's resistance change in bright light?
○ a: Increases.
◉ b: Decreases.
○ c: Remains constant.
○ d: Varies randomly.

3. What type of signal does a pushbutton provide to Arduino?
○ a: Analog signal
◉ b: Digital signal
○ c: PWM signal
○ d: Serial signal

4. Which Arduino pin type is used to read a potentiometer?
○ a: Digital pin
◉ b: Analog pin (A0–A5)
○ c: PWM pin
○ d: TX/RX pin

5. What does LDR stand for?
○ a: Light Detecting Resistor
◉ b: Light Dependent Resistor
○ c: Low Dynamic Resistance
○ d: Light Delay Receiver

Figure 28: Pre-Test Screenshot

Pre-Test

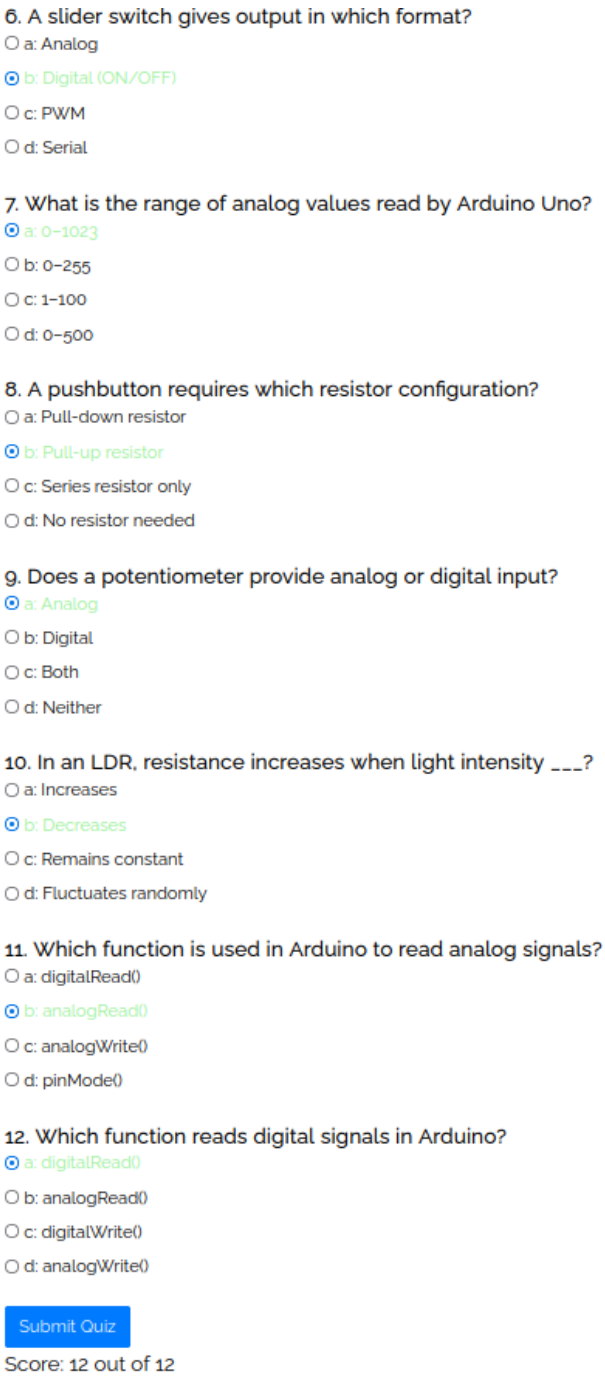


Figure 29: Pre-Test Screenshot

Experiment Performed on Virtual Lab

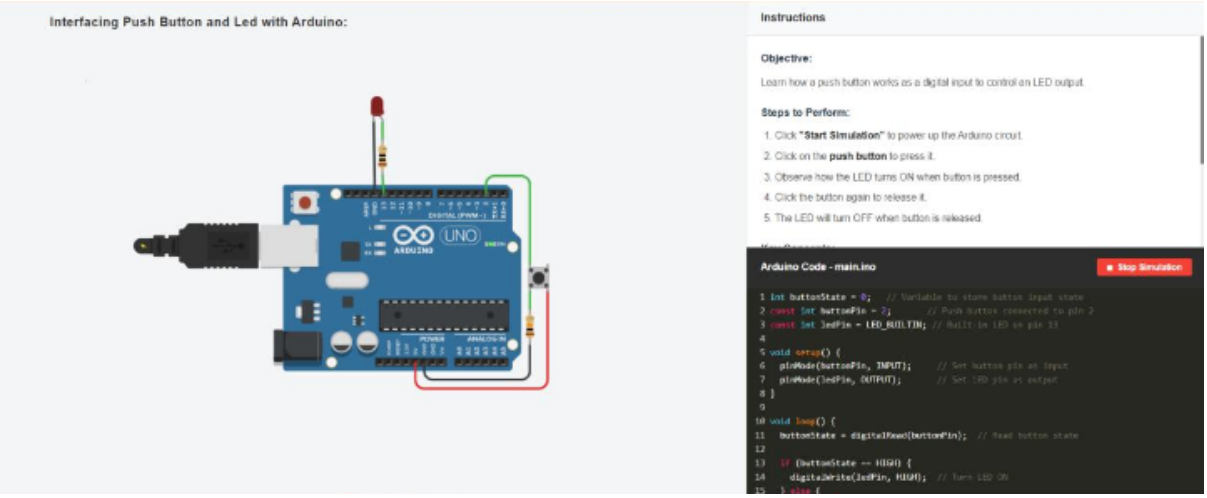


Figure 30: Intial Setup

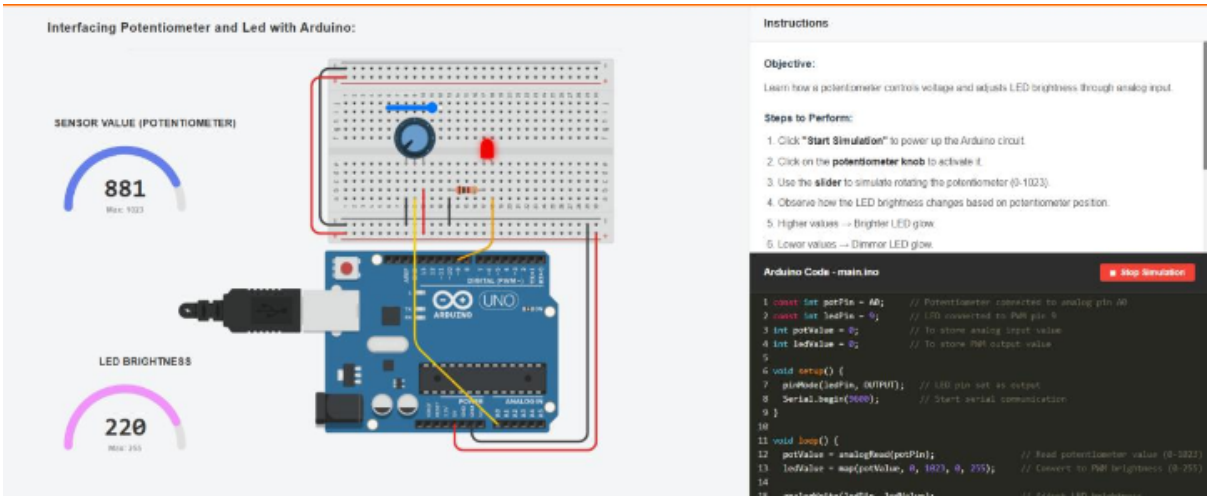


Figure 31: Sensor outputs

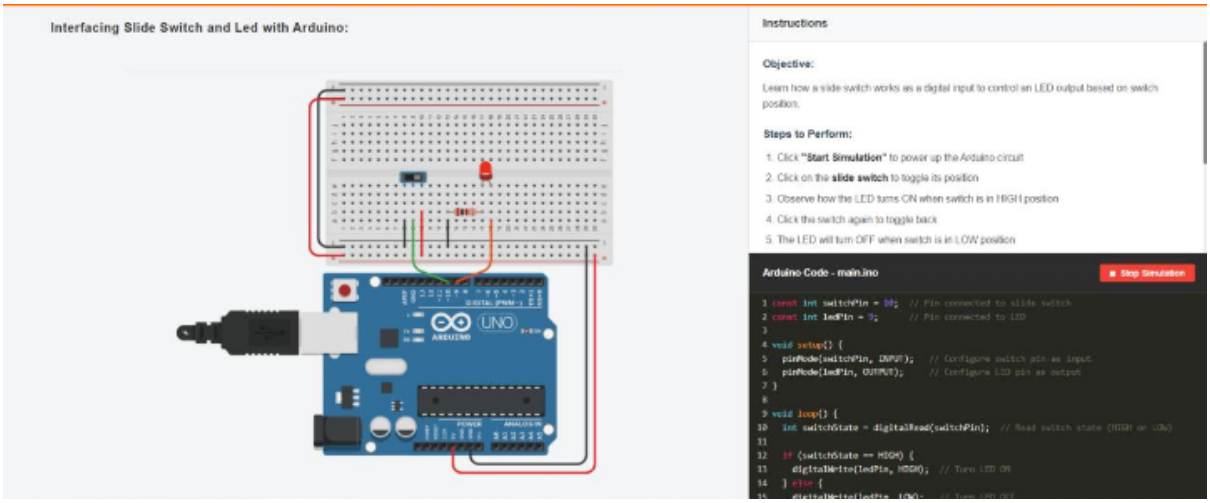


Figure 32: Running state

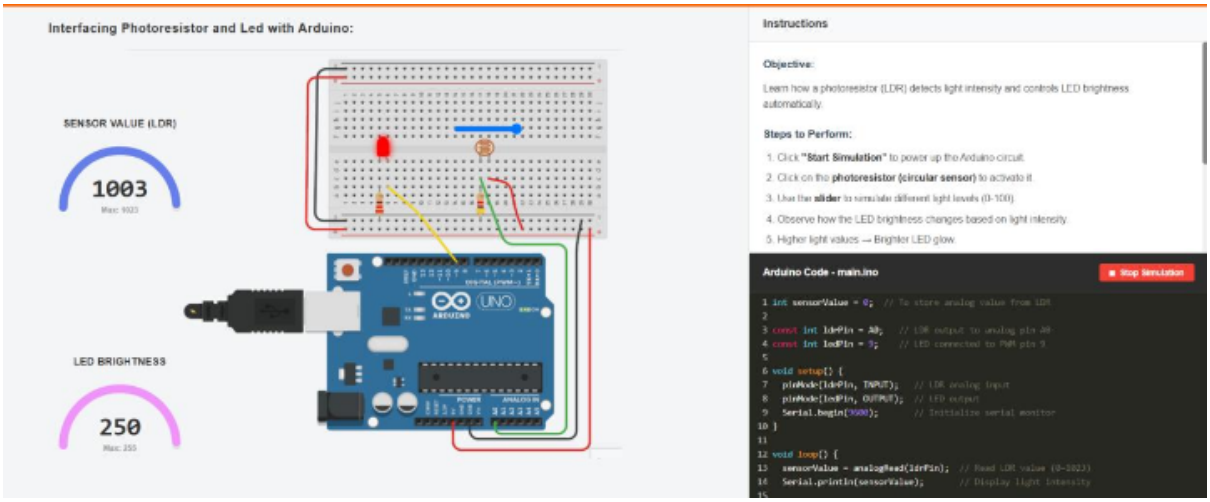


Figure 33: Final output

Post-Test

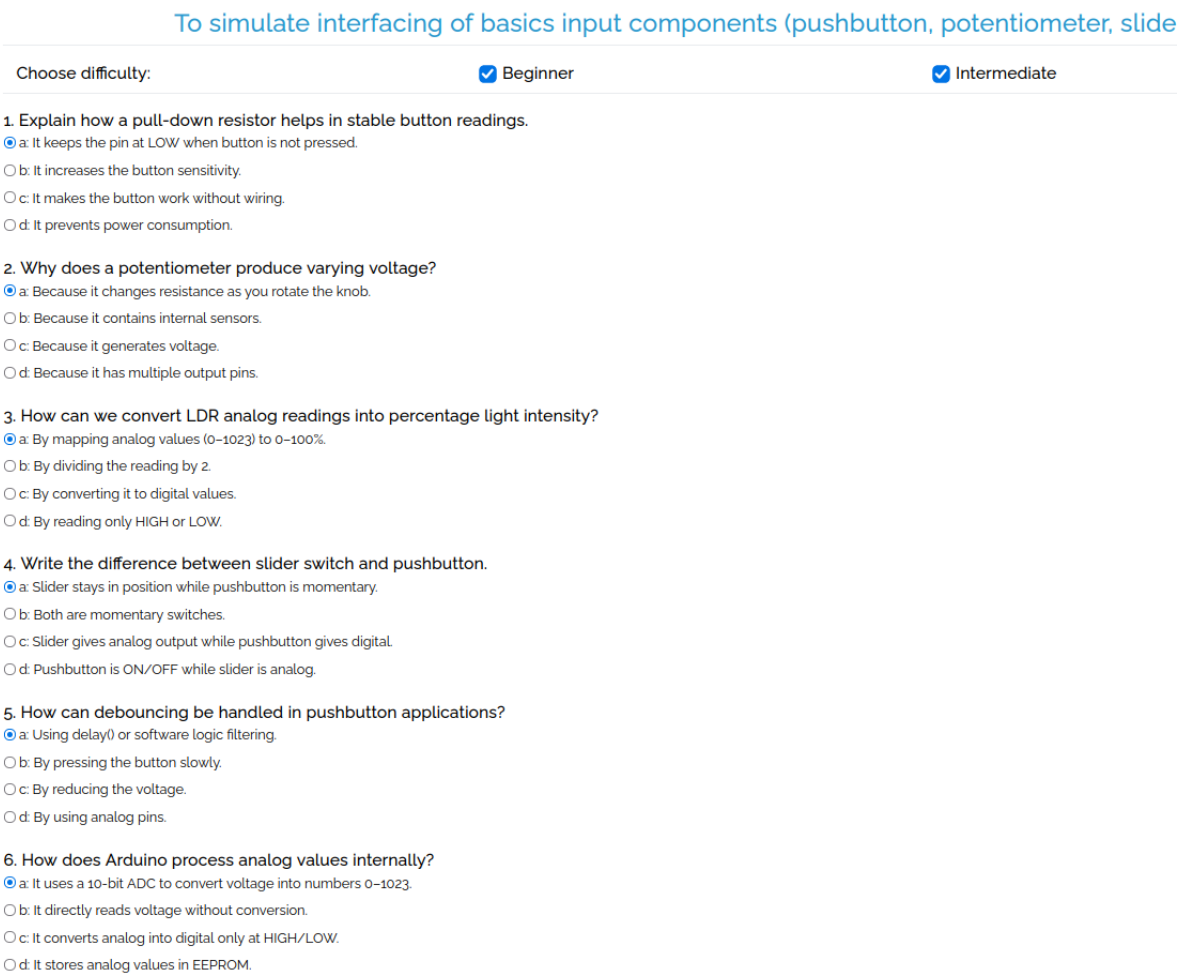


Figure 34: Post-Test Screenshot

Post-Test

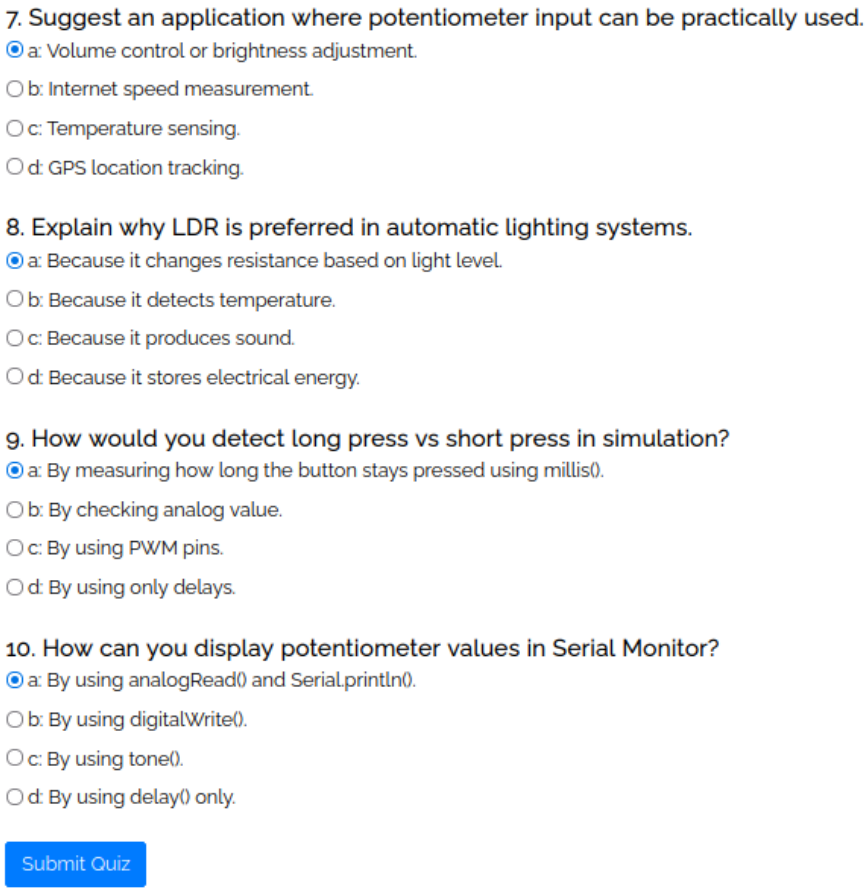


Figure 35: Post-Test Screenshot

Conclusion

In this experiment, we interfaced a pushbutton, potentiometer, slider switch and photoresistor with Arduino in the virtual lab. We saw that the pushbutton and slider switch give digital inputs, while the potentiometer and photoresistor give analog values that the Arduino ADC converts to a range of 0 to 1023. This shows how an Arduino reads user actions and environmental changes through its digital and analog pins.

Experiment 2: Interfacing Basic Output Components with Arduino

Aim

To simulate the working of commonly used output devices (RGB LED, 7-segment display, OLED, 16×2 LCD and buzzer) with Arduino and analyze how a microcontroller generates visual or audio responses. The experiment also demonstrates control over color, display characters, numeric output and sound.

Pre-Test

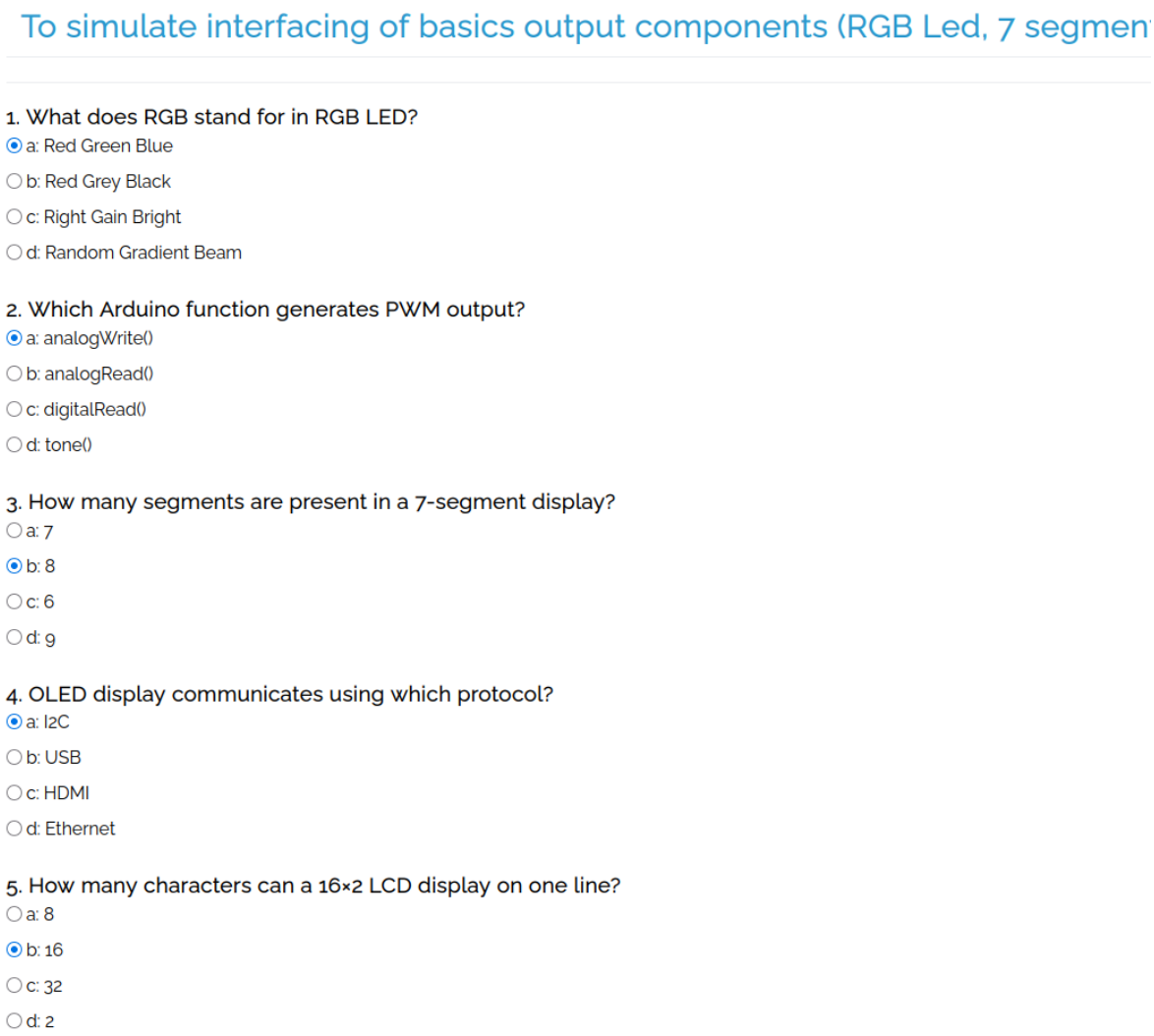


Figure 36: Pre-Test Screenshot

Pre-Test

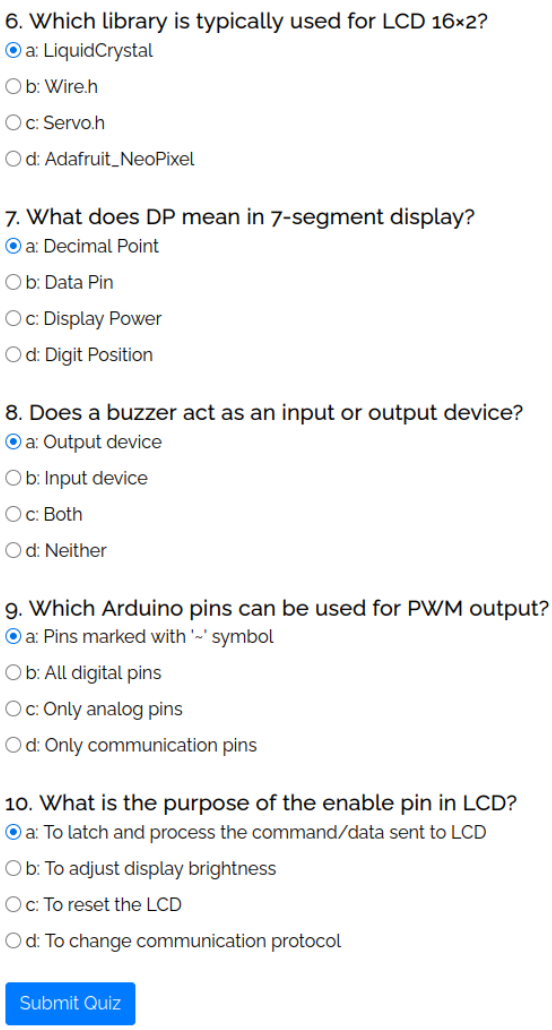


Figure 37: Pre-Test Screenshot

Experiment Performed on Virtual Lab

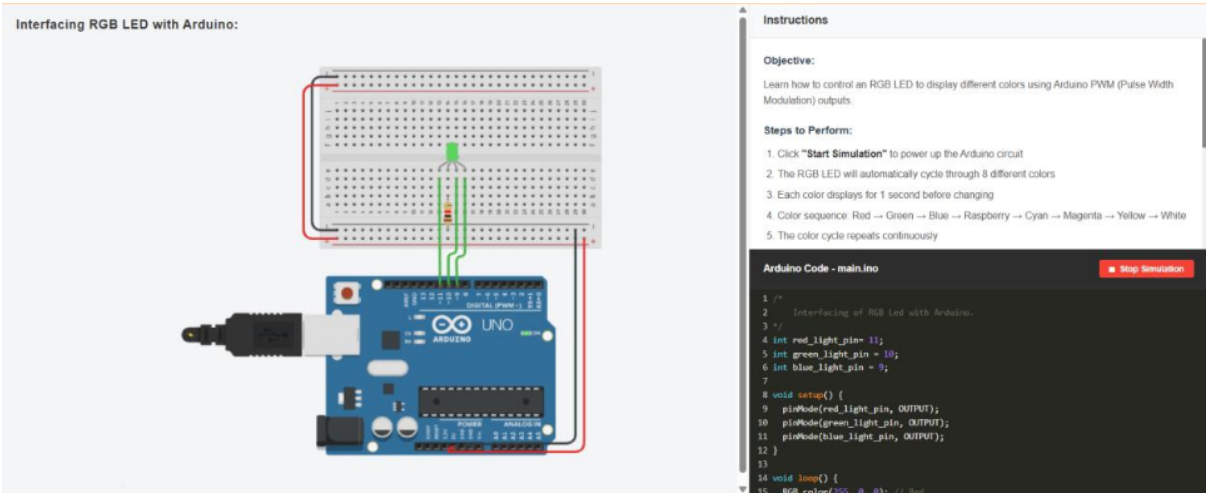


Figure 38: Interfacing RGB LED with Arduino

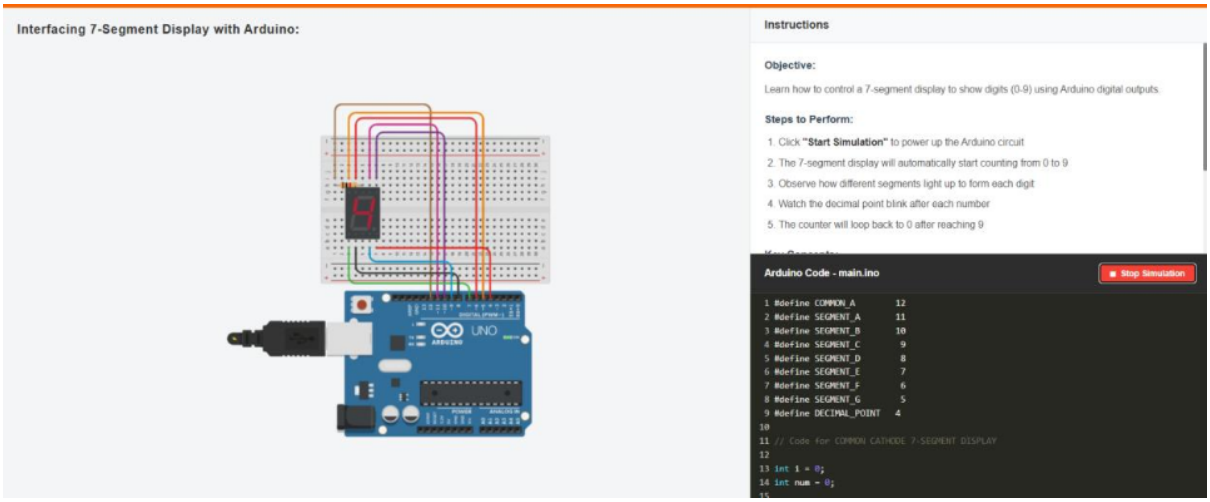


Figure 39: Interfacing 7-segment display with Arduino

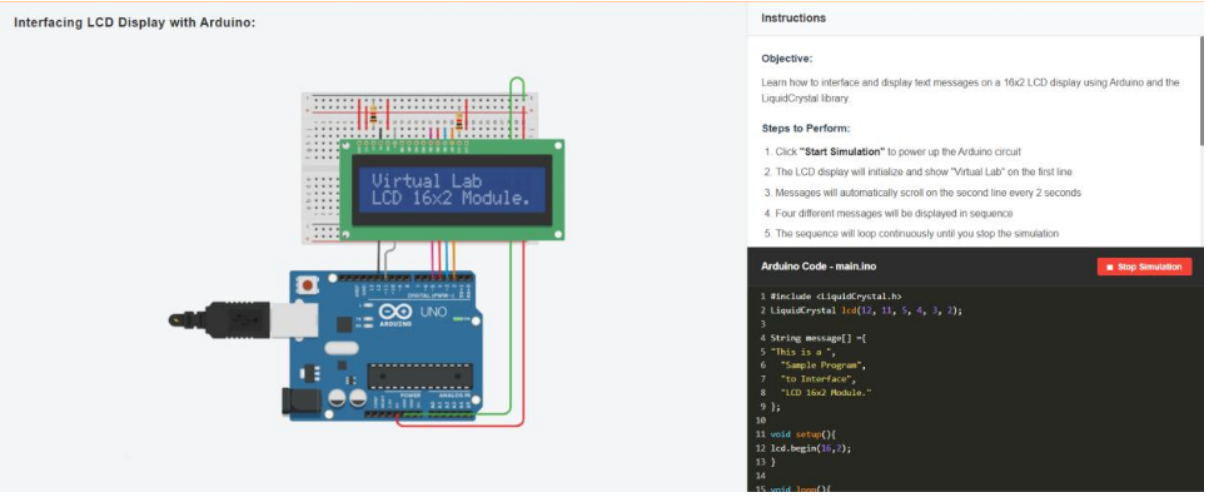


Figure 40: Interfacing LCD display with Arduino

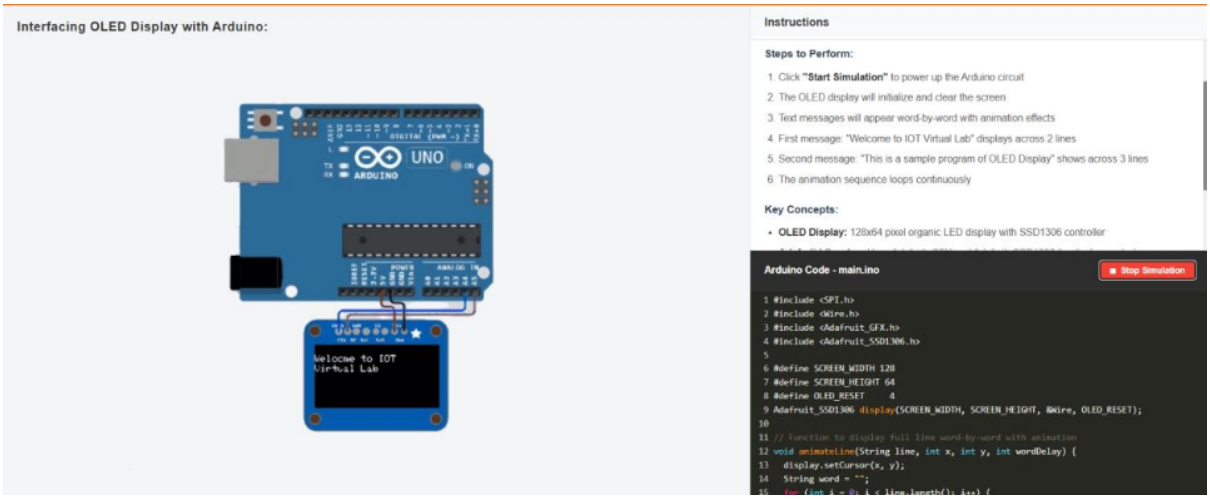


Figure 41: Interfacing OLED Display with Arduino

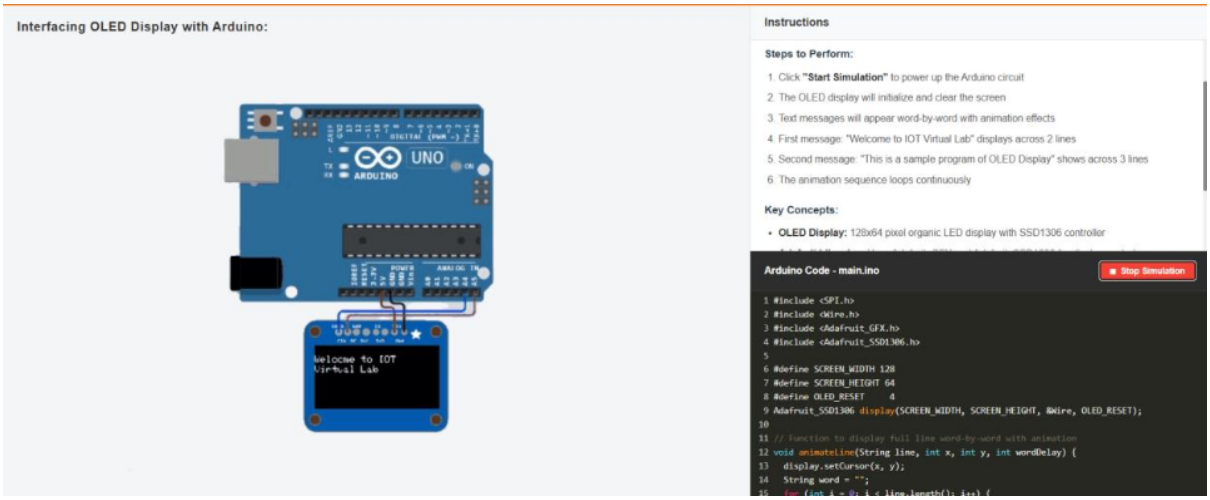


Figure 42: Interfacing Buzzer and Push Button with Arduino

Post-Test

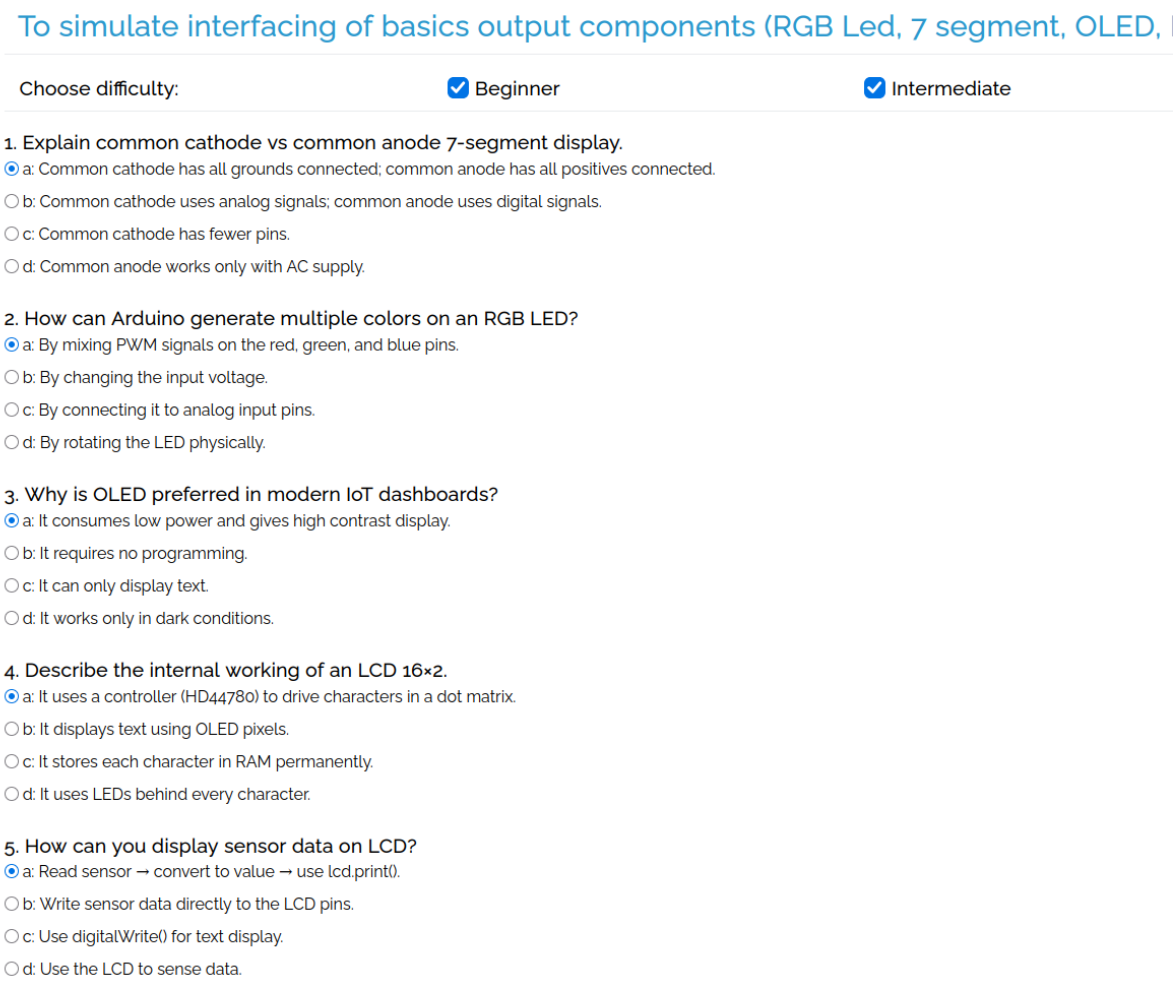


Figure 43: Post-Test Screenshot

Post-Test

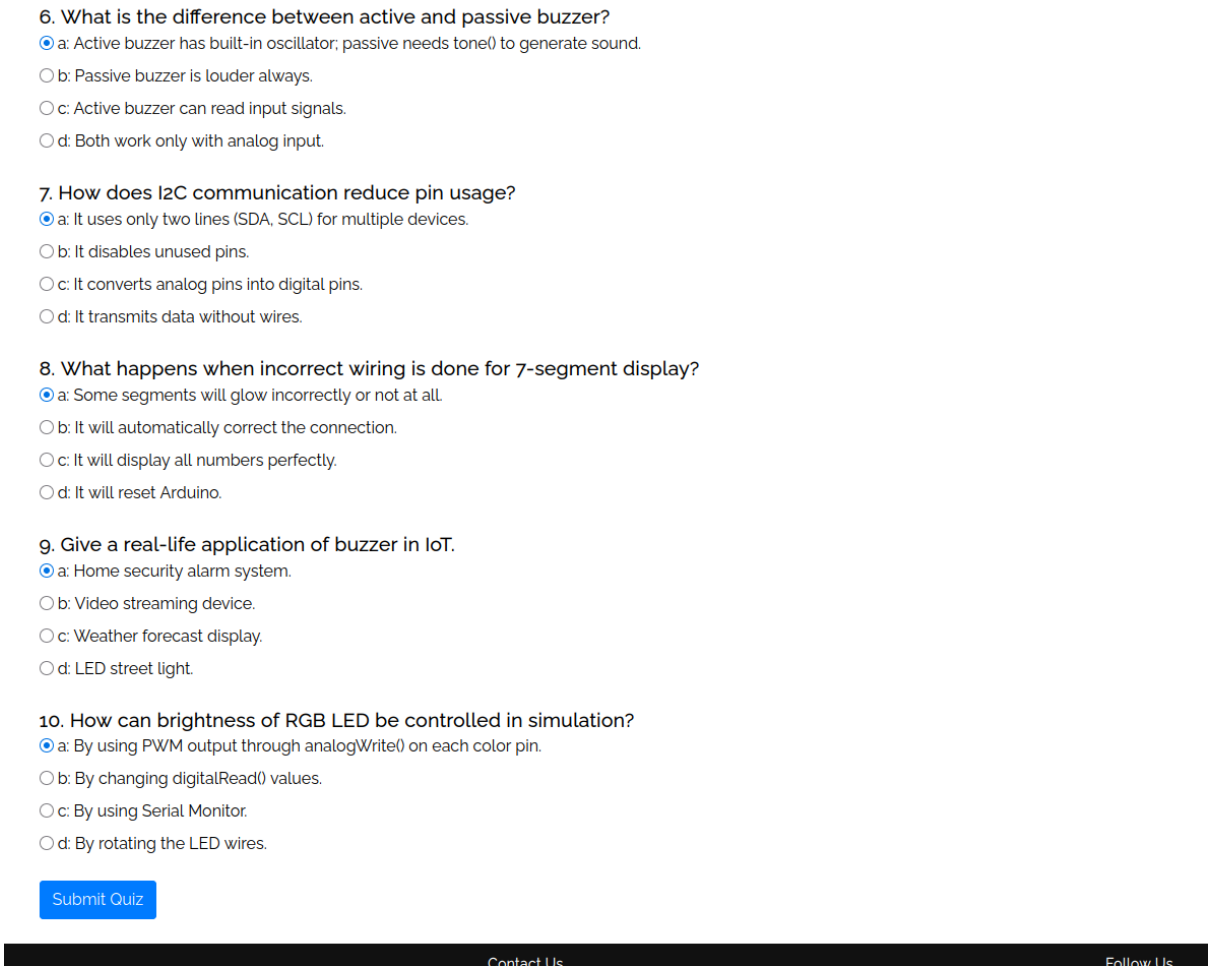


Figure 44: Post-Test Screenshot

Conclusion

In this experiment, we interfaced an RGB LED, 7-segment display, OLED, 16×2 LCD and buzzer with Arduino and observed their outputs in the simulation. We learned how digital pins, PWM signals and I2C communication are used to control colors, numbers, characters and sound. These output devices are the basis of user interfaces and alert systems in embedded and IoT applications.

Experiment 3: Interfacing Multiple Motors with Arduino

Aim

To simulate the interfacing and control of different motors (DC motor, servo motor and stepper motor) using Arduino. The objective is to understand their operating principles, including speed control, angle positioning and step-based movement, as used in robotics and automation.

Pre-Test

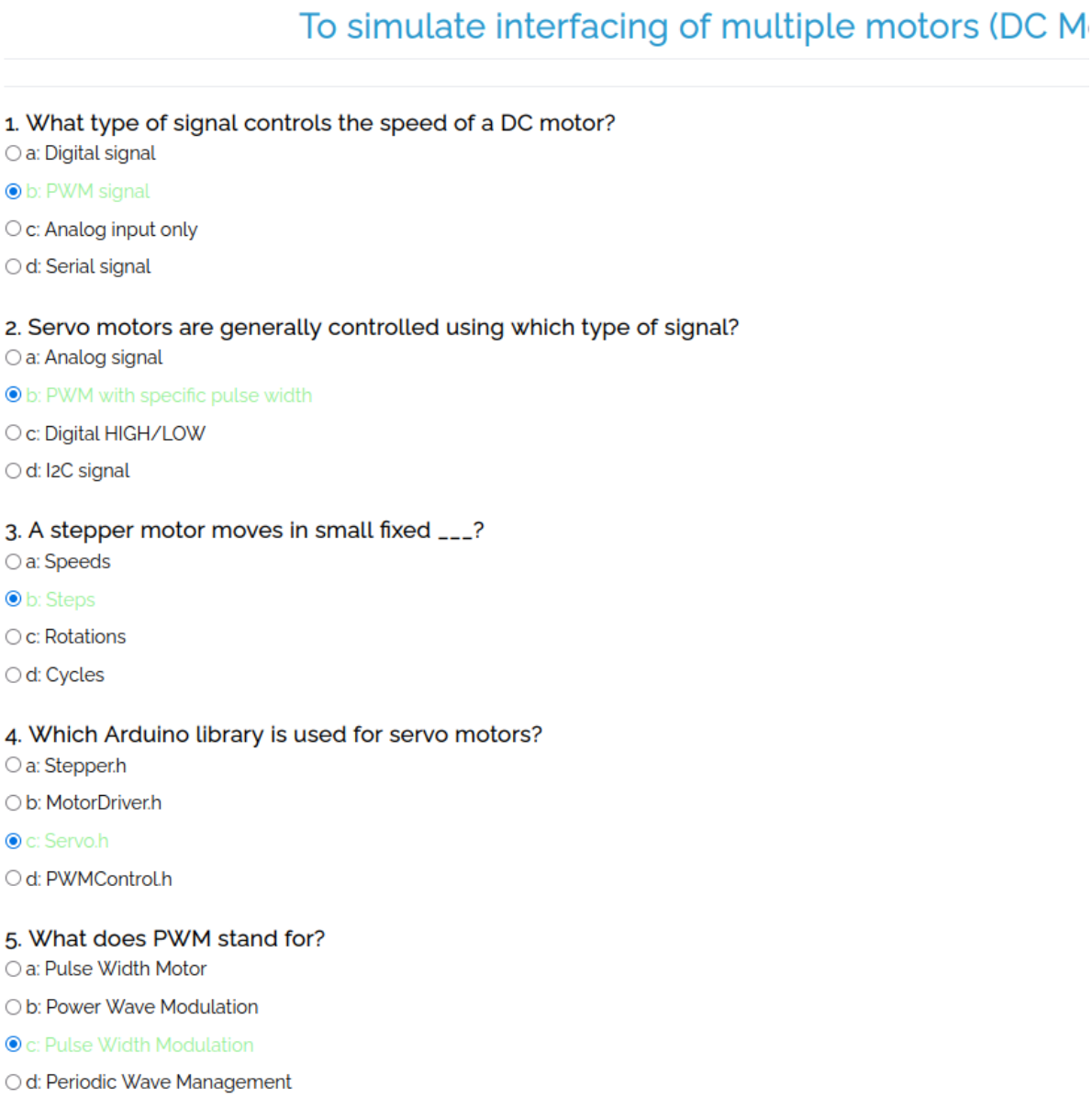


Figure 45: Pre-Test Screenshot

Pre-Test

6. DC motor movement is __-directional by nature.
- a: Bi-directional
- b: Uni-directional
- c: Multi-directional
- d: Non-directional

7. A stepper motor typically requires how many control pins?
- a: 1
- b: 2
- c: 4
- d: 6

8. Which motor is most suitable for precise angular control?
- a: DC motor
- b: Servo motor
- c: Brushless motor
- d: Universal motor

9. Which motor moves in discrete steps?
- a: DC motor
- b: Servo motor
- c: Stepper motor
- d: Vibration motor

10. Which motor is commonly used in robotic arms?
- a: Vibration motor
- b: Stepper motor
- c: DC cooling motor
- d: Induction motor

Submit Quiz

Score: 10 out of 10

Figure 46: Pre-Test Screenshot

Experiment Performed on Virtual Lab

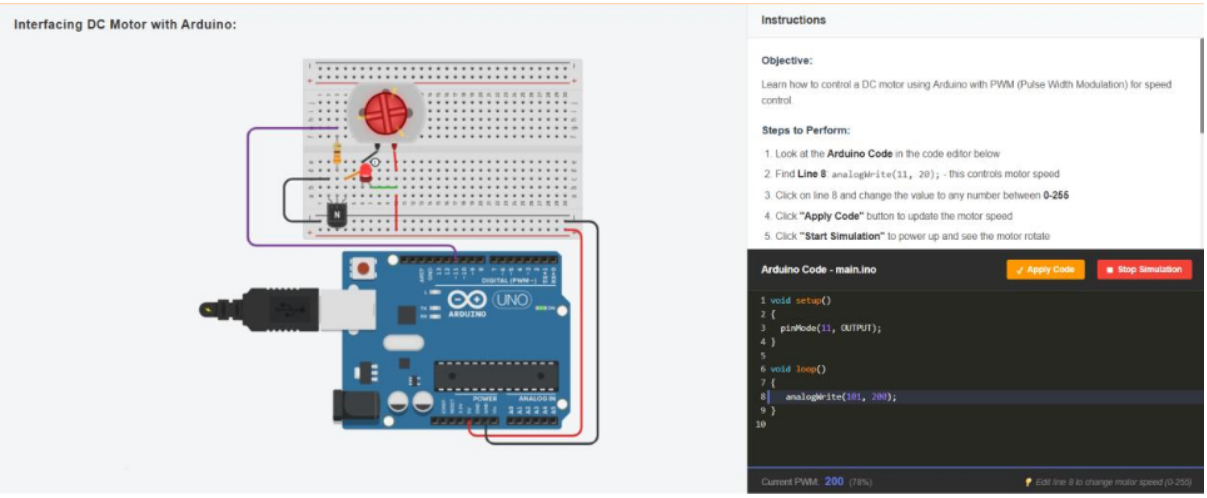


Figure 47: Interfacing DC Motor with Arduino

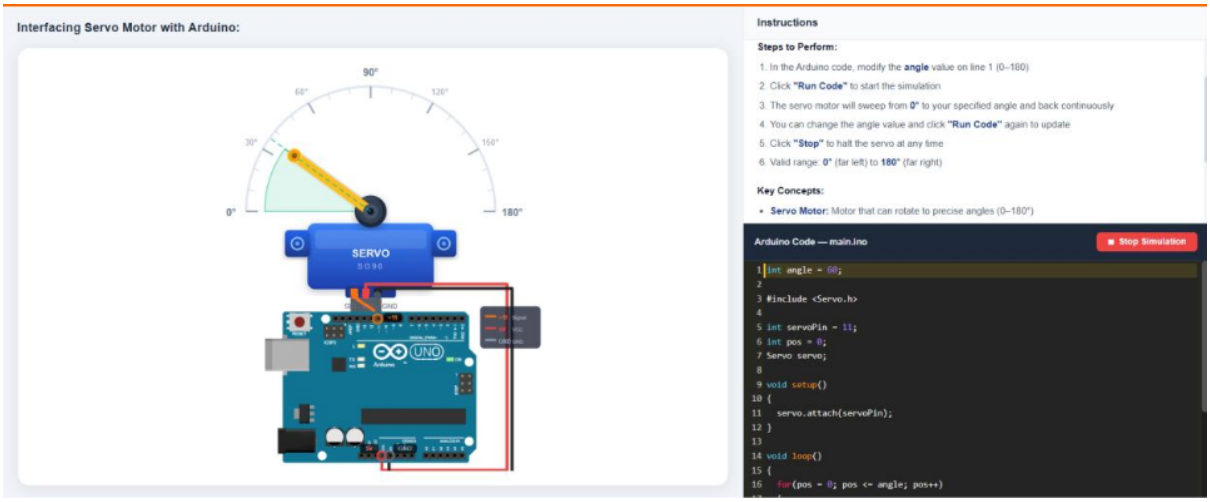


Figure 48: Interfacing Servo Motor with Arduino

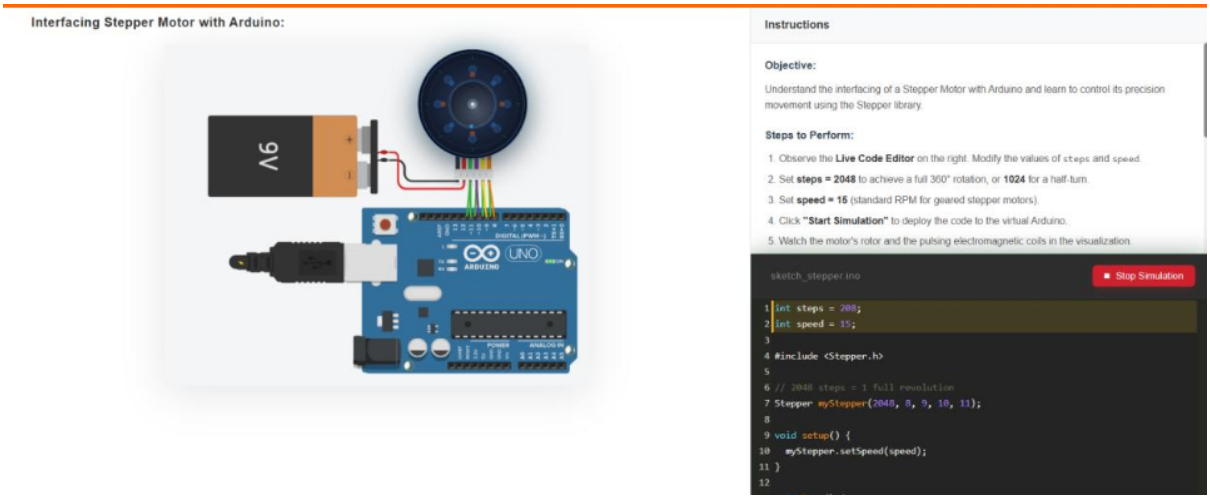


Figure 49: Interfacing Stepper Motor with Arduino

Post-Test

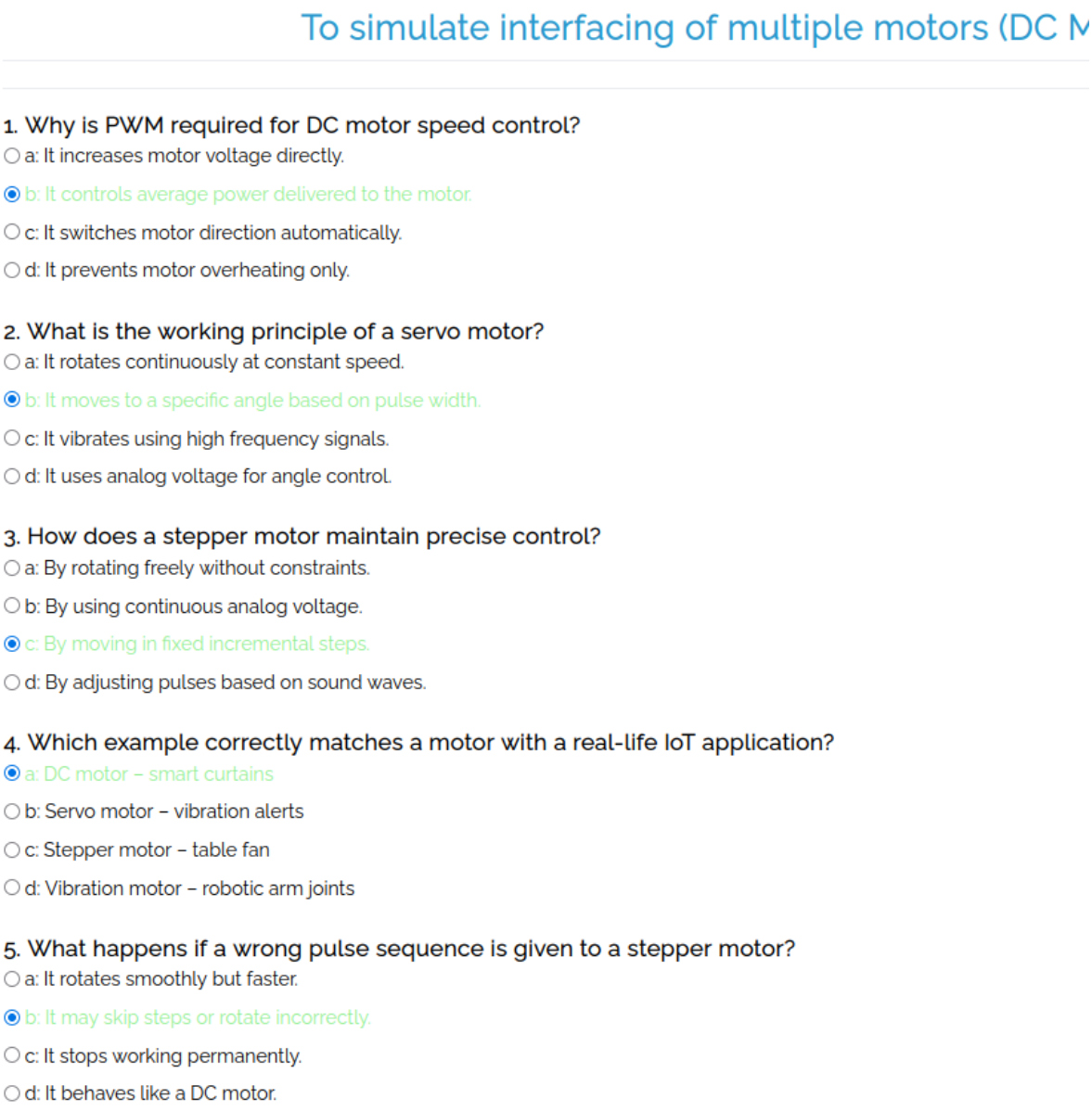


Figure 50: Post-Test Screenshot

Post-Test

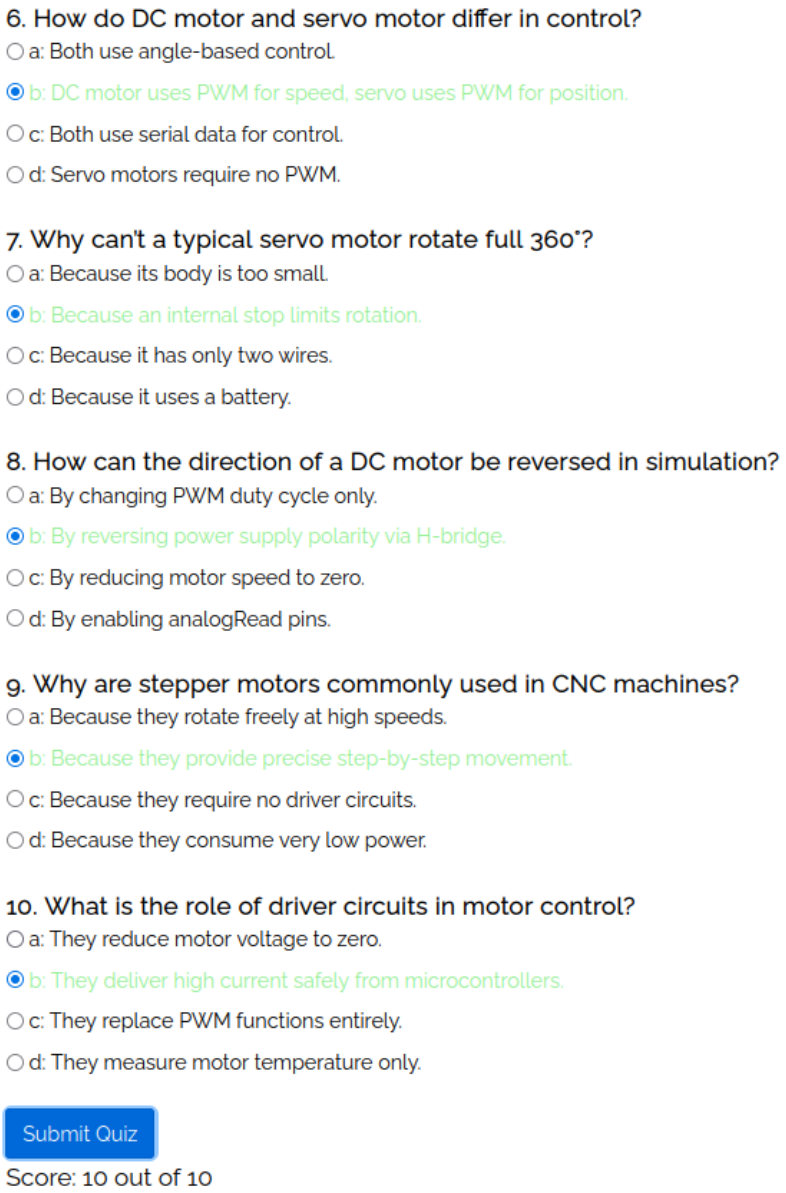


Figure 51: Post-Test Screenshot

Conclusion

In this experiment, we controlled a DC motor, a servo motor and a stepper motor using Arduino. We observed speed control of the DC motor using PWM, angular positioning of the servo, and step-wise rotation of the stepper motor. Because Arduino pins supply very little current, motor drivers are needed to run the motors safely.

Experiment 4: Interfacing Multiple Sensors with Arduino

Aim

To simulate the interfacing of multiple sensors (ultrasonic sensor, PIR sensor, DHT sensor, soil moisture sensor and MQ135 gas sensor) with Arduino. The experiment shows how physical parameters such as distance, motion, temperature, humidity, soil moisture and air quality are detected and read by a microcontroller.

Pre-Test

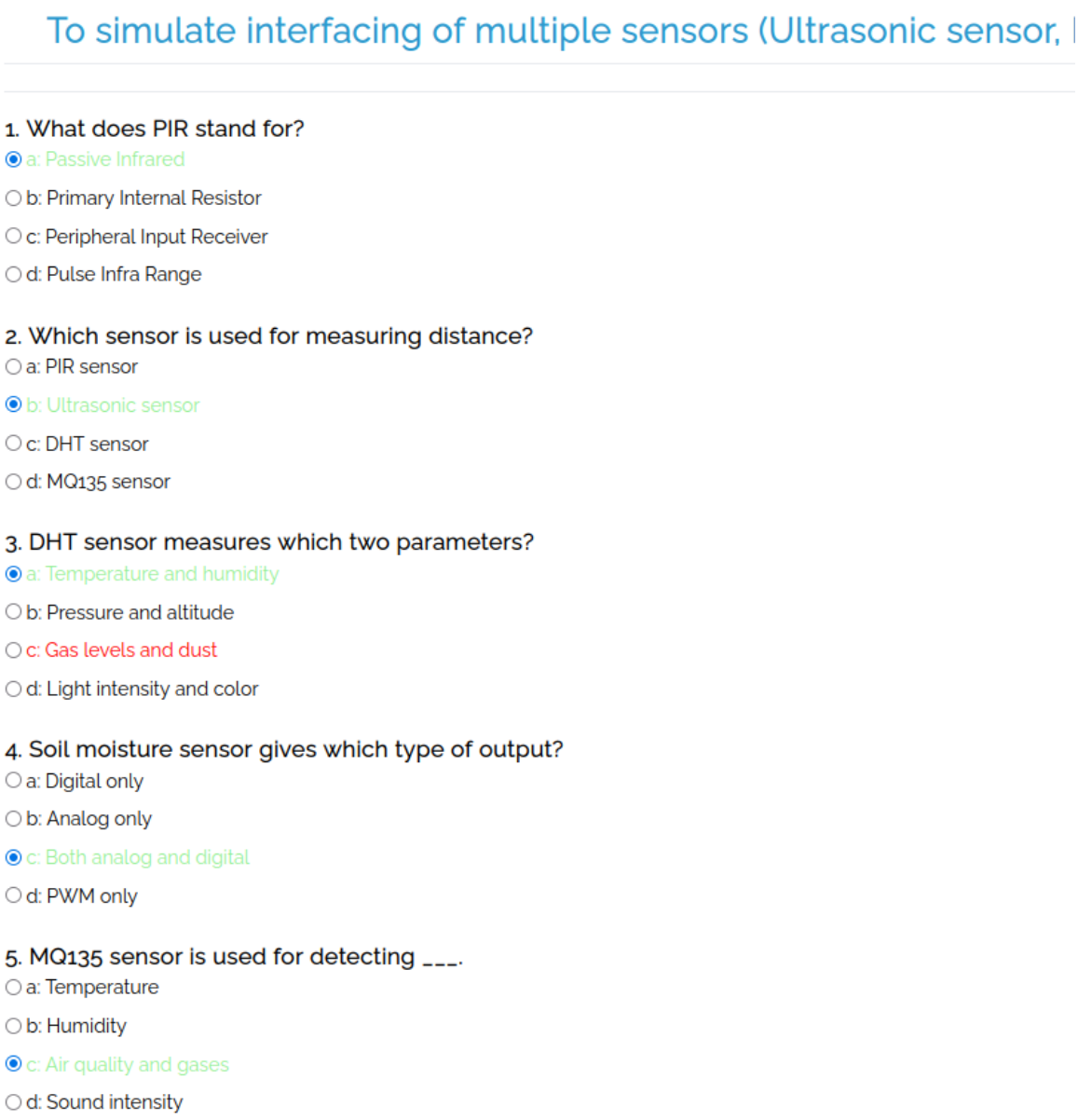


Figure 52: Pre-Test Screenshot

Pre-Test

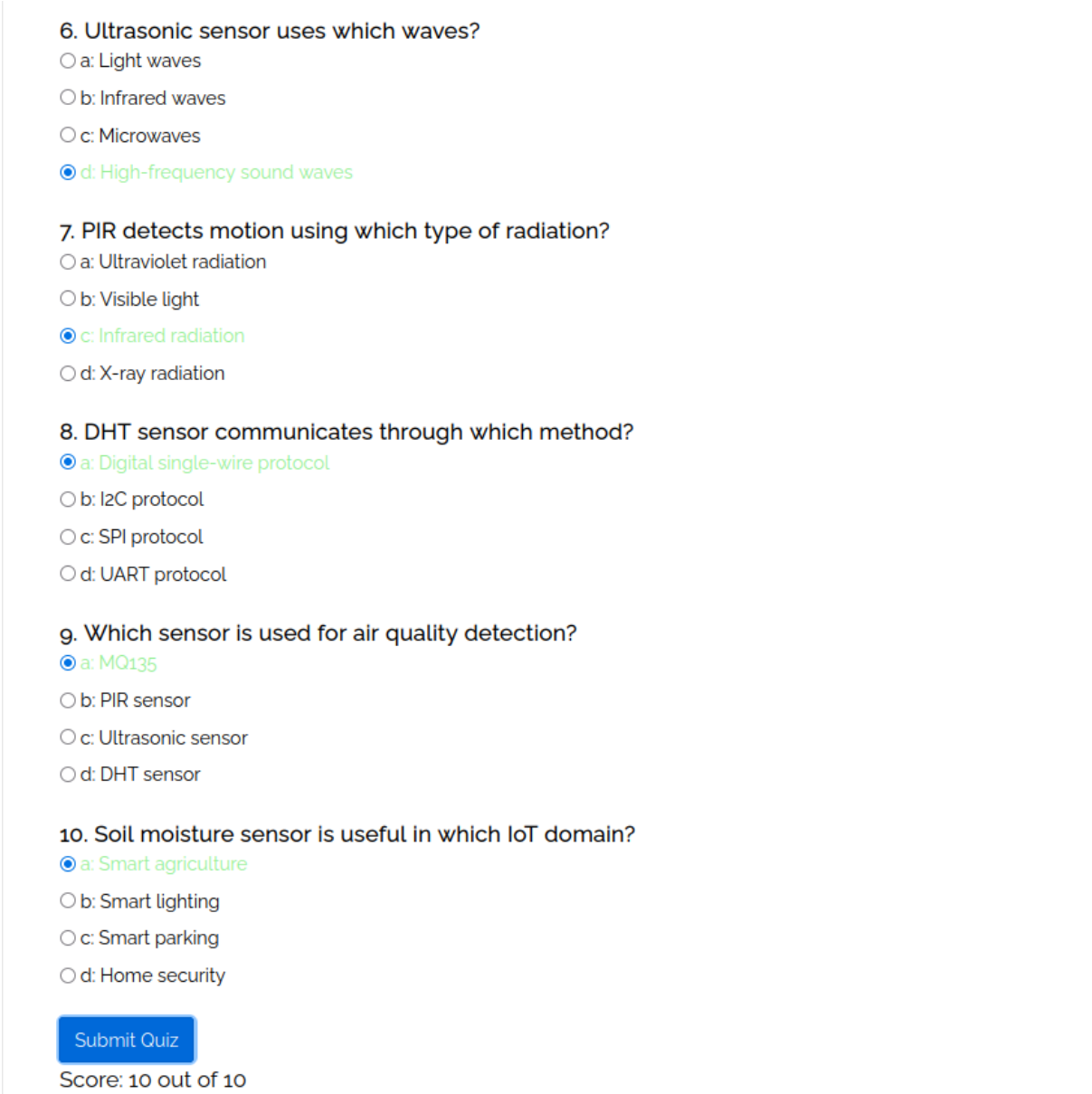


Figure 53: Pre-Test Screenshot

Experiment Performed on Virtual Lab

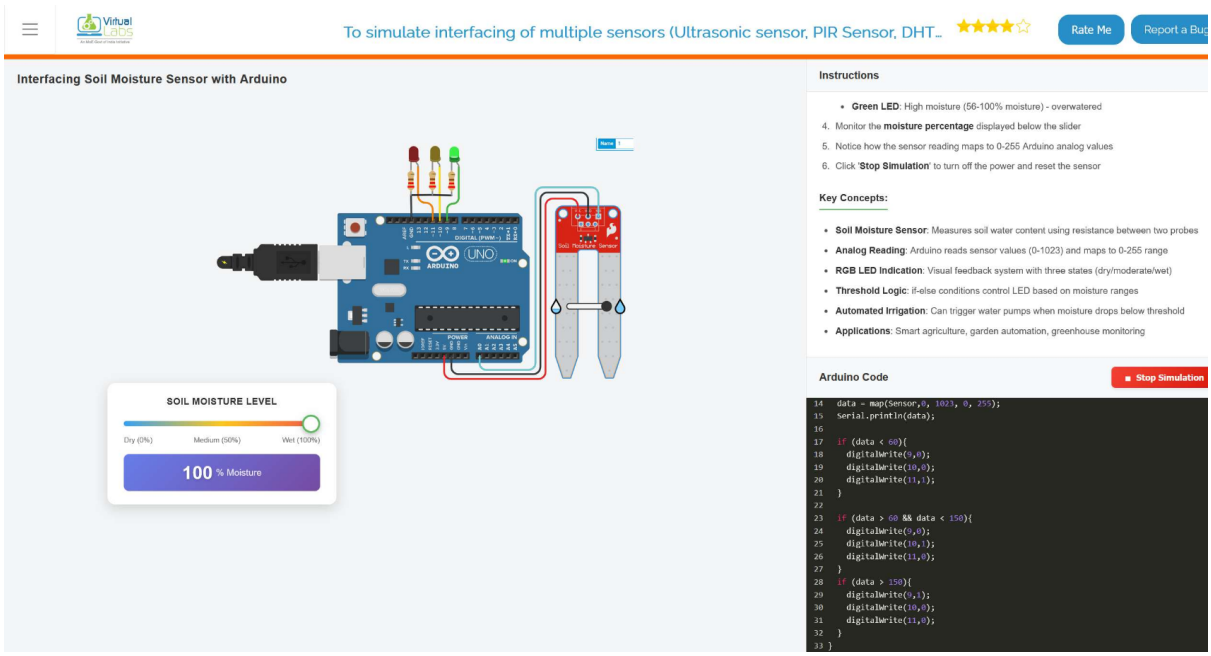


Figure 54: Interfacing soil moisture sensor with arduino

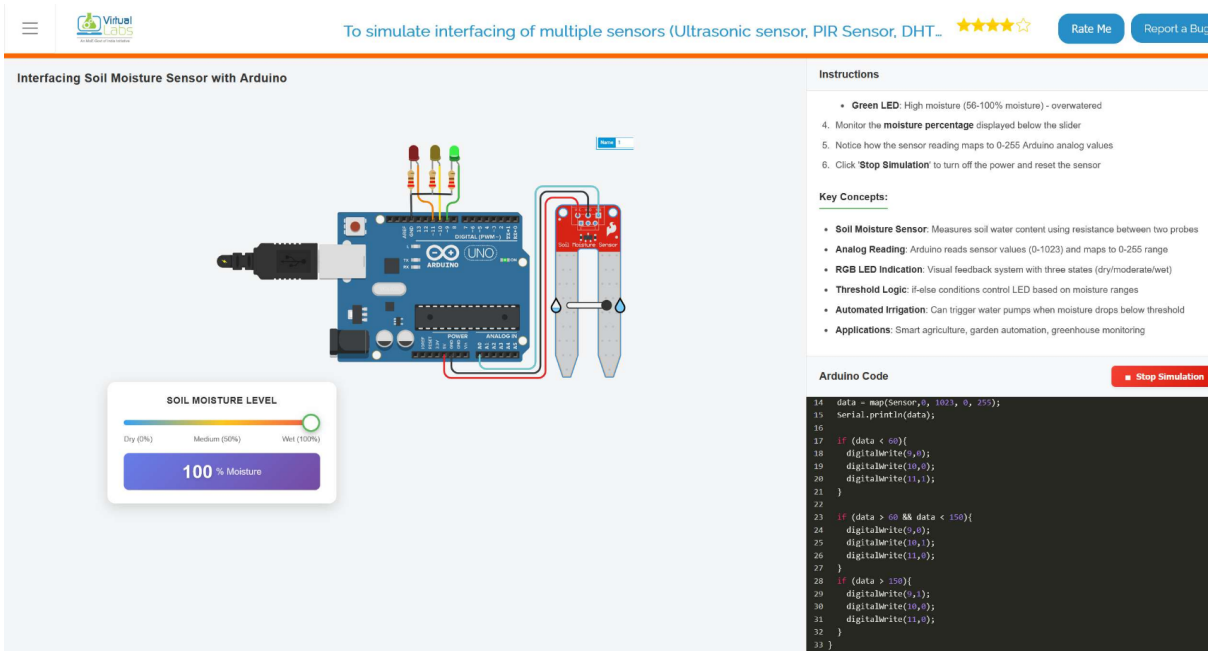


Figure 55: Interfacing MQ135 sensor with arduino

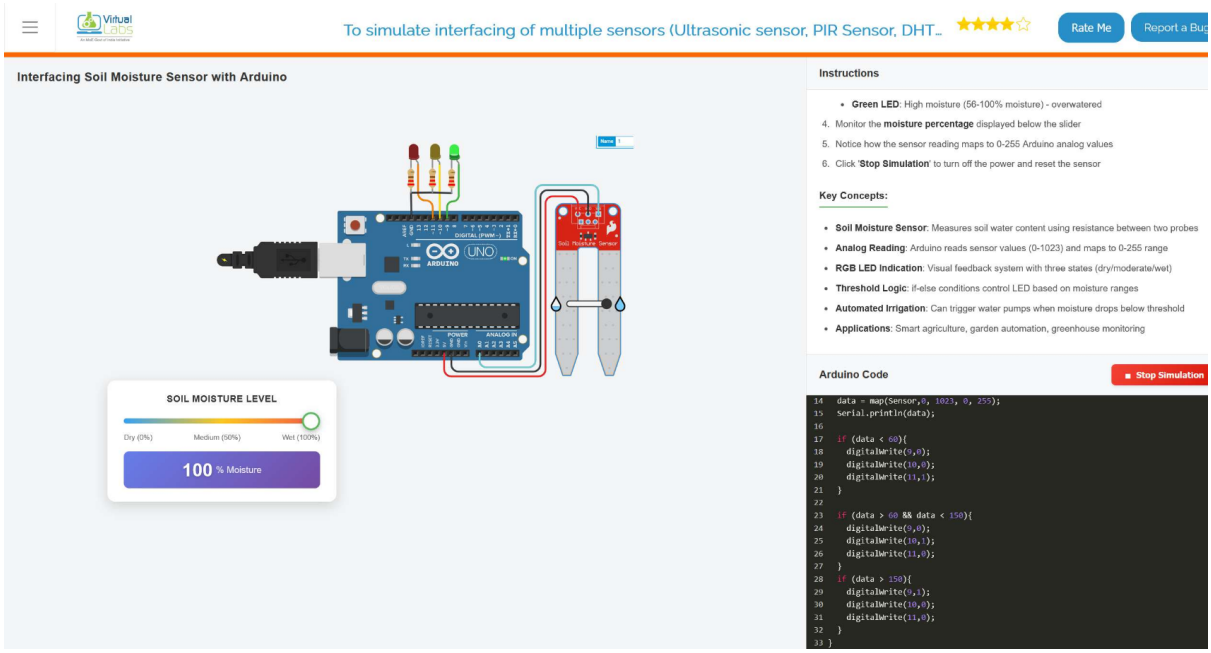


Figure 56: Interfacing PIR Motion sensor with arduino

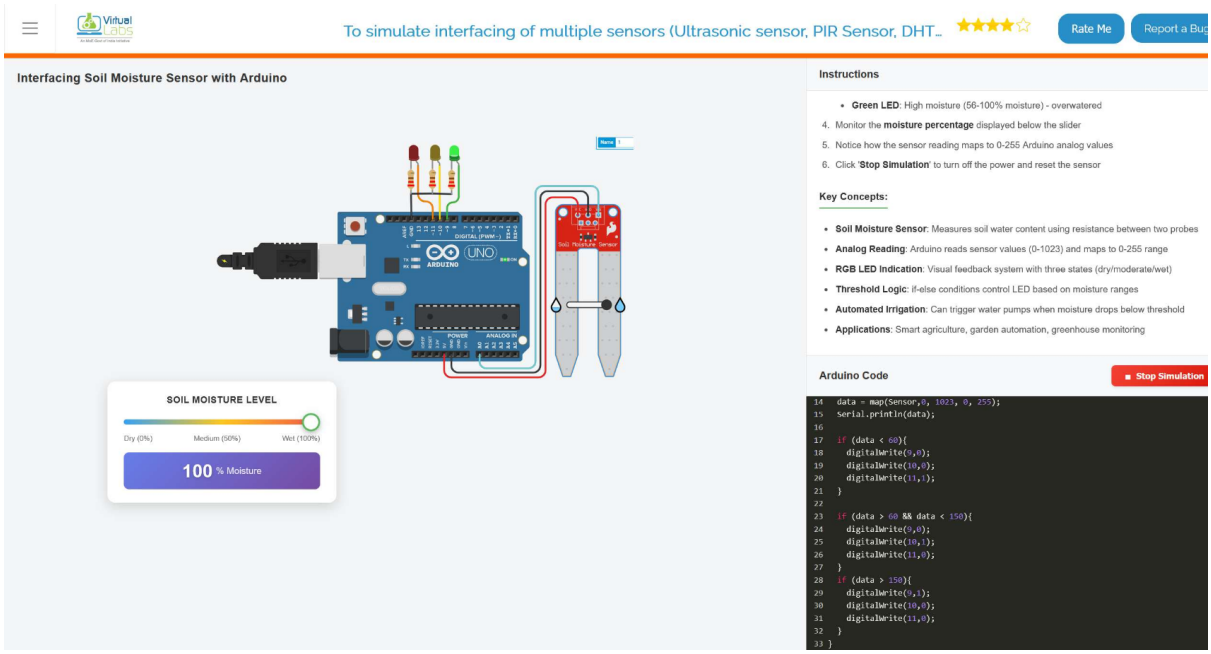


Figure 57: Interfacing Ultrasonic (HC-SR04) sensor with arduino

Post-Test

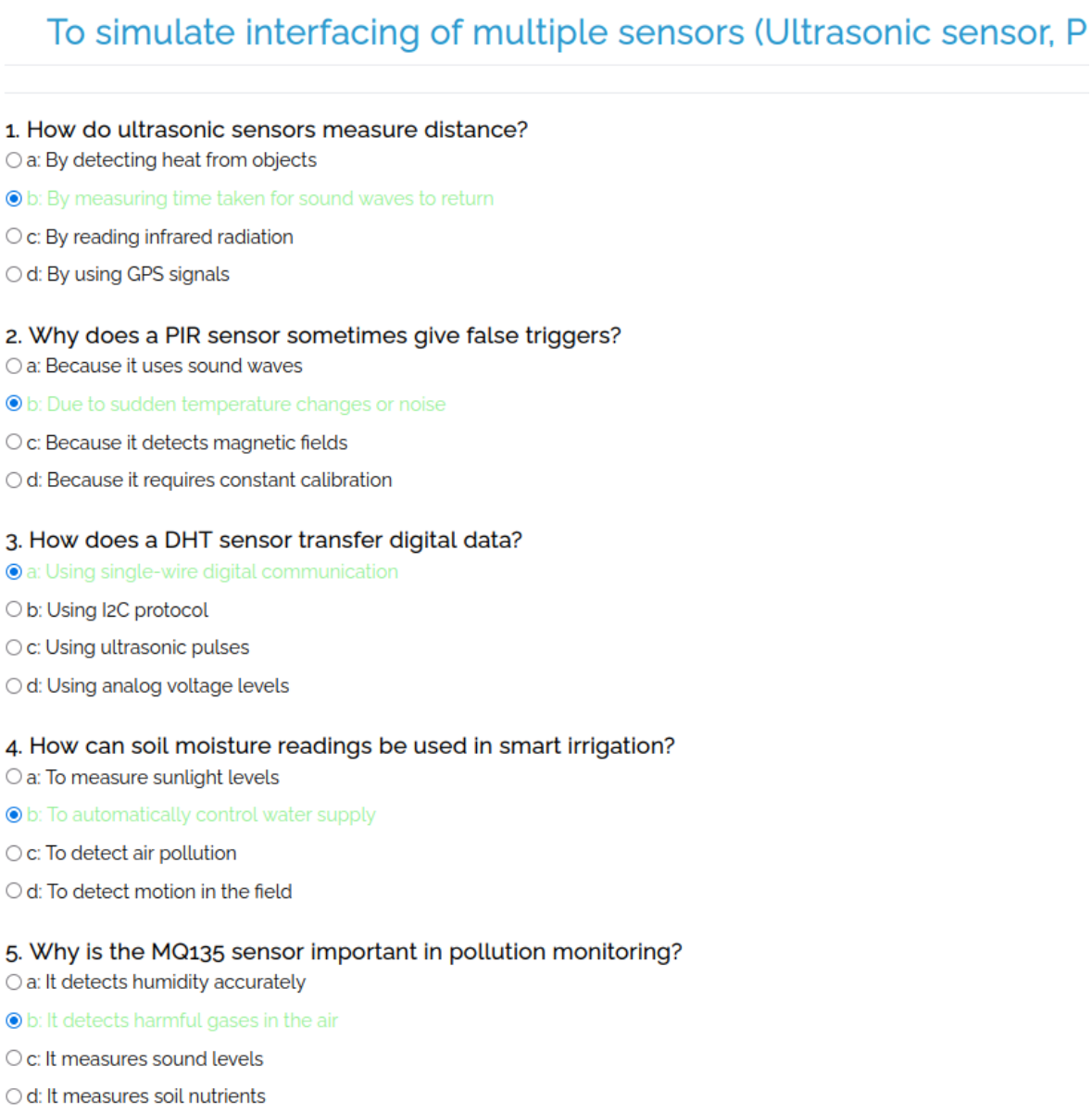


Figure 58: Post-Test Screenshot

Post-Test

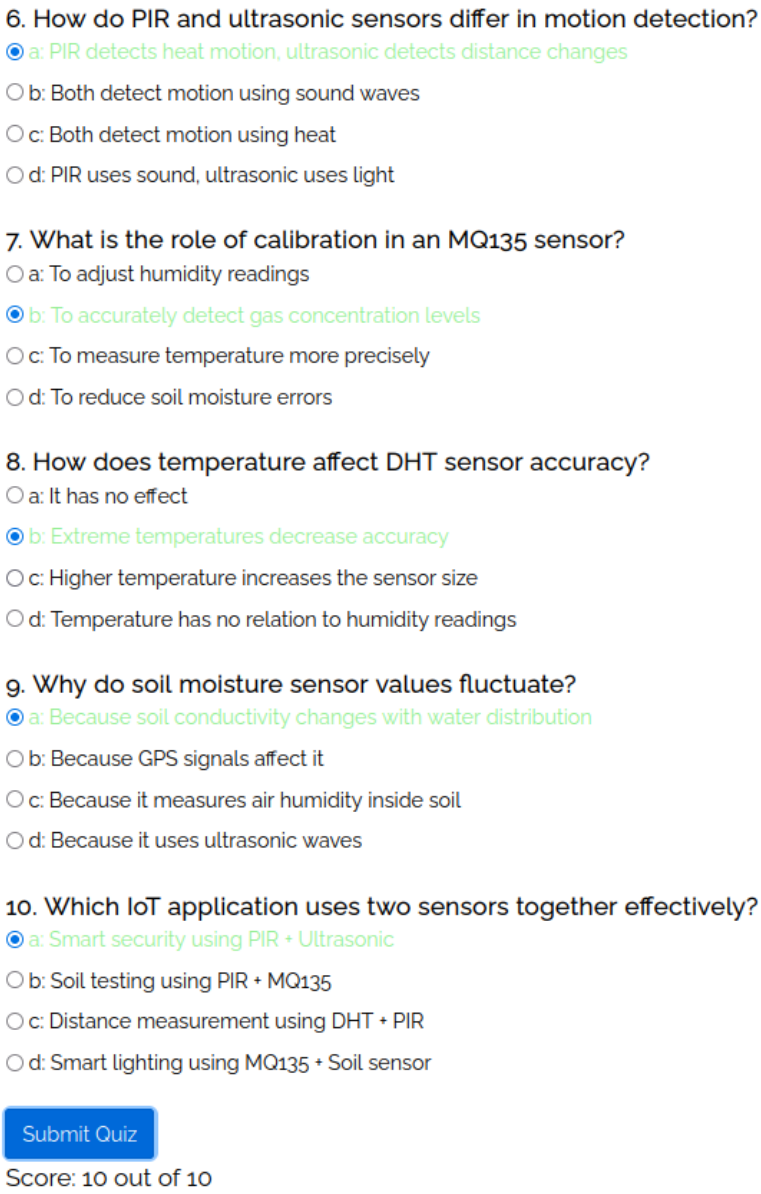


Figure 59: Post-Test Screenshot

Conclusion

In this experiment, we interfaced an ultrasonic sensor, PIR sensor, DHT sensor, soil moisture sensor and MQ135 sensor with Arduino and observed their readings together. We saw how digital and analog sensors are read by the Arduino at the same time. This gives a practical understanding of real-time data collection for environmental monitoring in IoT systems.

Experiment 5: Sending Live Sensor Data to an IoT Platform using HTTP

Aim

To simulate sending live sensor readings to a cloud platform using ESP8266/ESP32 over the HTTP protocol. The objective is to understand REST API communication, web requests and data logging for remote monitoring.

Pre-Test

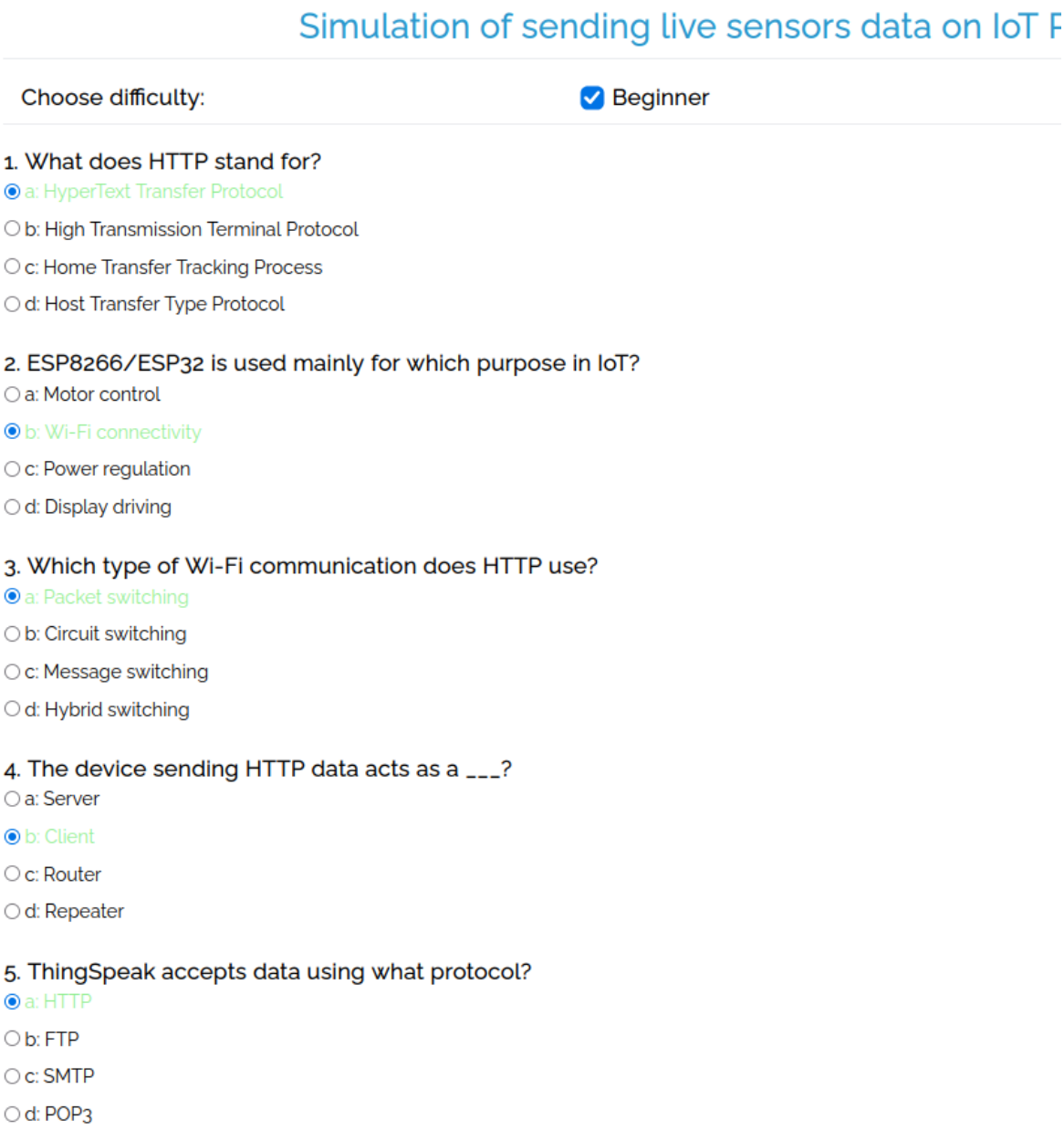


Figure 60: Pre-Test Screenshot

Pre-Test

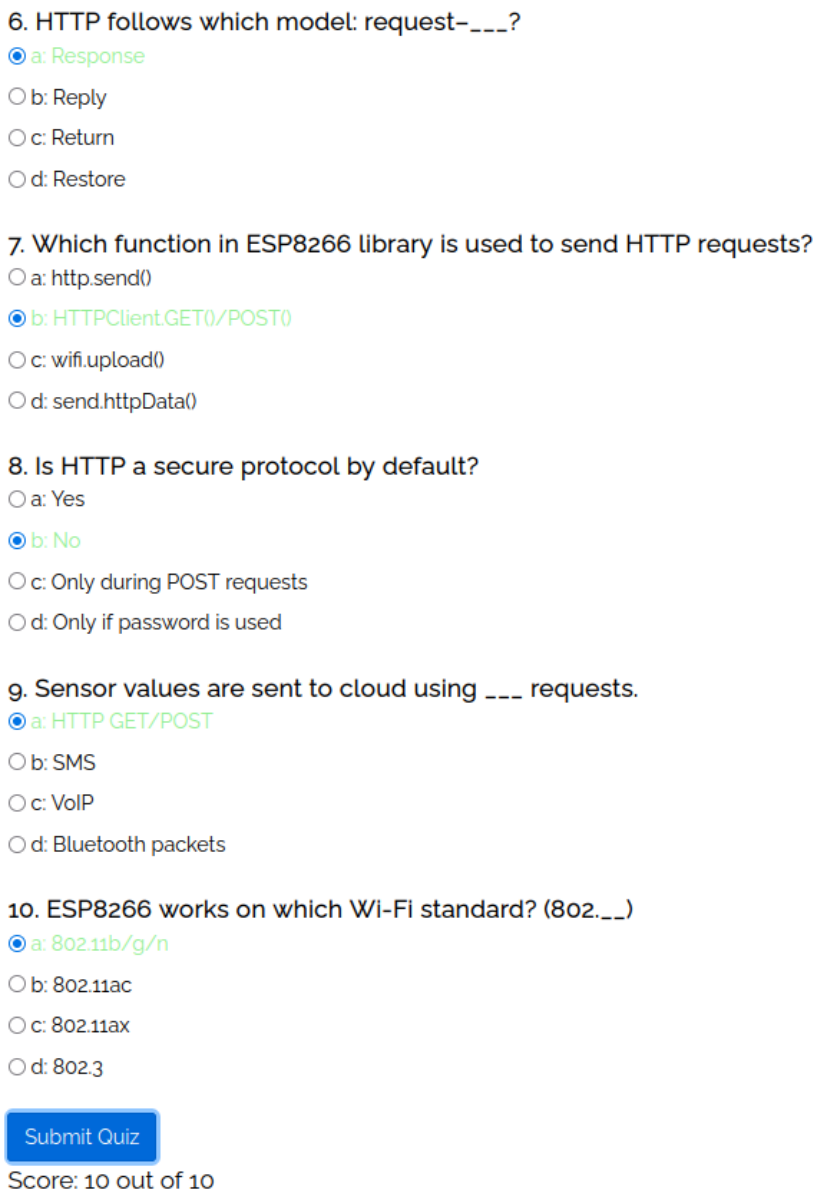


Figure 61: Pre-Test Screenshot

Experiment Performed on Virtual Lab

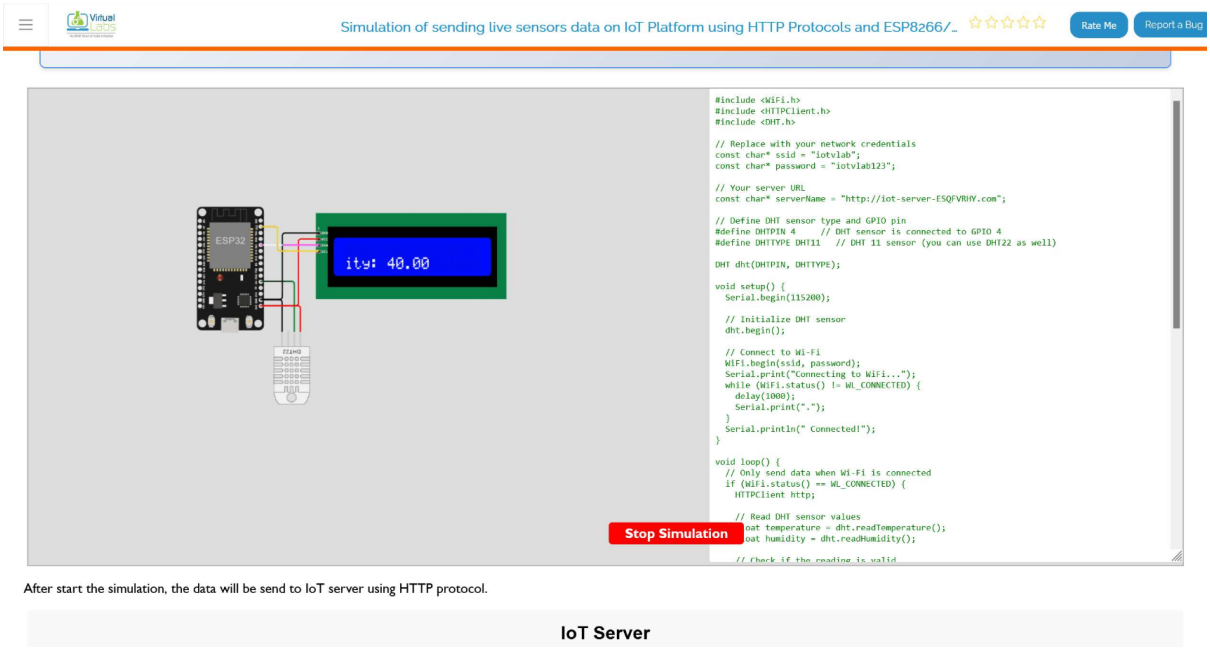


Figure 62: Connection between IOT devices

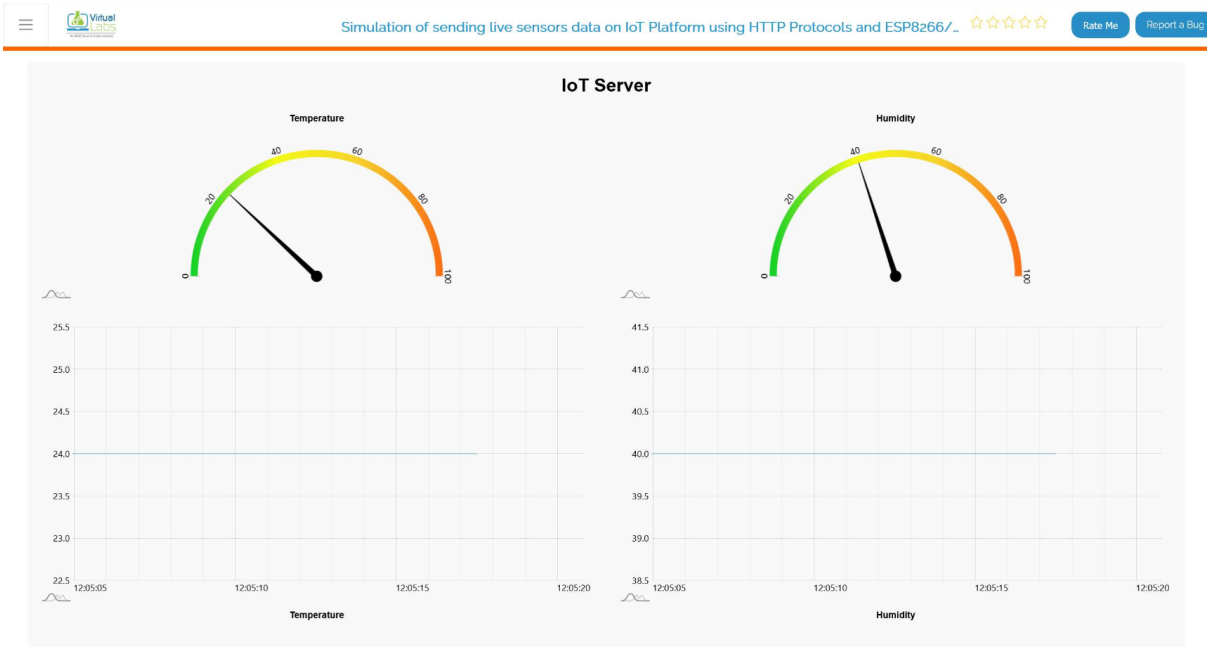


Figure 63: Readings

Post-Test

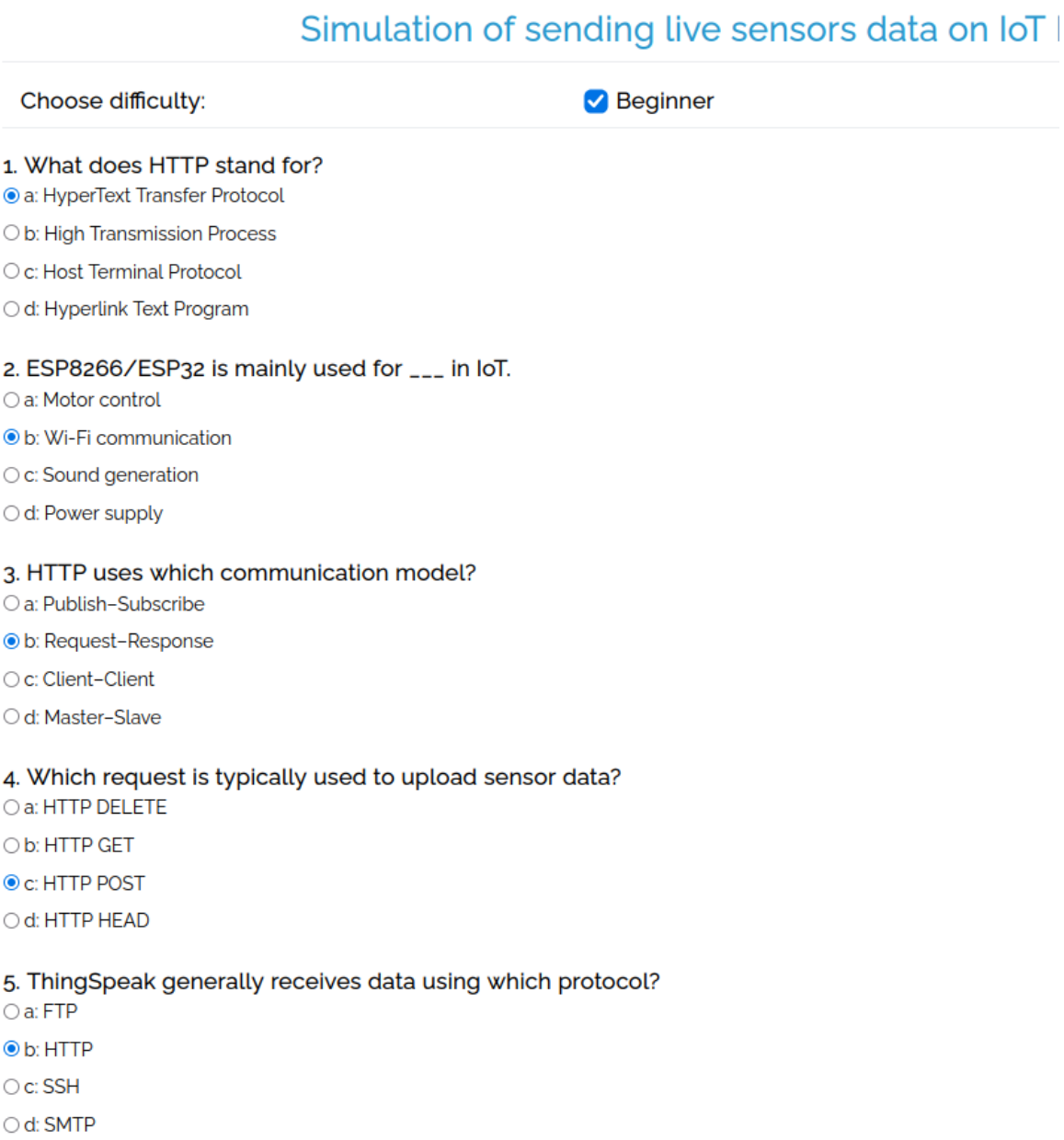


Figure 64: Post-Test Screenshot

Post-Test

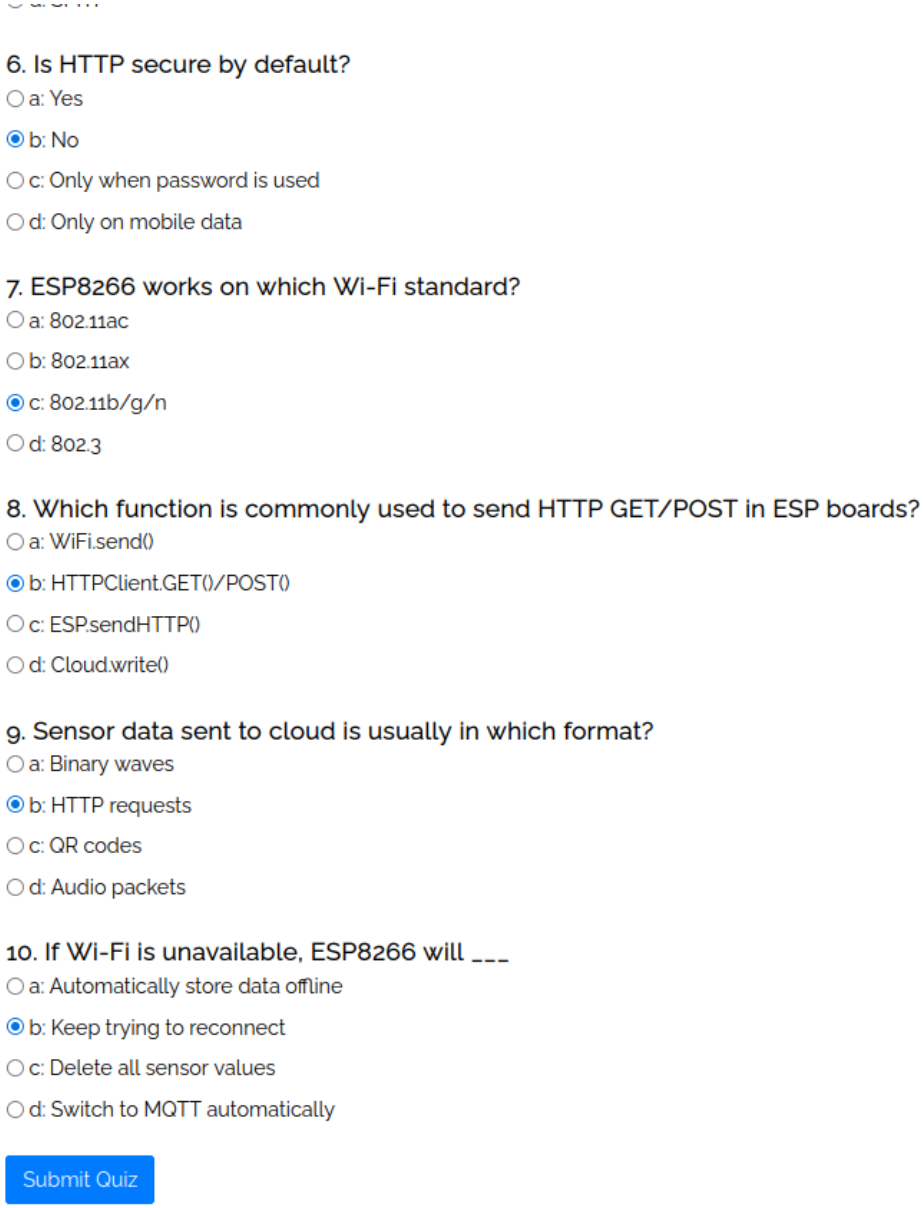


Figure 65: Post-Test Screenshot

Conclusion

In this experiment, we used an ESP32 with a DHT sensor to read temperature and humidity and send them to a web server using HTTP GET/POST requests. We saw the live values update on the web dashboard and learned how Wi-Fi connection, HTTP requests and server responses work together. This shows how HTTP can be used to push sensor data to a cloud platform for remote monitoring.

Experiment 6: Sending Live Sensor Data to an IoT Platform using MQTT

Aim

To simulate lightweight IoT communication using MQTT, where ESP8266/ESP32 publishes sensor data to topics through a broker. The experiment aims to understand broker-based messaging and its advantages in real-time IoT systems.

Pre-Test

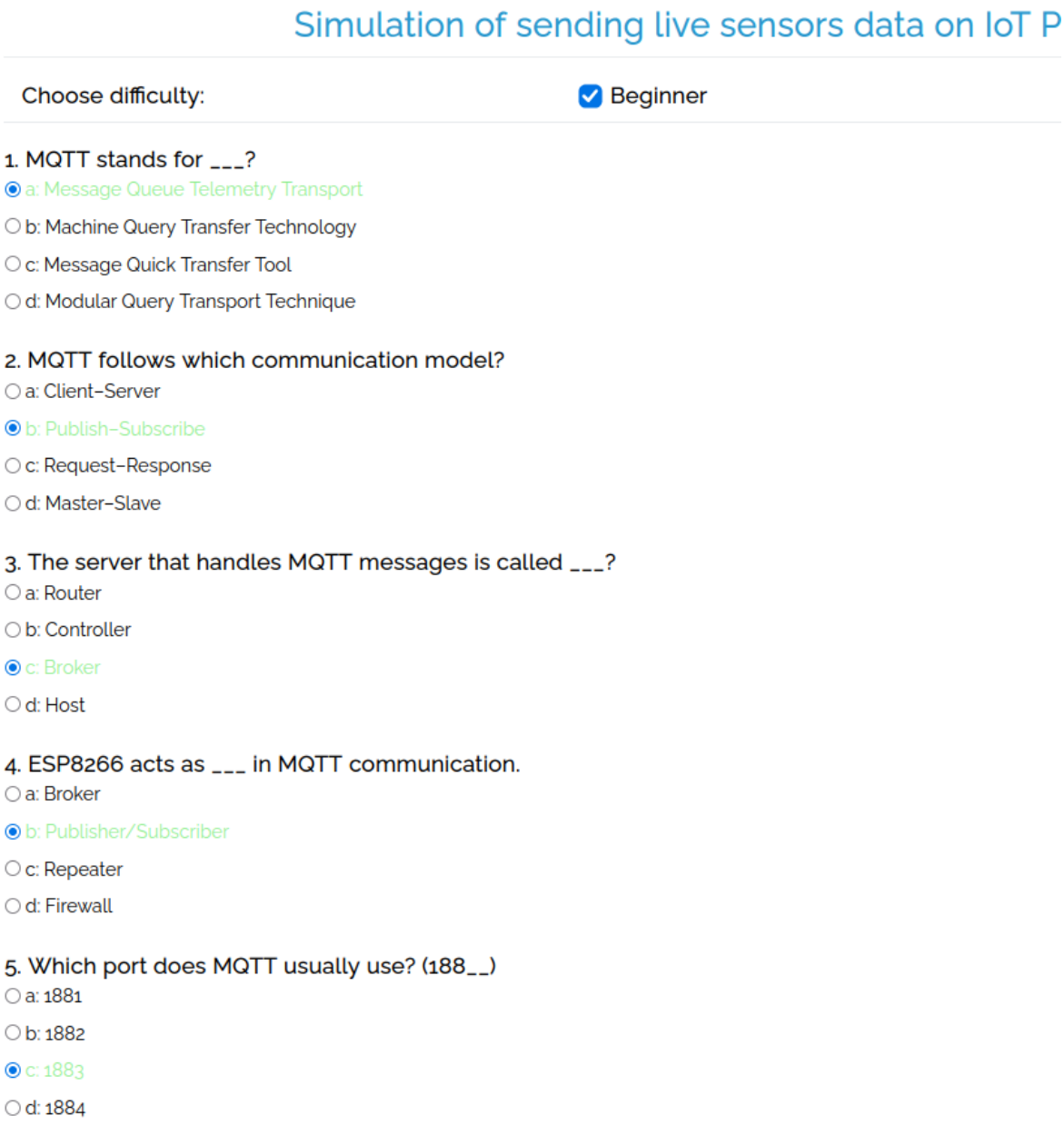


Figure 66: Pre-Test Screenshot

Pre-Test

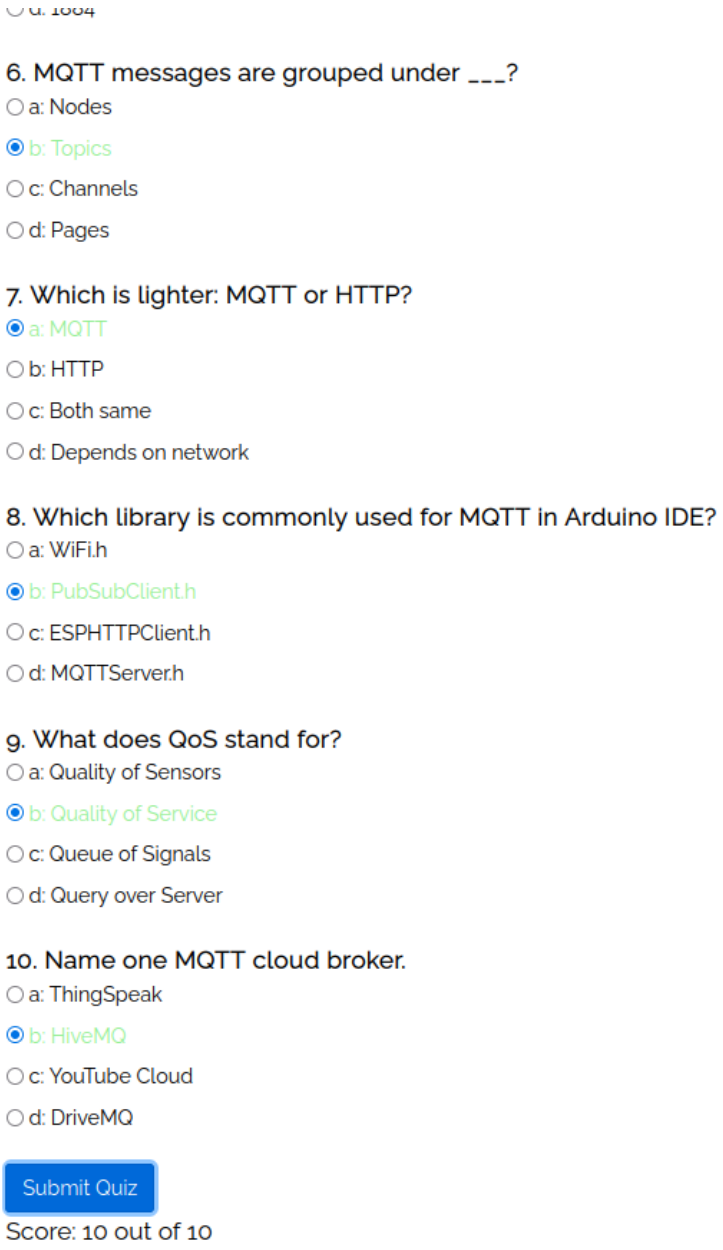


Figure 67: Pre-Test Screenshot

Experiment Performed on Virtual Lab

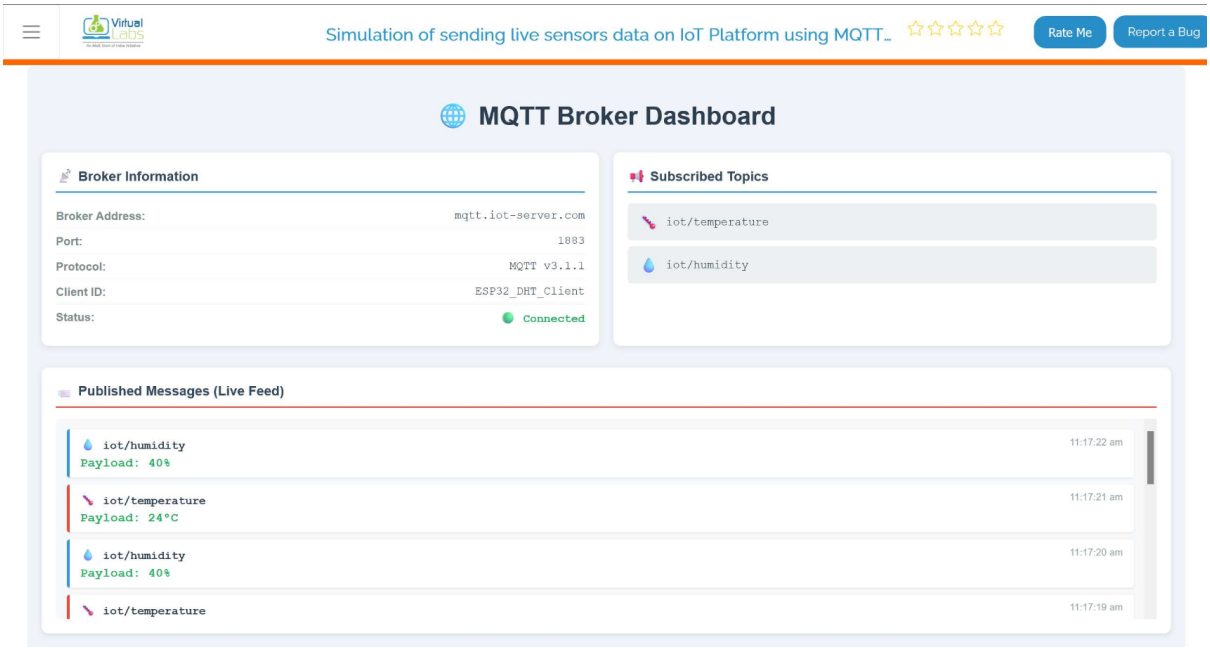


Figure 68: MQTT broker Dashboard



Figure 69: Statistics

Post-Test

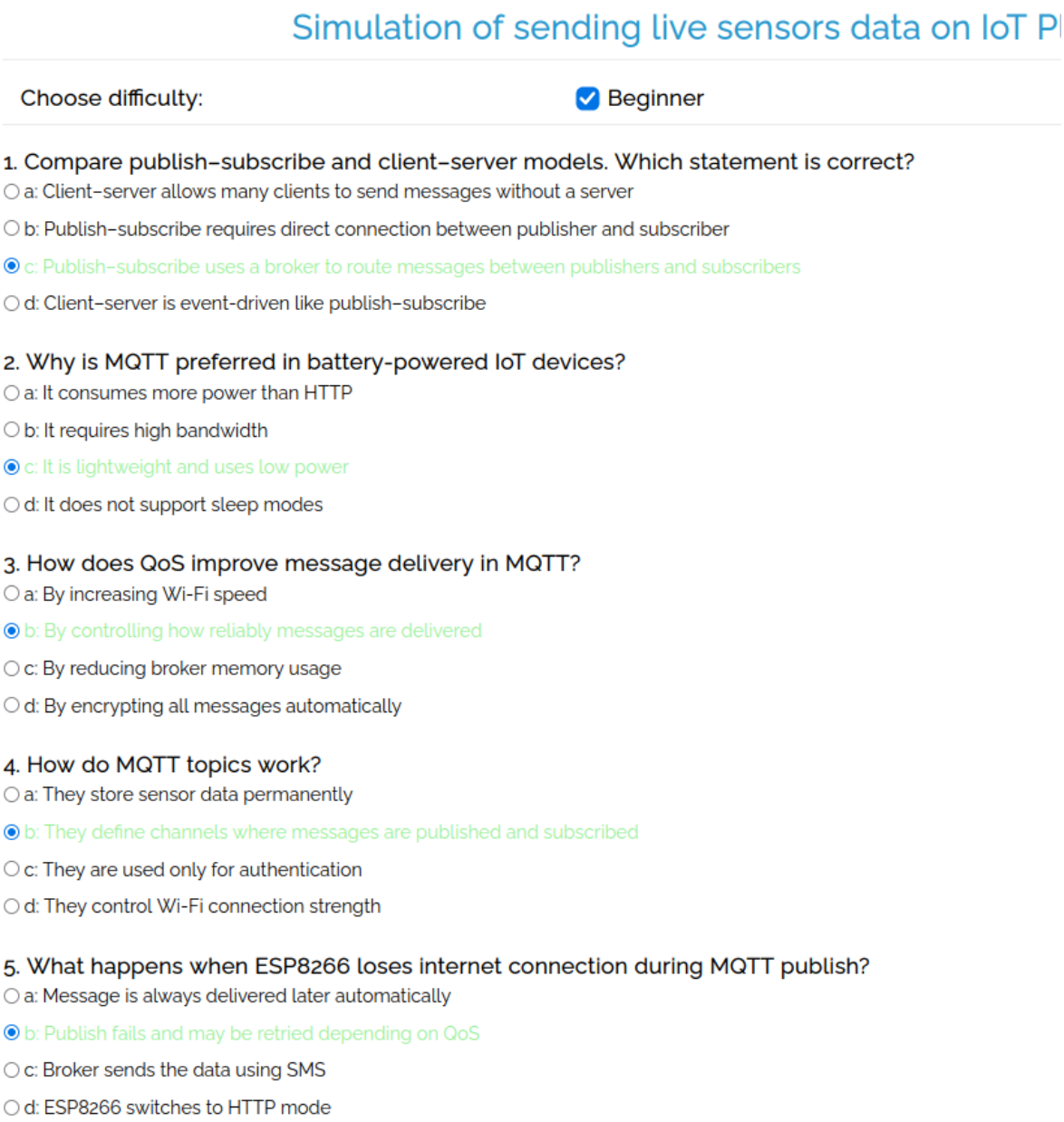


Figure 70: Post-Test Screenshot

Post-Test

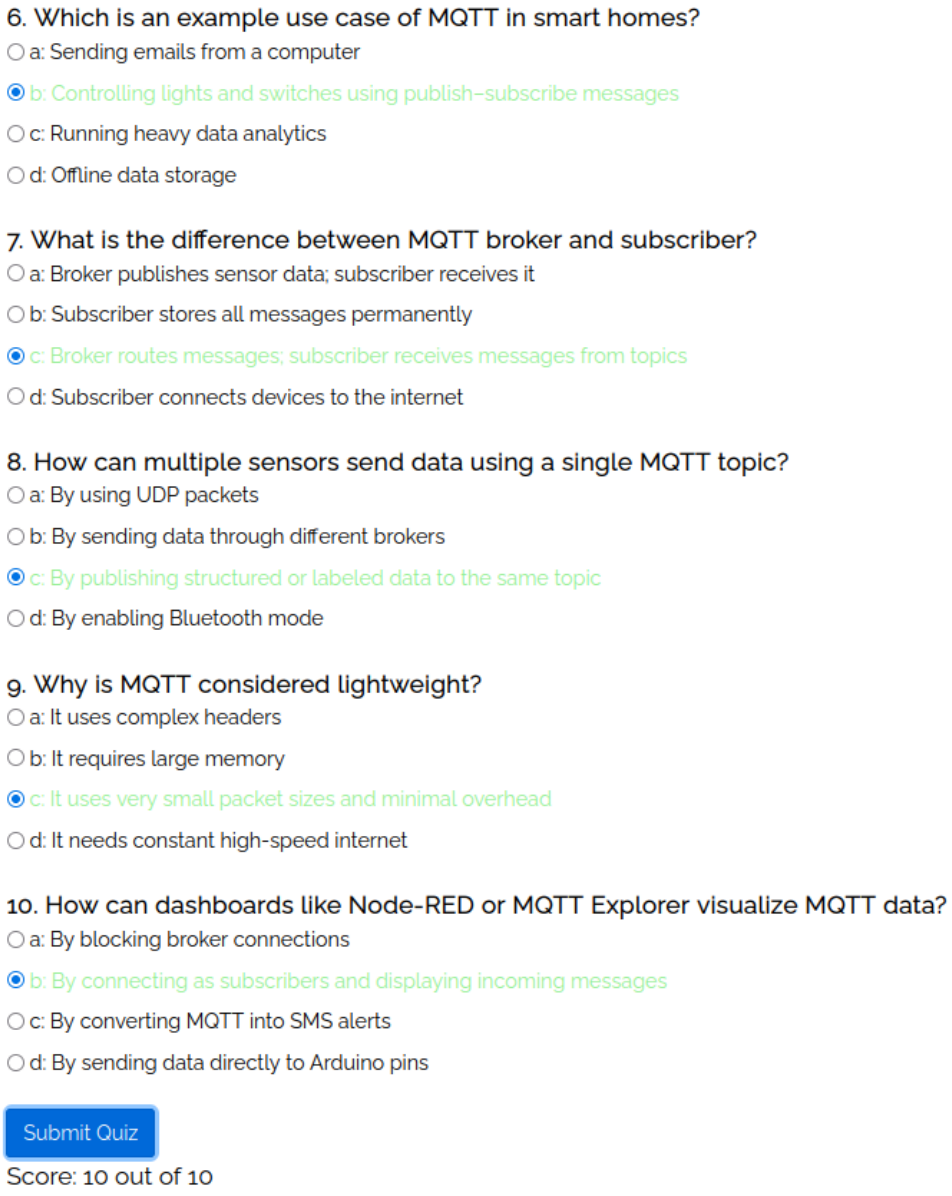


Figure 71: Post-Test Screenshot

Conclusion

In this experiment, we used an ESP32 with a DHT sensor to publish temperature and humidity values to an MQTT topic and watched them update live on the MQTT dashboard. We learned how the publish–subscribe model, the broker and topics work. MQTT is lightweight and reliable, which makes it well suited to low-power IoT devices with limited bandwidth.

Experiment 7: Master–Slave Model using Arduino and I2C

Aim

To simulate communication between multiple Arduino boards using the I2C protocol in a master–slave architecture. The experiment helps in understanding synchronized data transfer and multi-device control in embedded systems.

Pre-Test

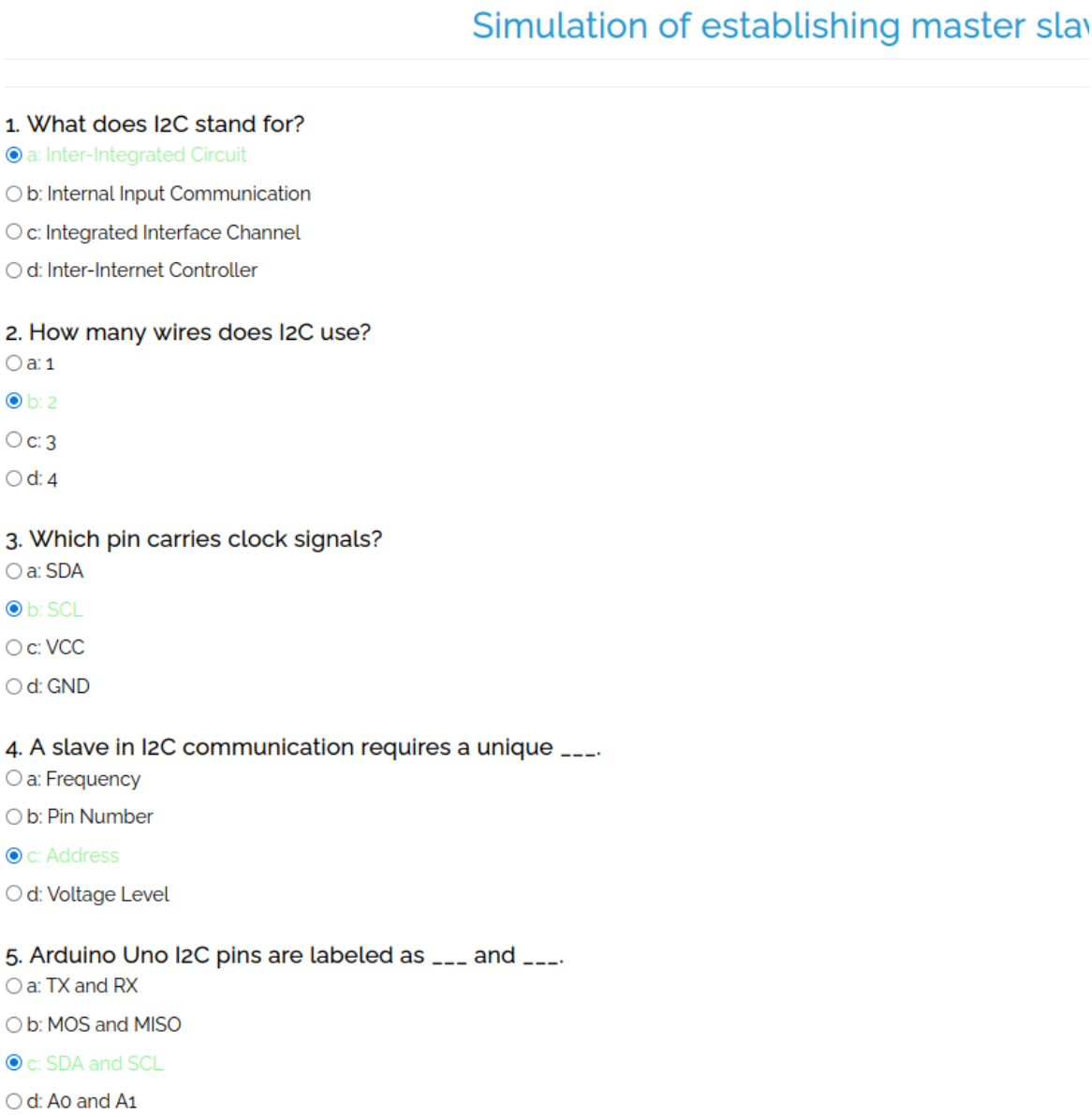


Figure 72: Pre-Test Screenshot

Pre-Test

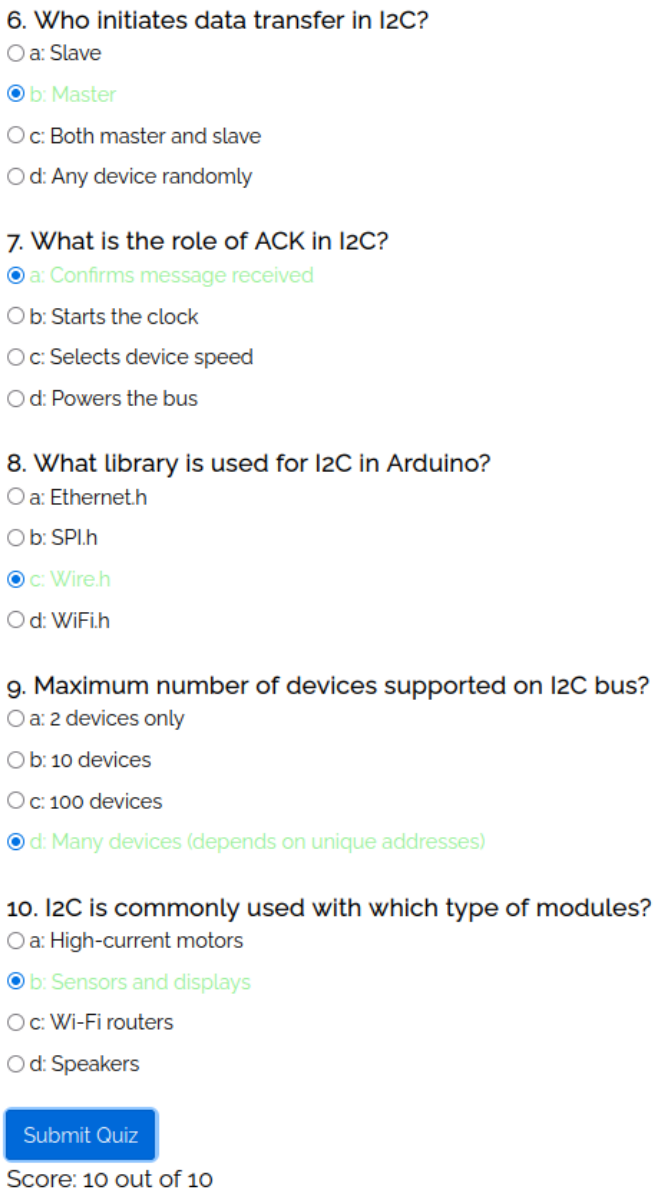


Figure 73: Pre-Test Screenshot

Experiment Performed on Virtual Lab

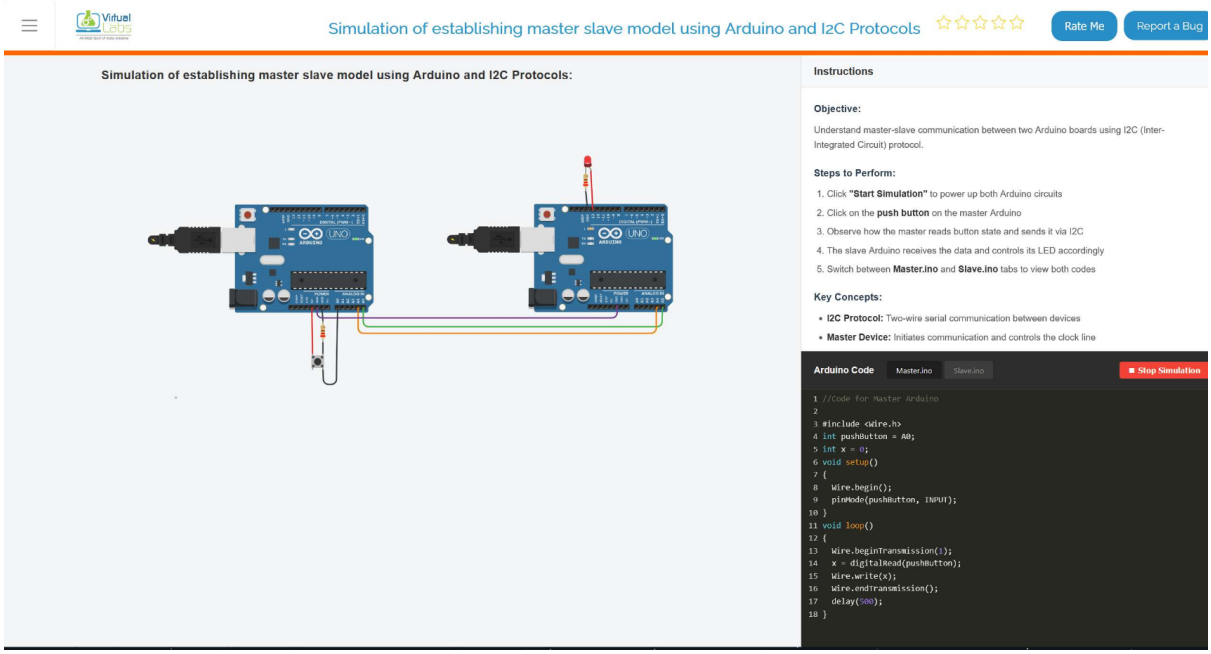


Figure 74: Simulation of master slave Model using arduino and I2C protocols

Post-Test

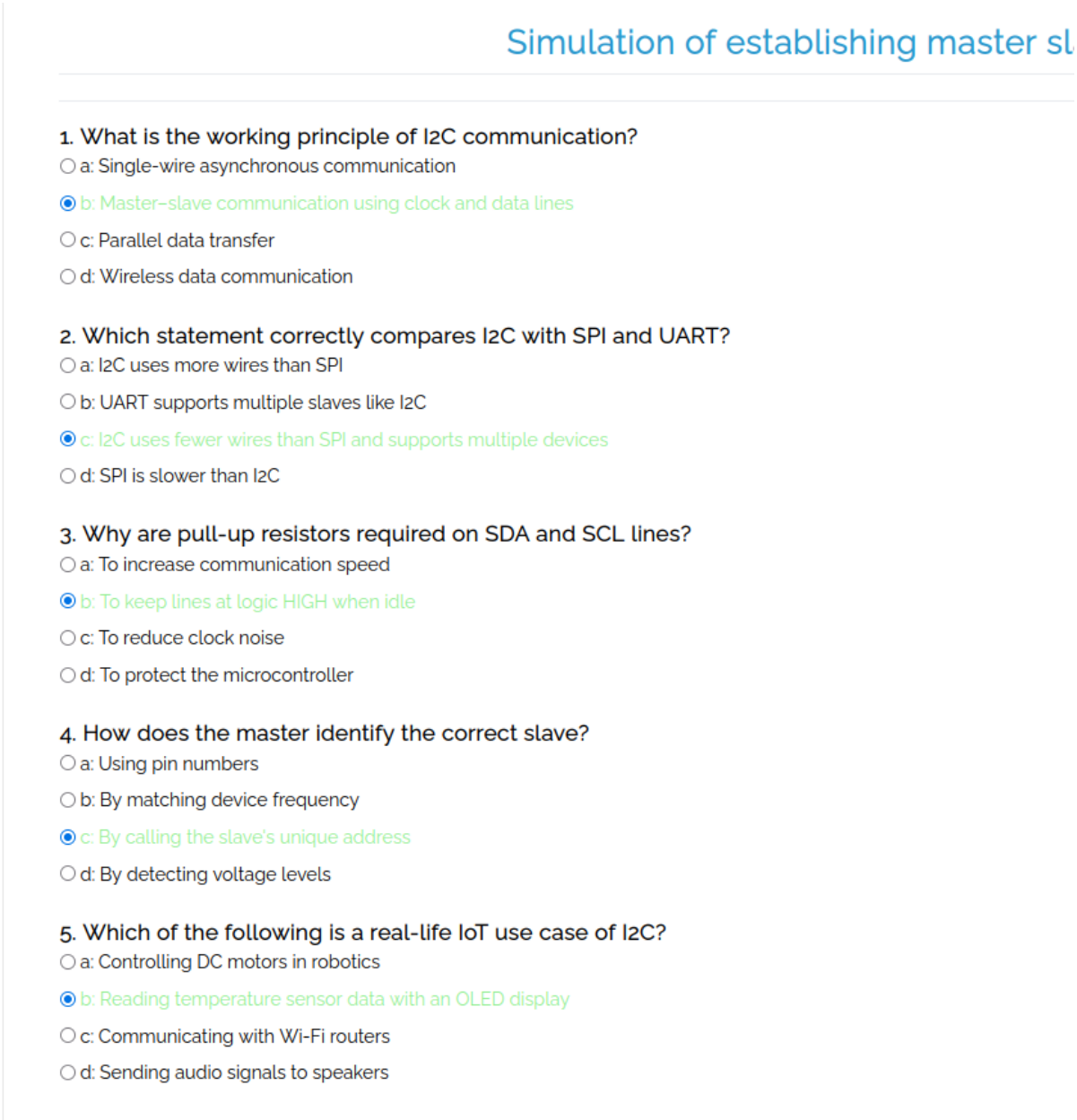


Figure 75: Post-Test Screenshot

Post-Test

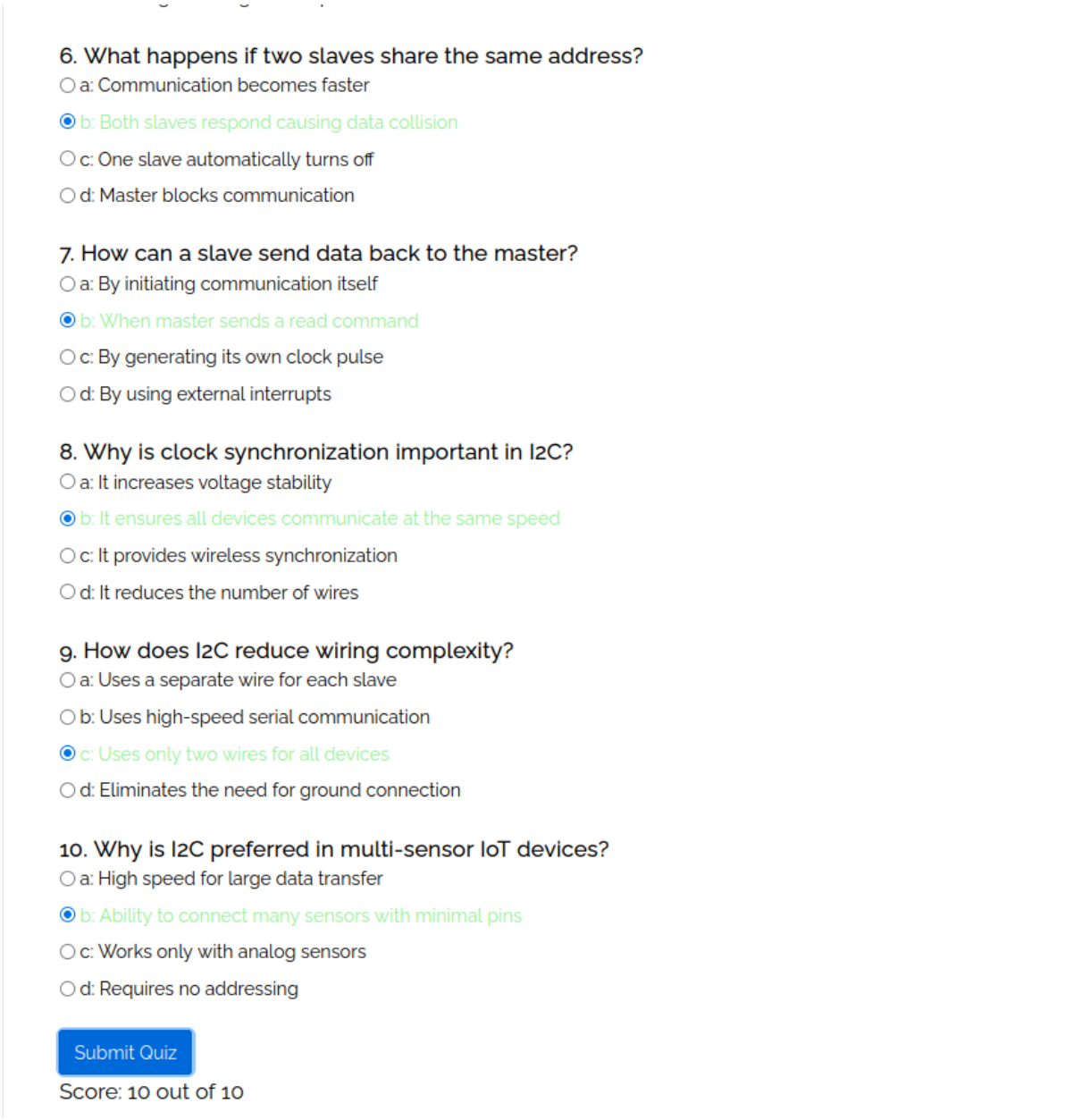


Figure 76: Post-Test Screenshot

Conclusion

In this experiment, we connected two Arduino boards over the I2C bus (SDA and SCL) as master and slave. Pressing the pushbutton on the master sent a command over I2C, and the slave turned its LED ON or OFF accordingly. This showed how I2C lets one master control slave devices using just two wires and unique addresses.

Experiment 8: Interrelation between Cloud Platforms and ESP8266/ESP32

Aim

To simulate and understand the interrelation between IoT cloud platforms (AWS, Microsoft Azure, ThingSpeak and IBM Bluemix) and ESP8266/ESP32 microcontrollers. The experiment shows how an ESP device sends sensor data to the cloud for storage, processing and visualization.

Pre-Test

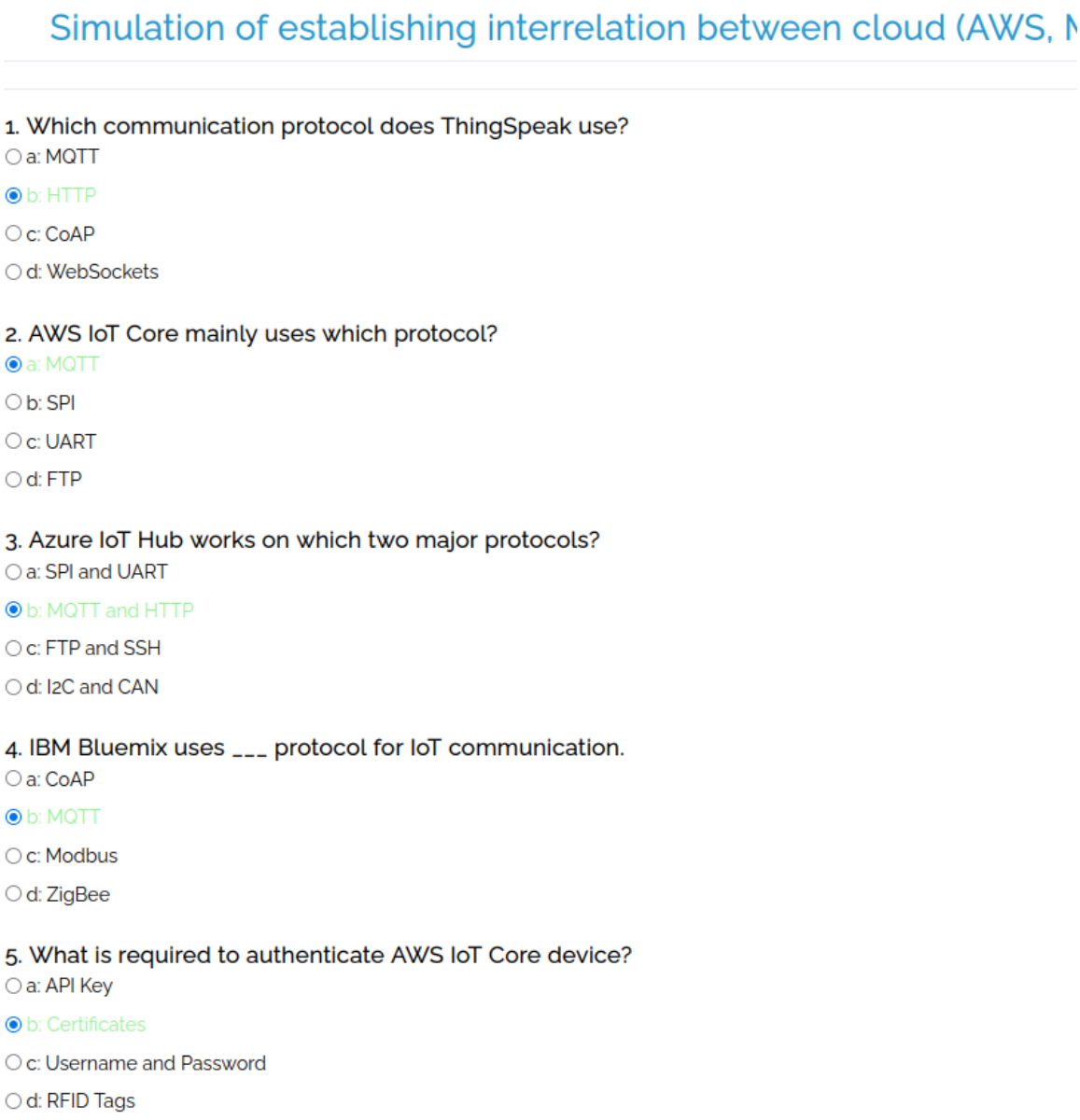


Figure 77: Pre-Test Screenshot

Pre-Test

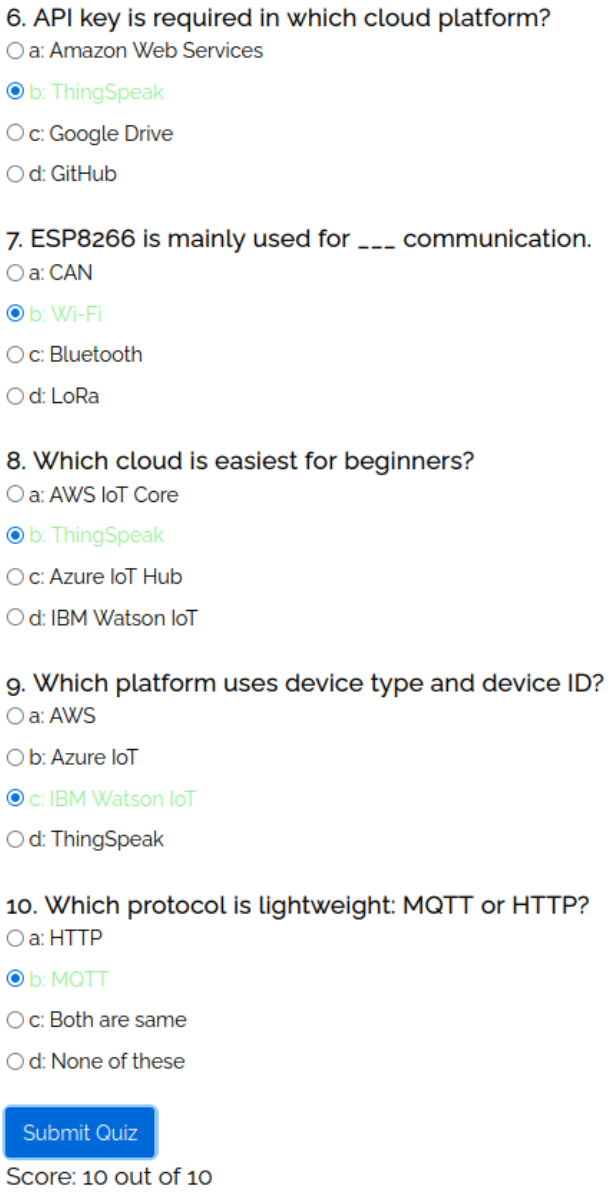


Figure 78: Pre-Test Screenshot

Experiment Performed on Virtual Lab

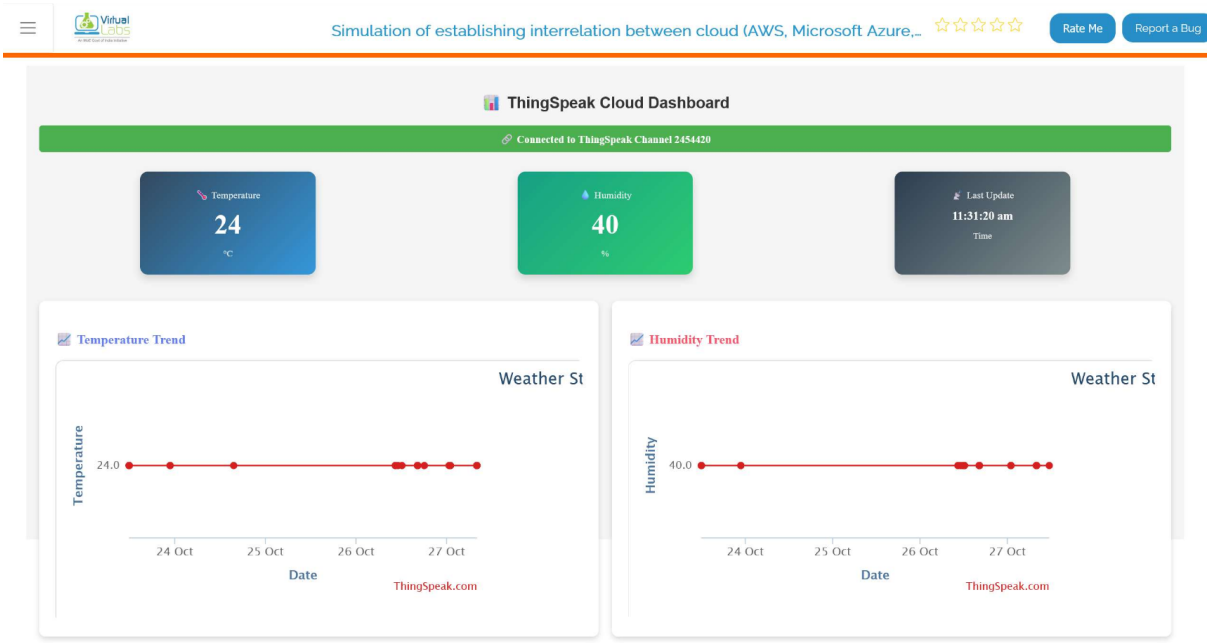


Figure 79: Think Cloud Dashboard

Post-Test

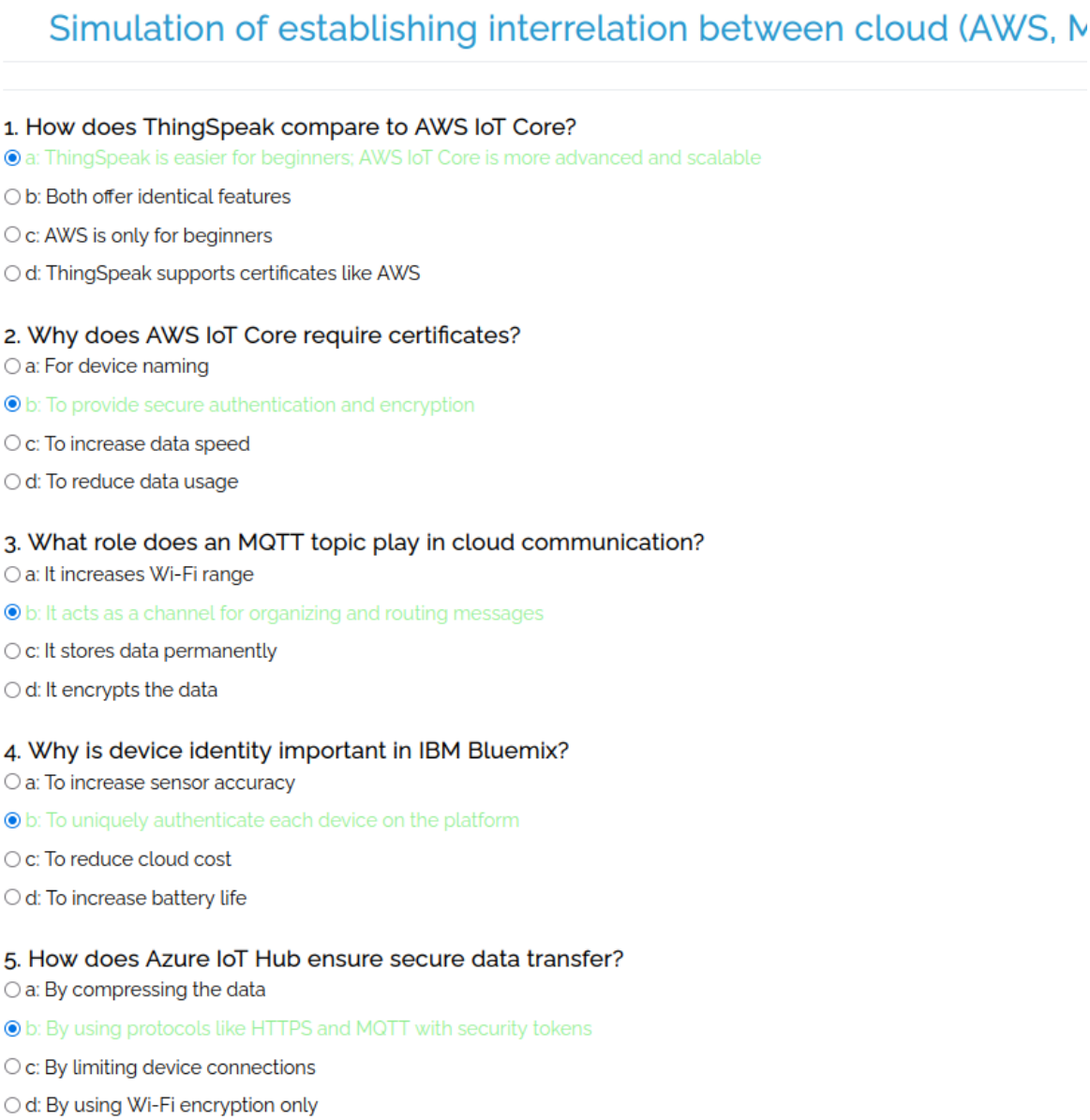


Figure 80: Post-Test Screenshot

Post-Test

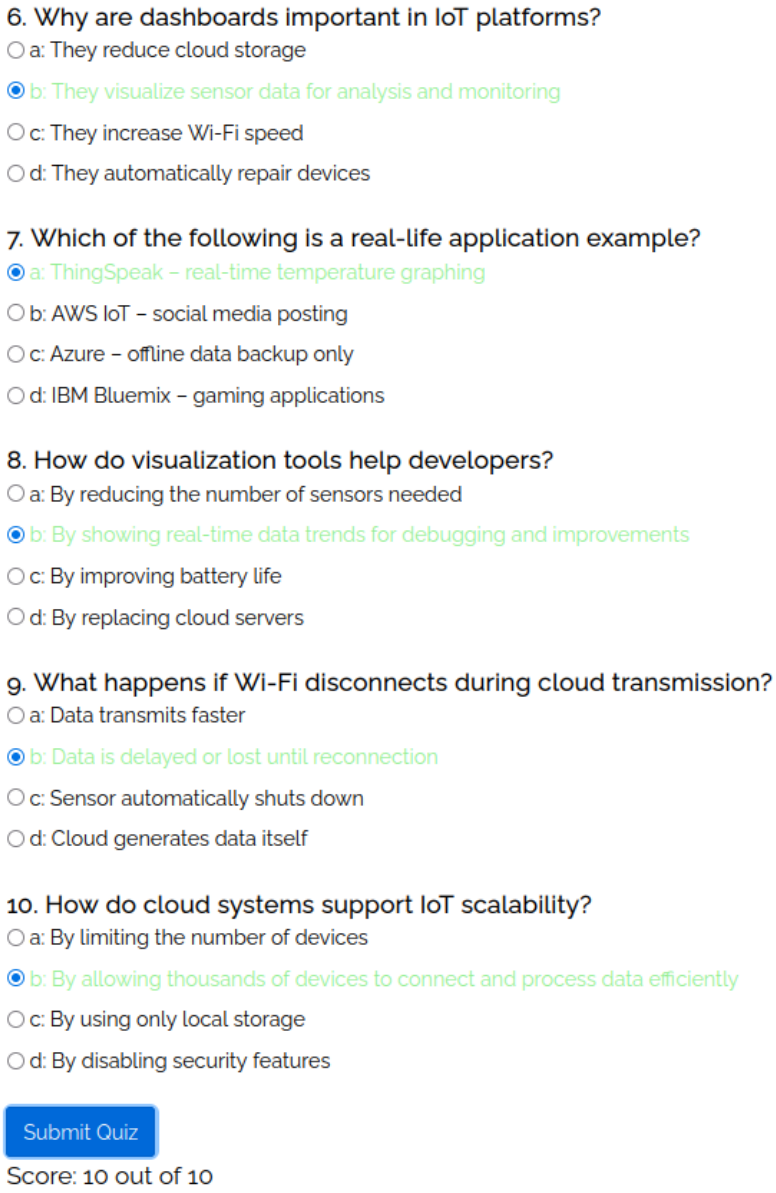


Figure 81: Post-Test Screenshot

Conclusion

In this experiment, we used an ESP32 with a DHT22 sensor to upload temperature and humidity readings to a ThingSpeak channel using an HTTP request and API key. We saw the live graphs update on the ThingSpeak dashboard. This shows how cloud platforms turn a local sensor node into a scalable, remotely accessible IoT system.

Experiment 9: IoT Dashboard for Remote Monitoring

Aim

To design and simulate an IoT dashboard that visualizes live sensor data in real time. The experiment aims to understand dashboard elements such as widgets, charts and APIs, and how they support remote monitoring through cloud or web interfaces.

Pre-Test

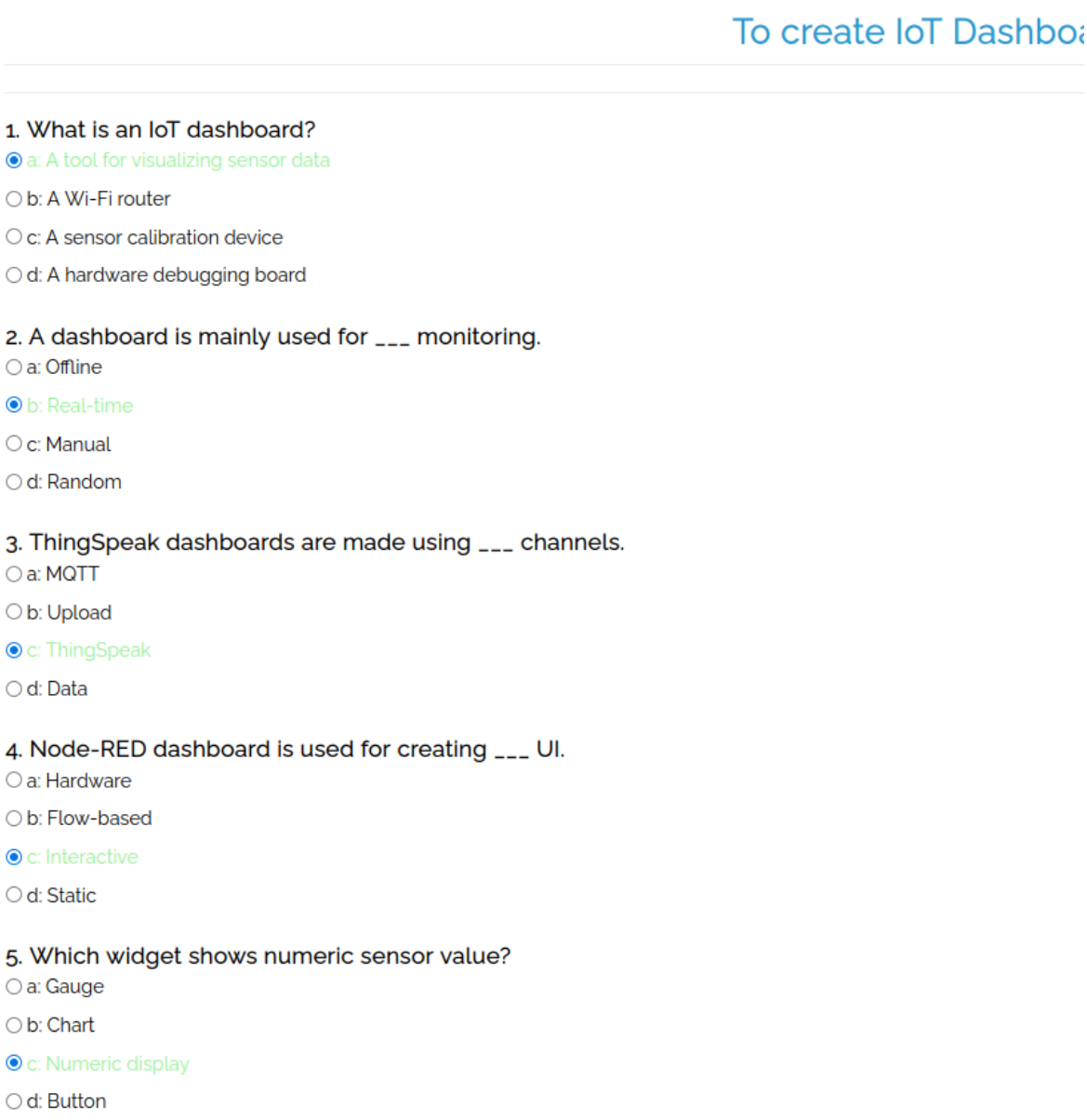


Figure 82: Pre-Test Screenshot

Pre-Test

6. A gauge widget represents data in ___ form.
- (●) a: Circular meter
- ( ) b: Text-only
- ( ) c: Table rows
- ( ) d: Audio tones

7. Which platform uses 'Feeds' for dashboards?
- ( ) a: Node-RED
- ( ) b: AWS IoT
- ( ) c: ThingSpeak
- (●) d: Adafruit IO

8. Power BI is connected with which cloud platform?
- (●) a: Azure
- ( ) b: ThingSpeak
- ( ) c: AWS
- ( ) d: IBM

9. An IoT dashboard requires data from ___.
- (●) a: Sensors
- ( ) b: Speakers
- ( ) c: Motors
- ( ) d: Switches only

10. Which cloud platform is best for beginners?
- ( ) a: AWS
- ( ) b: IBM Watson
- (●) c: ThingSpeak
- ( ) d: Azure

Submit Quiz

Score: 10 out of 10

Figure 83: Pre-Test Screenshot

Experiment Performed on Virtual Lab

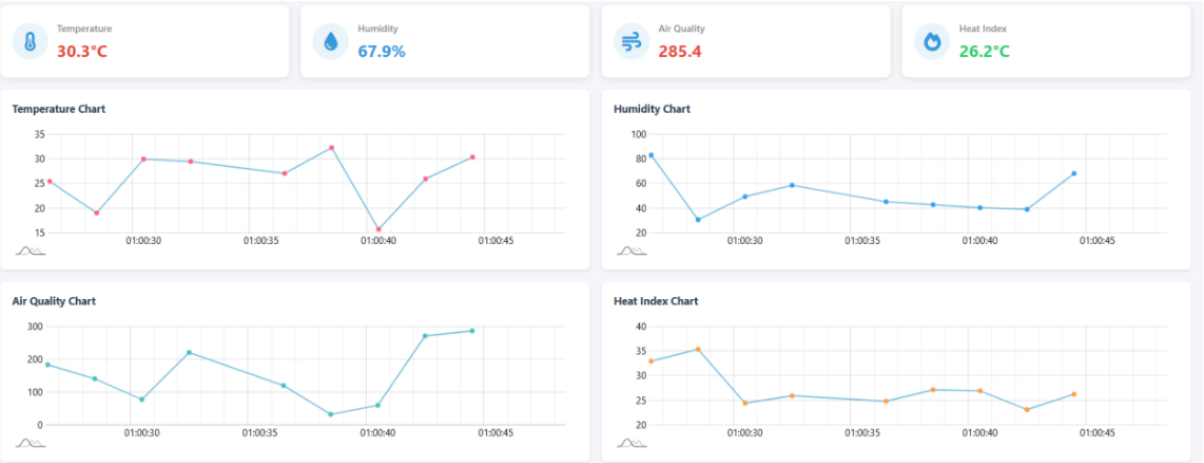


Figure 84: IOT dashboard

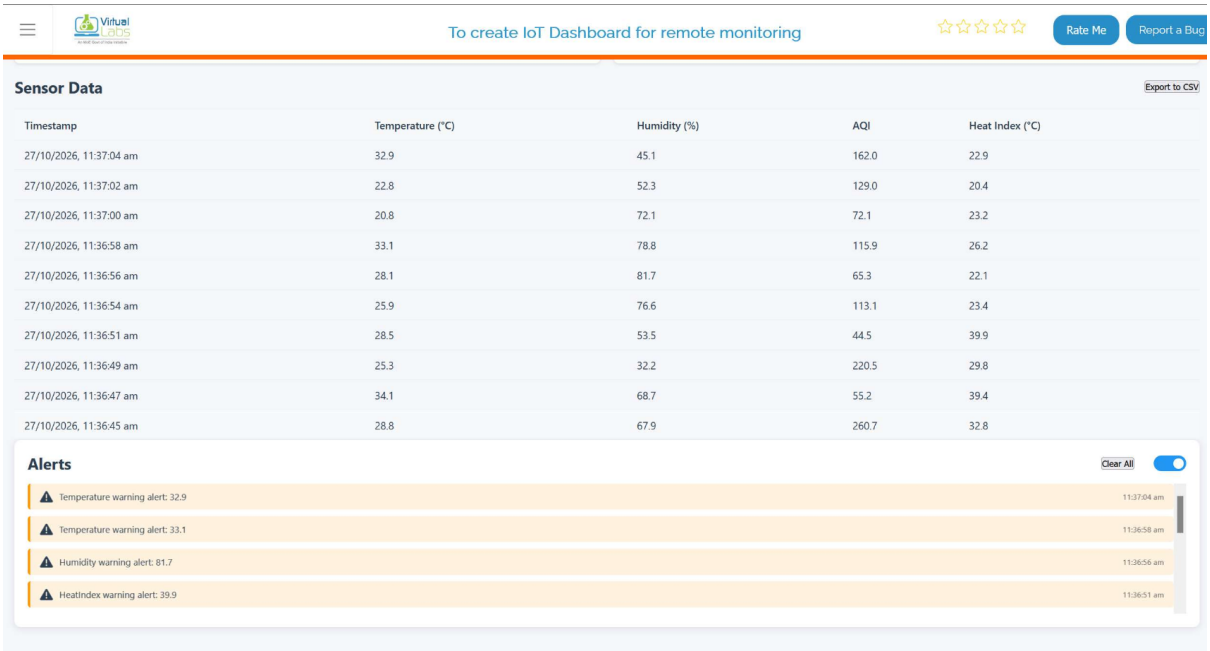


Figure 85: Graphs

Post-Test

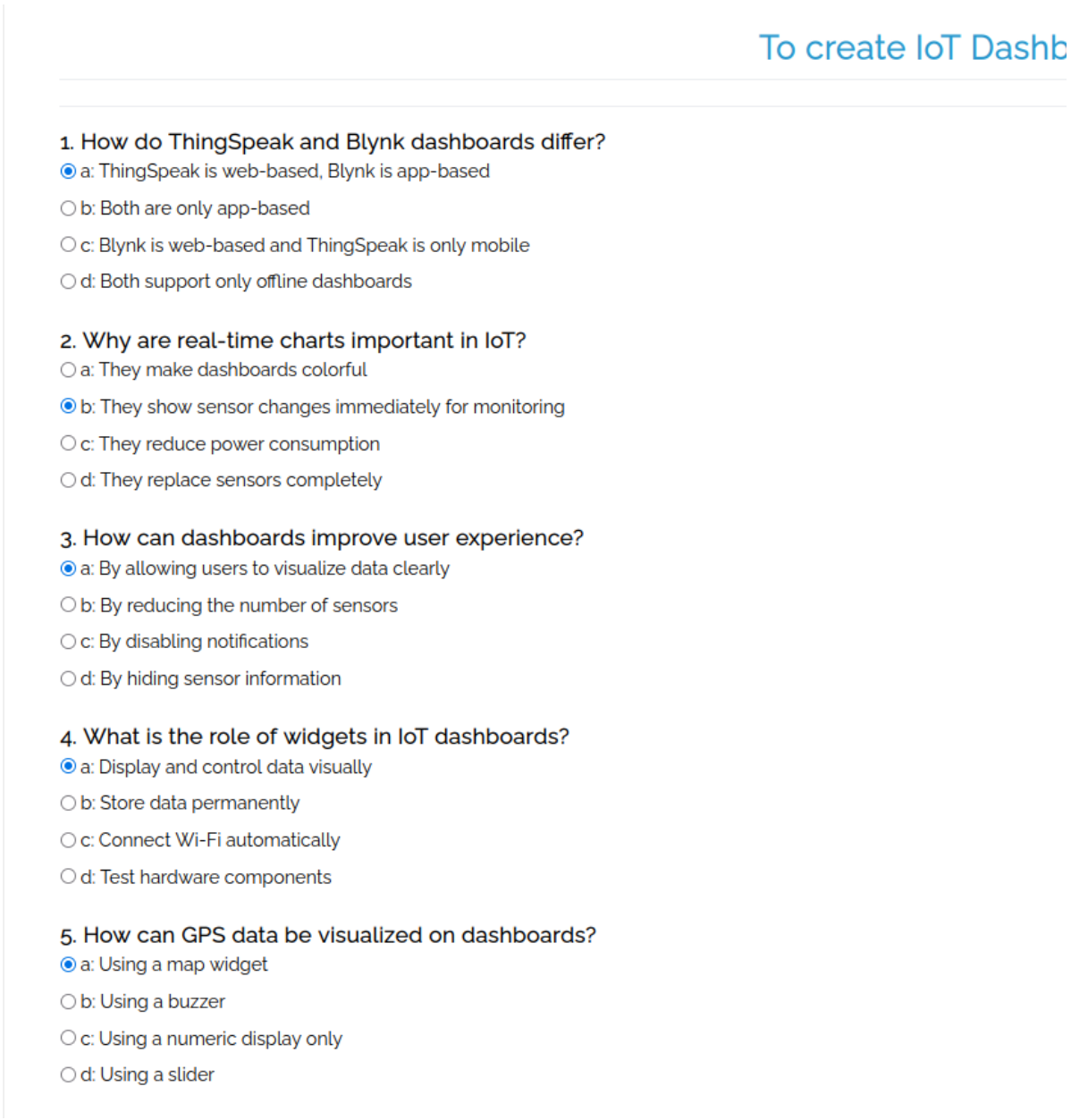


Figure 86: Post-Test Screenshot

Post-Test

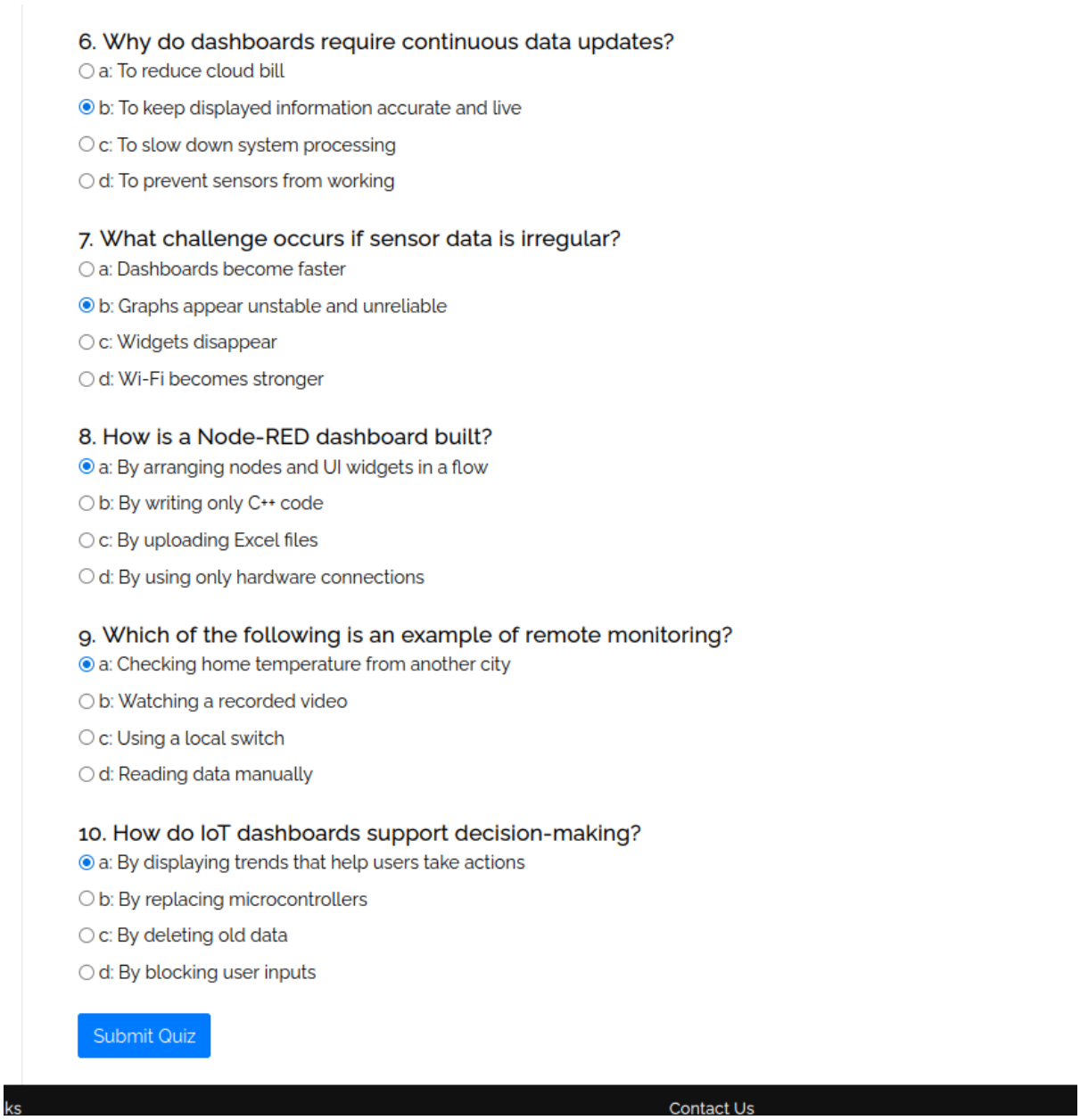


Figure 87: Post-Test Screenshot

Conclusion

In this experiment, we studied an IoT dashboard with charts, gauges and indicators that updated continuously with simulated sensor data. We learned how sensor data flows through the cloud and APIs to the widgets and charts the user sees. Dashboards give real-time visibility and help in decision making, which is useful in smart agriculture, smart homes and industrial monitoring.

Experiment 10: IoT Application using LoRaWAN and ESP8266/ESP32

Aim

Simulate long-range low-power communication using LoRaWAN with ESP8266/ESP32 for IoT applications. The objective is to understand gateway communication, device registration and LPWAN-based data transmission.

Pre-Test

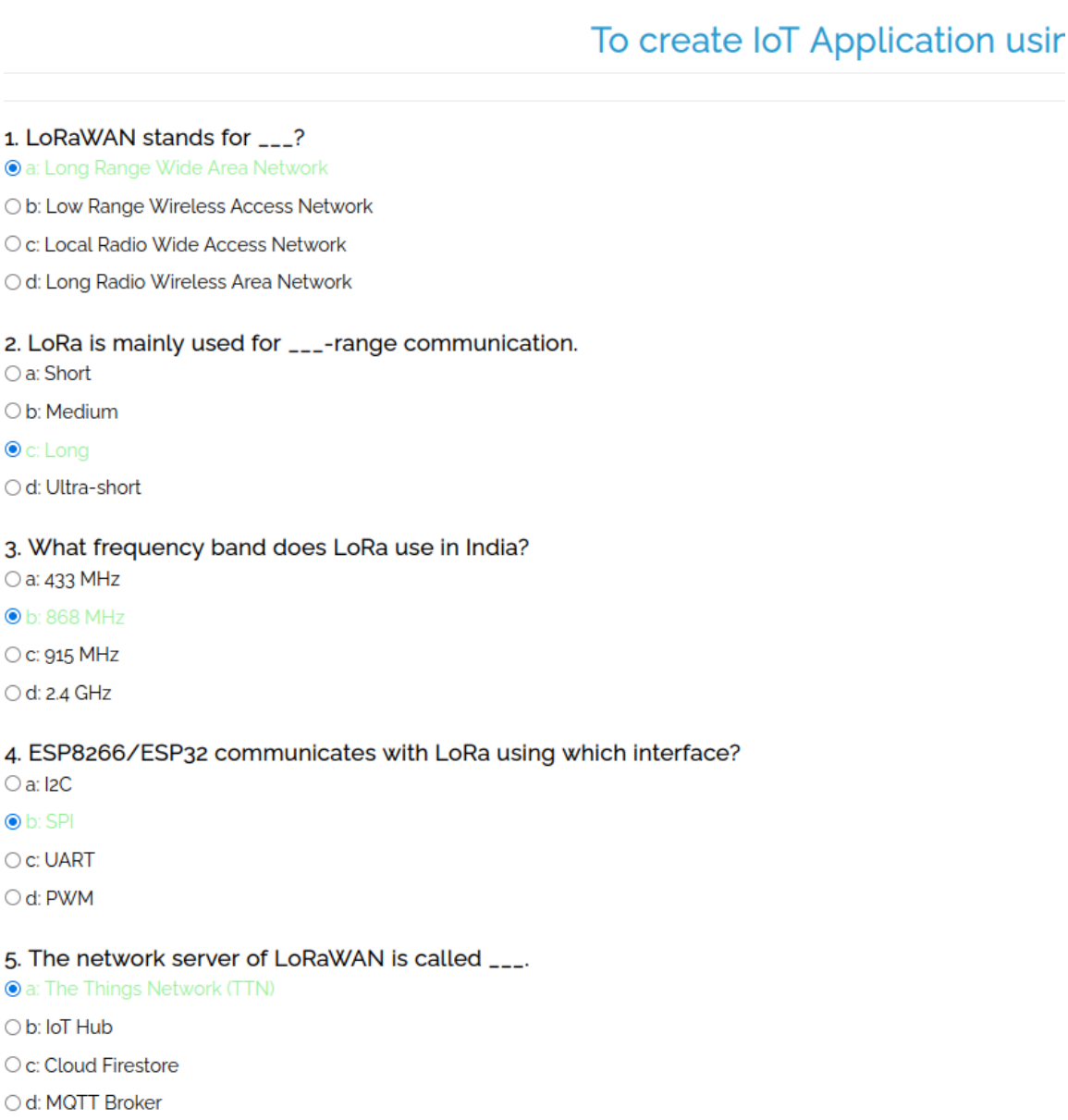


Figure 88: Pre-Test Screenshot

Pre-Test

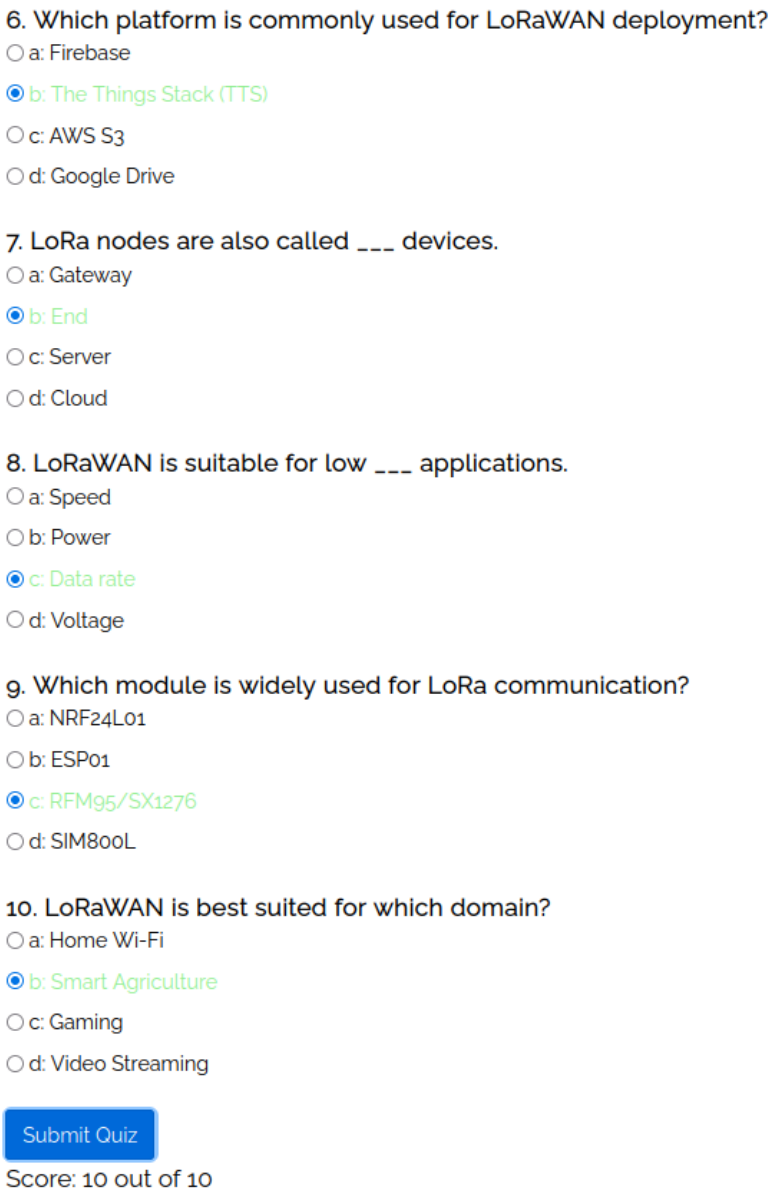


Figure 89: Pre-Test Screenshot

Experiment Performed on Virtual Lab

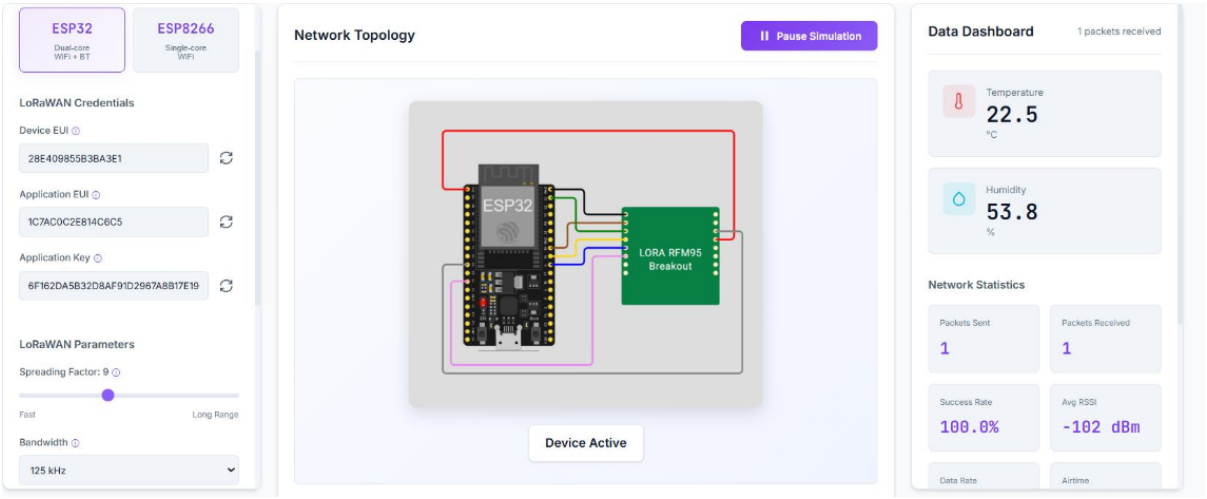


Figure 90: IOT Dashboard

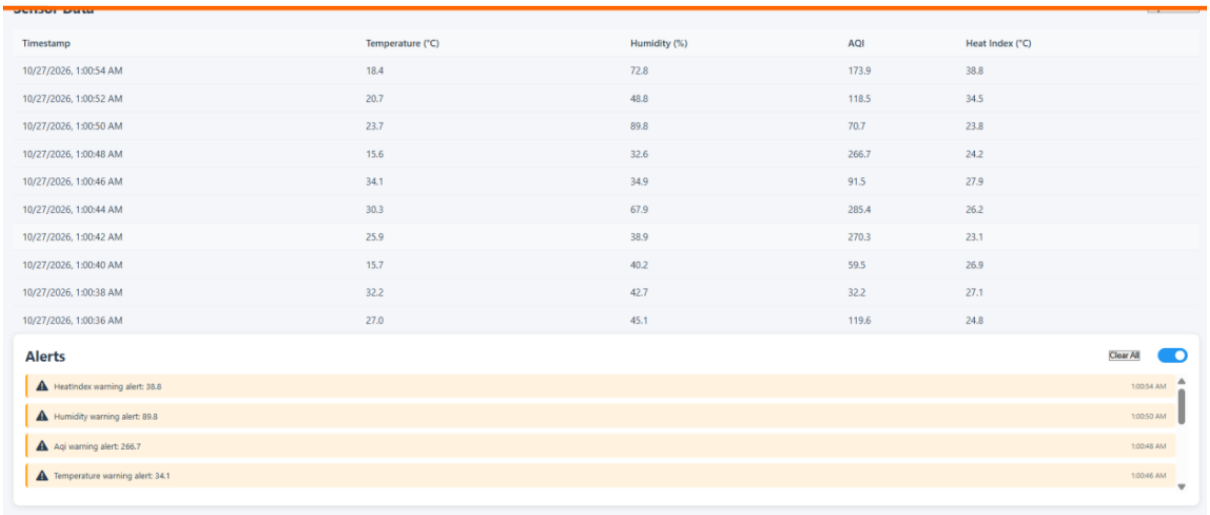


Figure 91: Transmission logs

Post-Test

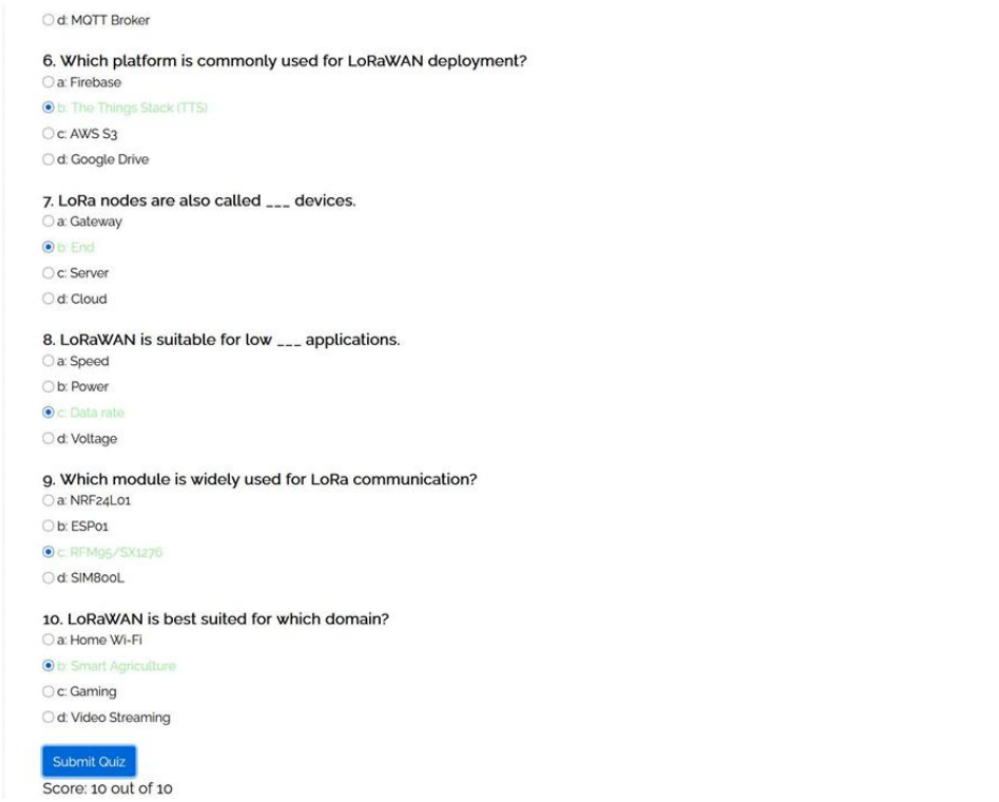


Figure 92: Post-Test Screenshot

Conclusion

In this experiment, we activated a virtual LoRaWAN device, which joined the network and sent simulated sensor packets that updated the dashboard charts. We learned how end nodes, gateways, network servers and application servers work together in a LoRaWAN system. LoRaWAN gives long-range, low-power and secure communication, which suits applications like environmental monitoring and asset tracking.